



UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA Y
DISEÑO INDUSTRIAL

Grado en Ingeniería Electrónica Industrial y Automática

TRABAJO FIN DE GRADO

**DISEÑO E IMPLEMENTACIÓN DE UN
ROBOT MÓVIL AUTÓNOMO CON
VSLAM BASADO EN CÁMARA
RGB-D**

Luis Daniel Alvarez Castillo

Tutor: David Álvarez Sánchez

Departamento: Ingeniería Eléctrica, Electrónica Automática y Física
Aplicada

Madrid, Septiembre, 2025

Copyright © 2025. Luis Daniel Alvarez Castillo

Esta obra está licenciada bajo la licencia Creative Commons

Atribución-NoComercial-SinDerivadas 3.0 Unported (CC BY-NC-ND 3.0). Para ver una copia de esta licencia, visite <http://creativecommons.org/licenses/by-nc-nd/3.0/deed.es> o envíe una carta a Creative Commons, 444 Castro Street, Suite 900, Mountain View, California, 94041, EE.UU.

Todas las opiniones aquí expresadas son del autor, y no reflejan necesariamente las opiniones de la Universidad Politécnica de Madrid.

Agradecimientos

Este Trabajo de Fin de Grado es la culminación de un largo camino de aprendizaje, y sería imposible haberlo recorrido sin la ayuda y el apoyo de muchas personas a quienes hoy quiero dar las gracias.

En primer lugar, a mi tutor, David, por abrirme las puertas a un proyecto tan desafiante y, sobre todo, por confiar en mi capacidad para llevarlo a cabo. Su guía me permitió no solo aplicar mis conocimientos, sino descubrir una nueva pasión por la robótica autónoma que, sin duda, marcará mi futuro profesional. A María Sanz, por su tiempo y por compartir su experiencia en los momentos clave. Su ayuda ha sido fundamental para que este robot cobrara vida.

A mi hermano José Carlos, por ofrecer siempre un punto de vista técnico que me ayudaba a ver los problemas desde otra perspectiva y por ayudarme en toda la fase de diseño. Detrás de cada hora dedicada a este proyecto ha estado siempre el apoyo incondicional de mi esposa, María Eugenia. Su paciencia, su comprensión y sus palabras de aliento han sido fundamentales para que pudiera culminar este trabajo.

A mis padres, cuyo sacrificio y dedicación han sido el pilar sobre el que se ha construido todo mi camino. De ellos aprendí la voluntad, la determinación y los valores que hoy me permiten alcanzar este logro. A mis abuelos, que fueron una parte fundamental en mi crianza y una presencia constante a lo largo de mi vida. El cariño y la sabiduría que me transmitieron me han dado siempre la confianza necesaria para avanzar.

Por último, quiero extender mi gratitud a toda mi familia y a mis amigos. Su cariño y apoyo constantes, aunque a veces desde la distancia, han sido la base sobre la que se construye todo lo demás.

Resumen

Este Trabajo de Fin de Grado aborda el diseño completo, la construcción y la implementación del sistema de software de un robot móvil autónomo de bajo costo. Partiendo de una plataforma de tracción diferencial de cuatro ruedas, se ha desarrollado una arquitectura electrónica y de control jerarquizada, utilizando una Raspberry Pi 5 como unidad de procesamiento de alto nivel y controladoras dedicadas para la gestión de los actuadores.

El sistema de percepción se basa en la fusión de datos de una cámara *Red Green Blue + Depth (RGB-D)* y una Unidad de Medición Inercial (IMU) externa. Sobre esta base, se ha implementado un sistema de *Visual Simultaneous Localization and Mapping (VSLAM)* utilizando el paquete *Real-Time Appearance-Based Mapping (RTAB-Map)* sobre la plataforma *Robot Operating System (ROS) 2*, configurado específicamente para generar mapas de ocupación 2D optimizados para la navegación.

El desarrollo documenta en profundidad los desafíos prácticos encontrados, como la sobrecarga computacional en sistemas embebidos y la crucial sincronización de sensores en arquitecturas distribuidas, detallando las soluciones implementadas. Finalmente, se valida la capacidad del robot para generar mapas métricamente precisos y se configura el *stack* de navegación Navigation 2 (Nav2), sentando las bases para una navegación autónoma completa.

Palabras clave: Robótica Móvil Autónoma, SLAM, VSLAM, ROS 2, RTAB-Map, Navegación Autónoma, Raspberry Pi.

Abstract

This Final Degree Project addresses the complete design, construction, and software system implementation of a low-cost autonomous mobile robot. Starting with a four-wheel differential drive platform, a hierarchical electronic and control architecture has been developed, using a Raspberry Pi 5 as the high-level processing unit and dedicated controllers for actuator management.

The perception system is based on the fusion of data from an RGB-D camera and an external IMU. On top of this foundation, a VSLAM system has been implemented using the ROS 2 package RTAB-Map, specifically configured to generate 2D occupancy grid maps optimized for navigation.

The development thoroughly documents the practical challenges encountered, such as computational overload on embedded systems and the critical synchronization of sensors in distributed architectures, detailing the implemented solutions. Finally, the robot's ability to generate metrically accurate maps is validated, and the Nav2 navigation *stack* is configured, laying the groundwork for complete autonomous navigation.

Keywords: Autonomous Mobile Robotics, SLAM, VSLAM, ROS 2, RTAB-Map, Autonomous Navigation, Raspberry Pi.

Lista de abreviaturas

AMCL Adaptive Monte Carlo Localization

API Application Programming Interface

BoW Bag of Visual Words

CPU Unidad de Procesamiento Central

DC Corriente Continua

DDS Data Distribution Service

DWA Dynamic Window Approach

IMU Unidad de Medición Inercial

LiDAR Light Detection and Ranging

LiPo Lithium Polymer

Nav2 Navigation 2

PCB Placa de Circuito Impreso

PID Proporcional Integral Derivativo

PIV Proportional-Integral with Velocity Feedforward

pps Pulsos por Segundo

ppr Pulsos por Revolución

RGB-D Red Green Blue + Depth

ROS Robot Operating System

RPi5 Raspberry Pi 5

RTAB-Map Real-Time Appearance-Based Mapping

SLAM Simultaneous Localization and Mapping

SSH Secure Shell

TF2 Transform Library v2

TFG Trabajo Fin de Grado

USB Universal Serial Bus

VISLAM Visual Inertial SLAM

VIO Odometría Visual-Inercial

VPU Unidad de Procesamiento de Visión

VSLAM Visual Simultaneous Localization and Mapping

Índice general

Agradecimientos	IV
Resumen	V
Abstract	VI
Lista de abreviaturas	VII
Índice	XI
1. Introducción	1
1.1. Motivación del proyecto	1
1.2. Objetivos	1
1.2.1. Objetivo Principal	1
1.2.2. Objetivos Secundarios	2
1.3. Materiales utilizados	2
1.4. Estructura del documento	3
2. Estado de la Técnica	4
2.1. Robots Móviles	4
2.2. Robots Móviles con Tracción Diferencial	5
2.3. SLAM: Localización y Mapeo Simultáneo	5
2.3.1. RTAB-Map: Ejemplo de Solución Integral de Graph-SLAM Basado en Apariencia	6
2.4. Navegación Autónoma: Localización y Planificación de Rutas	7
3. Marco Teórico	9
3.1. Cinemática del Robot móvil diferencial	9
3.2. Sensores y actuadores para la Robótica Móvil Autónoma	10
3.2.1. Sensores para Percepción Visual-Inercial	11
3.2.2. Actuación y Propiocepción	12
3.3. RTAB-Map	12
3.3.1. Detección de Cierres de Bucle mediante <i>Bag-of-Words</i>	12
3.3.2. Gestión de Memoria y Optimización del Grafo	13
3.3.3. Generación de Mapas y Modo de Localización	13
3.4. Navegación Autónoma con Nav2	13
3.4.1. Mapas de Costos para la Evitación de Obstáculos	13
3.4.2. Planificación Global y Local	14
3.4.3. Árboles de Comportamiento para la Lógica de Navegación	14
4. Diseño y Construcción de la Plataforma Robótica Móvil	15
4.1. Descripción General del Sistema	15
4.1.1. Plataforma Base: Chasis goBILDA Recon	16
4.1.2. Sistema de Actuación y Propiocepción	16
4.1.3. Unidades de Procesamiento	18

4.1.4.	Unidad de Percepción Visual-Inercial	21
4.1.5.	Comunicación	23
4.1.6.	Sistema de Alimentación	24
4.1.7.	Placa de Interfaz Personalizada para Controladores	27
4.2.	Ensamblaje y Disposición de Componentes	28
5.	Arquitectura de Software	31
5.1.	Conceptos Fundamentales de ROS 2	31
5.2.	Árbol de Transformaciones	32
5.3.	Configuración del Espacio de Trabajo (<i>Workspace</i>)	33
5.4.	Arquitectura de Bajo Nivel	34
5.4.1.	Modos de Operación	36
5.5.	Configuración del Pipeline de Percepción de la Cámara OAK-D Lite	37
5.5.1.	Nodos y tópicos clave del pipeline de percepción	37
6.	Control de Alto Nivel: Mapeo y Navegación	39
6.1.	Mapeo y Localización con RTAB-Map	39
6.1.1.	Arquitectura de nodos durante el mapeo.	39
6.1.2.	Configuración de RTAB-Map para Mapeo 2.5D	40
6.1.3.	Modo de Localización Pura	42
6.2.	Navegación Autónoma con Nav2	42
6.2.1.	Arquitectura del Sistema de Navegación	43
6.2.2.	Configuración de Parámetros de Nav2	43
7.	Pruebas, Resultados y Discusión	45
7.1.	Metodología y Entorno de Pruebas	45
7.2.	Pruebas de Bajo Nivel	45
7.2.1.	Metodología de Prueba: Control por Coordenadas	45
7.2.2.	Resultados Experimentales	46
7.2.3.	Análisis y Discusión de Resultados	47
7.3.	Desafíos de la Odometría y Fusión de Sensores	47
7.3.1.	Pruebas con Fusión de Encoders e IMU	47
7.4.	Implementación y Optimización del SLAM Visual-Inercial (VIO)	48
7.4.1.	Sobrecarga Computacional y Pérdida de Seguimiento	48
7.4.2.	Filtrado de la Nube de Puntos para Mapeo 2D	51
7.4.3.	Pruebas de Estabilidad	52
7.4.4.	Resultado del Mapeo Final	55
7.5.	Pruebas de Localización Autónoma	56
7.5.1.	Análisis del Proceso de Localización	56
7.5.2.	Análisis de los Datos de Diagnóstico	56
7.5.3.	Discusión de los Resultados de Localización	57
7.5.4.	Conclusión sobre la Localización Pura	58
7.6.	Pruebas de Localización con AMCL y LaserScan	58
7.6.1.	Generación de LaserScan Sintético	58
7.6.2.	Discusión de Resultados y Problemas de Sincronización	58
7.7.	Pruebas de Navegación Autónoma con Nav2	59
7.7.1.	Configuración y Puesta en Marcha del Sistema de Navegación	59
7.7.2.	Discusión de Resultados: Fallo en la Localización	60
8.	Gestión del Proyecto	61
8.1.	Planificación y Ciclo de Vida	61
8.2.	Presupuesto	62
8.2.1.	Coste de Personal	62
8.2.2.	Coste de Material	63
8.2.3.	Resumen de Costes	63

<i>ÍNDICE GENERAL</i>	XI
9. Conclusiones	64
9.1. Conclusión	64
9.2. Desarrollos Futuros	65
A. Anexo 1: Diseño de Circuito y PCB	66
B. Anexo 2: Diseño de Piezas Mecánicas	69
Bibliografía	73

Índice de figuras

2.1. Configuraciones de los RMR. Imagen adaptada de [38]	5
3.1. Cinemática cuerpo libre. Imagen adaptada de [23]	9
4.1. Diagrama de bloques de la arquitectura general del sistema. Flujos de datos senso- riales, comandos de control e interacción entre procesador y periféricos.	15
4.2. Vista del chasis goBilda Recon utilizado como base de la plataforma.[15]	16
4.3. Vista general y dimensiones del motor goBILDA 5203 Series Yellow Jacket con encoder integrado. Imágenes adaptadas de [14].	18
4.4. Vista superior de la Raspberry Pi 5 con identificación de componentes claves. Imagen adaptada de [9].	18
4.5. Controlador de motores inteligente RoboClaw 2x7A. Imagen adaptada de [5].	19
4.6. Captura de pantalla de la configuración de las controladoras RoboClaw en el software Motion Studio.	20
4.7. Cámara OAK-D-Lite. Imagen adaptada de [33].	22
4.8. Sensor inercial IMU WT901C de WitMotion.[43]	23
4.9. Módulo WiFi TP-Link TL-WR802N utilizado en el robot. Imagen tomada de [41]	24
4.10. Batería LiPo de 4 celdas Gens Ace Tattu (14.8V, 3700mAh). Imagen adaptada de [13].	24
4.11. Módulo de expansión para Raspberry Pi 5 (RPi5). Imagen adaptada de [44].	25
4.12. Diagrama del circuito <i>Voltage Clamp</i> para disipación de energía regenerativa. Adap- tado del manual de usuario de RoboClaw [4].	26
4.13. Placa de interfaz personalizada para la controladora RoboClaw. Integra el circuito de protección Voltage Clamp y los conectores de encoders.	28
4.14. Disposición interna de la electrónica de potencia y control en la base del chasis.	29
4.15. Vista del robot completamente ensamblado, mostrando la disposición final de todos los componentes de hardware.	30
5.1. Diagrama del árbol de transformaciones del robot. Muestra la relación jerárquica entre los sistemas de referencias principales.	32
5.2. Diagrama de la estructura de paquetes dentro del <i>workspace Robot</i>	33
5.3. Diagrama de clases simplificado de la arquitectura del nodo <i>RobotDriverNode</i>	35
6.1. Grafo de nodos y tópicos de ROS 2 durante el proceso de mapeo con RTAB-Map.	40
6.2. Mapa de ocupación 2D del entorno del laboratorio generado con RTAB-Map.	41
6.3. Interfaz de RViz2 durante la ejecución del <i>stack</i> de navegación Nav2, mostrando el mapa, el costmap global y la posición del robot.	44
7.1. (a) Robot en la prueba. (b) Trayectoria registrada por la odometría donde se observa la rotación inicial y el posterior avance lineal.	47
7.2. Visualización en RViz de la odometría generada por la fusión de encoders e IMU. La flecha roja muestra una estimación de pose errática causada por el deslizamiento de las ruedas.	48

7.3.	Mapa fragmentado generado durante las pruebas iniciales. Cada segmento representa un periodo en el que la odometría funcionó antes de perderse y resetearse, un síntoma de sobrecarga computacional en la Raspberry Pi 5.	49
7.4.	Mapa generado durante las pruebas iniciales. Podemos apreciar que los puntos negros no se corresponden con ningún objeto.	51
7.5.	Entorno de prueba del mapa estático.	52
7.6.	Mapa estático de la cajonera.	53
7.7.	Movimiento lineal del robot.	54
7.8.	Resultado de la rotación del robot. La flecha verde muestra la posición inicial. . . .	55
7.9.	Ventana de <code>rtabmap_viz</code> mostrando una hipótesis de cierre de bucle. A pesar de las numerosas correspondencias visuales (líneas cian), la hipótesis es rechazada.	56
7.10.	Vista de RViz2 con el <i>stack</i> de navegación Nav2 activo. Se observa el mapa cargado, el costmap global (púrpura y cian), la nube de partículas de la localización (verde) y el árbol TF completo.	60
8.1.	Diagrama de Gantt donde se muestra la planificación del proyecto.	62
A.1.	Circuito de la PCB.	66
A.2.	Vista superior de la PCB mostrando el layout de componentes y las pistas de cobre. .	67
A.3.	Vista de la PCB completa.	68
B.1.	Plano del soporte para la batería.	69
B.2.	Plano del soporte para la batería.	70
B.3.	Plano de la base de montaje para la RoboClaw	70
B.4.	Plano de la Tapa superior del robot donde van alojados los componentes	71

Índice de tablas

4.1.	Especificaciones técnicas del motor planetario goBILDA 5203 Series[14].	17
4.2.	Especificaciones de la Batería Tattu 4S 3700mAh.	25
5.1.	Tópicos esenciales publicados por el pipeline de la cámara y su uso en el proyecto.	38
7.1.	Resultados de las pruebas de navegación a bajo nivel.	46
7.2.	Comparativa de la carga de la CPU del nodo de la cámara.	49
7.3.	Métricas clave del topic <code>/info</code> durante una hipótesis de localización.	57
8.1.	Presupuesto detallado del <i>hardware</i> del robot.	63

Capítulo 1

Introducción

La robótica móvil autónoma ha evolucionado enormemente en los últimos años, pasando de ser una disciplina experimental a convertirse en una tecnología clave en sectores como la logística, la asistencia sanitaria y la exploración. Uno de los retos más importantes y campos de estudio en la ingeniería actual es lograr que un robot entienda su entorno, se ubique en él y se mueva de manera autónoma. Este Trabajo de Fin de Grado se adentra en este fascinante campo, abordando el diseño y la construcción de una plataforma robótica móvil, desde sus componentes más básicos hasta la implementación de algoritmos de percepción avanzada.

El proyecto se ha guiado por una filosofía de diseño centrada en la creación de una plataforma de bajo costo, modular y accesible. Se busca demostrar que, mediante la integración de componentes comerciales cuidadosamente seleccionados y el uso de software de código abierto de última generación, es posible desarrollar un sistema de mapeo y navegación autónoma robusto y funcional, sentando las bases para futuras investigaciones y aplicaciones prácticas.

1.1. Motivación del proyecto

La motivación principal de este trabajo surge del creciente interés en el campo de los vehículos autónomos y la necesidad de plataformas de *hardware* accesibles para la investigación y el aprendizaje. Mientras que las soluciones comerciales de robótica móvil suelen tener un coste prohibitivo para entornos académicos o de desarrollo individual, la disponibilidad de potentes ordenadores de placa única, como la Raspberry Pi 5, y de sensores de percepción avanzados y asequibles, como la cámara OAK-D Lite, ha democratizado el acceso a esta tecnología.

Este proyecto busca aprovechar este ecosistema tecnológico para construir una base móvil completa. El objetivo no es solo ensamblar componentes, sino abordar de manera integral los desafíos de ingeniería que implican, desde el diseño electrónico para la alimentación y el control de motores, hasta la implementación de un *stack* de software complejo sobre ROS 2 que permita al robot ver y entender su entorno. El enfoque en VSLAM se elige específicamente por ser una técnica puntera que utiliza cámaras y sensores de bajo costo por naturaleza para resolver el problema fundamental de la localización y el mapeo, en contraposición a los sistemas tradicionales basados en Light Detection and Ranging (LiDAR), que suelen ser más costosos.

1.2. Objetivos

Para dirigir este proyecto, se definió un objetivo central que abarca el propósito del trabajo. Además, se definieron una serie de objetivos secundarios que detallan las etapas y las tecnologías clave a implementar.

1.2.1. Objetivo Principal

El objetivo principal de este Trabajo de Fin de Grado es el diseño, construcción, implementación y validación de un sistema robótico móvil de cuatro ruedas, capaz de generar un mapa 2D de un

entorno interior desconocido y utilizarlo para localizarse y navegar de forma autónoma. El desarrollo se enfocará en aprovechar componentes previamente disponibles en el laboratorio y en seleccionar nuevas tecnologías que ofrezcan la mejor relación entre costo y rendimiento.

1.2.2. Objetivos Secundarios

Para alcanzar el objetivo principal, se definieron los siguientes objetivos específicos:

- Desarrollar el sistema a partir de una plataforma inicial compuesta por un chasis diferencial y sus actuadores previamente adquiridos. Se llevará a cabo un estudio del estado de la técnica orientado a la validación de dicha base y a la selección de los componentes electrónicos complementarios, con énfasis en un diseño modular y de bajo costo.
- Diseñar e implementar la electrónica de control y potencia necesaria, incluyendo los controladores de motores y los sistemas de protección.
- Desarrollar el software de control de bajo nivel sobre ROS 2 para la comunicación con el *hardware*, implementando un nodo *driver* que gestione la cinemática y la odometría de las ruedas.
- Implementar y configurar un sistema de percepción basado exclusivamente en sensores visuales e inerciales (una cámara RGB-D y una IMU), marcando como objetivo explícito no depender de un sensor LiDAR, que tradicionalmente es el estándar para estas tareas, pero que tiene un costo más elevado.
- Implementar un sistema de VSLAM utilizando el paquete RTAB-Map para la generación de mapas 2D del entorno y localización.
- Configurar el *stack* de navegación Nav2 para realizar tareas de planificación de rutas utilizando los mapas generados.
- Validar el funcionamiento del sistema completo mediante pruebas experimentales, analizando los resultados y discutiendo los desafíos encontrados.

1.3. Materiales utilizados

Para llevar a cabo el proyecto, se ha empleado una combinación de componentes de *hardware* y herramientas de *software*, seleccionados por su rendimiento, costo y compatibilidad con el ecosistema de código abierto. A continuación, se enumeran los más relevantes, que se detallarán en capítulos posteriores:

- **Hardware:**
 - Chasis goBILDA Recon de 4 ruedas motrices.
 - Motores planetarios goBILDA 5203 Series con encoder integrado.
 - Controladoras de motor RoboClaw 2x7A.
 - Unidad de Procesamiento Principal: Raspberry Pi 5 (8GB).
 - Cámara inteligente RGB-D: Luxonis OAK-D Lite.
 - Unidad de Medición Inercial (IMU): WitMotion WT901C.
 - Batería LiPo 4S y placa de regulación de voltaje para la Raspberry Pi.
- **Software:**
 - Sistema Operativo: Ubuntu 24.04 LTS.
 - Framework Robótico: ROS 2 Jazzy Jalisco.
 - Lenguajes de Programación: Python y C++.
 - Entorno de Desarrollo: Visual Studio Code.

1.4. Estructura del documento

Este documento está organizado en varios capítulos que abarcan todo el desarrollo del proyecto:

- **Capítulo 1 - Introducción:** Se presentan la motivación, los objetivos y la estructura general del trabajo.
- **Capítulo 2 - Estado de la Técnica:** Se realiza una revisión de las tecnologías y enfoques actuales en robótica móvil.
- **Capítulo 3 - Marco Teórico:** Se describen los fundamentos teóricos necesarios para comprender el sistema, incluyendo la cinemática, los sensores y los algoritmos utilizados.
- **Capítulo 4 - Diseño y Construcción de la Plataforma Robótica Móvil:** Se detalla el proceso de selección y ensamblaje de todos los componentes de *hardware* del robot.
- **Capítulo 5 - Arquitectura de Software:** Se explica la implementación del software sobre ROS 2, la configuración del *workspace* y la arquitectura de los nodos desarrollados.
- **Capítulo 6 - Control de Alto Nivel: Mapeo y Navegación:** Se describe la configuración y el funcionamiento de los sistemas de *Simultaneous Localization and Mapping (SLAM)* (RTAB-Map) y navegación (Nav2).
- **Capítulo 7 - Pruebas, Resultados y Discusión:** Se presentan las pruebas experimentales, se analizan los resultados obtenidos y se discuten los desafíos técnicos encontrados, así como sus soluciones.
- **Capítulo 8 - Gestión del Proyecto:** Se muestra el presupuesto y la planificación del proyecto.
- **Capítulo 9 - Conclusiones:** Se resumen las conclusiones finales del trabajo y se proponen posibles líneas de desarrollo futuro.

Capítulo 2

Estado de la Técnica

En este capítulo se presenta una revisión del estado actual de la investigación y la tecnología en el campo de la robótica móvil autónoma, centrándose en los aspectos más relevantes para el desarrollo de un robot de locomoción diferencial de cuatro ruedas capaz de realizar mapeo y navegación. Se explorarán las configuraciones cinemáticas, las técnicas de localización y mapeo simultáneo (SLAM), los algoritmos de planificación de rutas y las plataformas de *hardware* y *software* comúnmente empleadas.

2.1. Robots Móviles

La robótica móvil ha experimentado un crecimiento exponencial en las últimas décadas, impulsada por avances en sensores de bajo coste y alta precisión (como LiDAR y cámaras de profundidad), mayor capacidad de cómputo y algoritmos cada vez más sofisticados. Los robots móviles autónomos, capaces de percibir su entorno, localizarse y navegar sin intervención humana directa, encuentran aplicaciones en campos tan diversos como la logística industrial, la exploración espacial, la asistencia personal, la vigilancia, la agricultura, misiones de búsqueda y rescate de personas, exploración minera, exploración marítima, transporte, investigación militar, etc. [38]. Esta diversidad de aplicaciones impulsa continuamente la investigación y el desarrollo de nuevas plataformas y capacidades.

Un robot móvil se puede definir como una máquina automática capaz de moverse en su entorno [37]. Para que estos robots sean verdaderamente útiles en tareas complejas, generalmente requieren autonomía, lo que implica la capacidad de percibir el entorno (sensores), interpretar esa información para construir modelos internos y localizarse, planificar acciones para alcanzar objetivos (planificación de tareas y rutas) y ejecutar esas acciones mediante actuadores (control), a menudo operando en lazo cerrado para adaptarse a las dinámicas del entorno y corregir errores [37].

La forma en que un robot se desplaza es una de sus características principales. Los robots móviles se pueden clasificar principalmente en función del tipo de locomoción que se utilizan para desplazarse por su entorno. Según Siegwart y Nourbakhsh [37], existen tres categorías principales ampliamente reconocidas:

- **Robots con ruedas:** Son los más comunes debido a su simplicidad mecánica, facilidad de control y alta eficiencia energética sobre superficies planas y regulares. Son ideales para aplicaciones en interiores o terrenos estructurados.
- **Robots con patas:** Inspirados en locomoción animal, ofrecen una gran adaptabilidad a terrenos irregulares o no estructurados. Su implementación resulta más compleja, ya que requieren mayor cantidad de grados de libertad y algoritmos avanzados de control de equilibrio y marcha.
- **Robots con orugas:** Utilizan bandas continuas que proporcionan una gran tracción y estabilidad, lo que los hace adecuados para terrenos difíciles. A pesar de su robustez, su maniobrabilidad en espacios reducidos es inferior comparada con otros tipos.

Además de estas tres categorías tradicionales, en los últimos años se ha desarrollado un tipo de locomoción híbrida que combina ruedas y patas. Este enfoque busca aprovechar la eficiencia y simplicidad de las ruedas en terrenos planos, junto con la capacidad de las patas para superar obstáculos y adaptarse a terrenos irregulares. Los robots con locomoción híbrida pueden, por ejemplo, desplazarse rápidamente en modo rodado y activar sus patas cuando se requiere mayor estabilidad o capacidad de ascenso en entornos complejos. Por ejemplo, el robot *Momaro* presenta un diseño híbrido que le permite desplazarse omnidireccionalmente en terrenos planos mediante ruedas, y utilizar sus patas para sortear obstáculos o escaleras. Este diseño ha demostrado ser efectivo en escenarios de rescate y exploración, como se evidenció en su participación en el DARPA Robotics Challenge [36].

No obstante, considerando las ventajas en términos de eficiencia, coste, robustez y la madurez de los algoritmos de control y navegación disponibles para entornos predominantemente planos, los robots móviles con ruedas y, en particular, la configuración diferencial, representan una base sólida para este tipo de aplicaciones.

2.2. Robots Móviles con Tracción Diferencial

Dentro de los robots móviles con ruedas, existen diversas configuraciones cinemáticas que determinan fundamentalmente cómo se controla su movimiento. Algunas de las más relevantes son la configuración Ackermann (típica de coches), triciclo, síncrona (todas las ruedas giran y se orientan juntas), omnidireccional y *skid-steer*. [38].

La tracción diferencial sigue siendo una de las arquitecturas más extendidas y estudiadas de la robótica móvil actual, tanto en investigación como en productos comerciales (robots educativos, de logística, aspiradores autónomos, etc.). Su popularidad se debe a un excelente compromiso entre simplicidad mecánica, coste, eficiencia energética y buena maniobrabilidad, destacando su capacidad para realizar giros sobre su propio eje. En la figura 2.1 podemos ver los diferentes tipos de configuración.

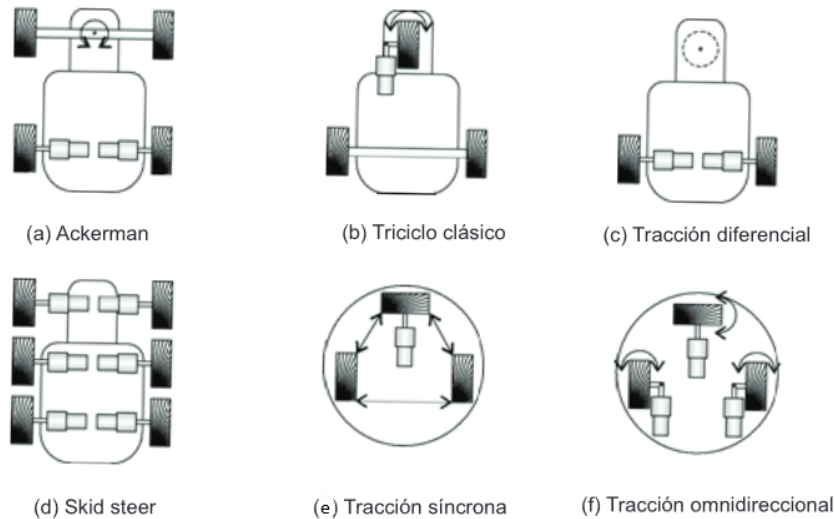


Figura 2.1: Configuraciones de los RMR. Imagen adaptada de [38]

2.3. SLAM: Localización y Mapeo Simultáneo

La capacidad de un robot móvil para operar de forma autónoma en entornos desconocidos o cambiantes depende críticamente de su habilidad para construir un mapa del entorno y localizarse dentro de él simultáneamente, un problema conocido como SLAM [40]. Si bien los fundamentos teóricos se establecieron hace décadas, su desarrollo ha evolucionado considerablemente y, en la

actualidad, constituye una tecnología clave ampliamente utilizada en ámbitos tanto académicos como industriales, incluyendo robots de almacén, vehículos autónomos y drones. El estado del arte en el SLAM se distingue por:

- **Dominio de enfoques basados en optimización/grafos:** Aunque los métodos iniciales basados en filtros probabilísticos (como EKF-SLAM o los filtros de partículas [16]) fueron pioneros, el estado del arte actual, particularmente para el mapeo a gran escala y robustez, está dominado por técnicas basadas en la optimización de grafos (Graph-SLAM) [16]. Estos métodos representan las poses del robot y los elementos de un mapa como nodos en un grafo, y las mediciones (odometría, observaciones de sensores) como restricciones o aristas entre ellos. El problema del SLAM se formula entonces como un problema de optimización no lineal sobre este grafo, buscando la configuración de nodos que mejor explique las mediciones observadas. Esta formulación permite manejar eficientemente grandes cantidades de datos y, crucialmente, incorporar de forma robusta la información del cierre de bucles, que es la habilidad de reconocer un lugar previamente visitado para corregir la deriva acumulada del mapa y la trayectoria.
- **Slam Visual (VSLAM):** Impulsado por la ubicuidad, bajo costo y riqueza de las cámaras, el VSLAM se ha consolidado como una alternativa o complemento muy potente al SLAM basado en LiDAR[6]. VSLAM utiliza una o varias cámaras para estimar simultáneamente la trayectoria del sensor y reconstruir la estructura 3D del entorno. Existen diferentes enfoques dentro del VSLAM:
 - **Basados en características:** Detectan puntos salientes en las imágenes y los rastrean entre fotogramas para estimar el movimiento. Son robustos a cambio de iluminación, pero pueden fallar en entornos con poca textura. ORB-SLAM3 es un referente actual en esta categoría[7].
 - **Directos:** Utilizan directamente la información de la densidad de los píxeles, sin extraer características explícitas. Pueden ser más densos y precisos en entornos con textura, pero son más sensibles a cambios de iluminación.
 - **RGB-D SLAM:** Aprovechan las cámaras que proporcionan tanto imagen a color (RGB) como información de profundidad por píxel, como las cámaras estéreo o de luz estructurada/ToF (Tiempo de Vuelo). La disponibilidad directa de la profundidad simplifica significativamente la estimación de la escala y la reconstrucción 3D haciendo este enfoque muy popular para la robótica.[22].
- **Fusión Visual-Inercial (VIO/SLAM):** Para aumentar la robustez del VSLAM, especialmente durante movimientos rápidos, en entornos con pocas características visuales o ante oclusiones temporales de la cámara, la fusión de datos visuales con una IMU se ha convertido en una práctica estándar [7]. La IMU proporciona mediciones de alta frecuencia de aceleración lineal y velocidad angular, que complementan la información visual de baja frecuencia, pero de alta precisión geométrica. Algoritmos como ORB-3SLAM o VINS-Mono son ejemplos punteros de sistemas VIO/SLAM. Muchas cámaras RGB-D modernas integran una IMU precisamente para facilitar esta fusión.

2.3.1. RTAB-Map: Ejemplo de Solución Integral de Graph-SLAM Basado en Apariencia

Dentro del panorama de sistemas SLAM disponibles, RTAB-Map[25] destaca como una solución de Graph-SLAM de código abierto, especialmente potente para aplicaciones a largo plazo y a gran escala utilizando sensores RGB-D o estéreo. Sus características clave incluyen:

- **Cierres de bucle basados en apariencia:** Utiliza una técnica de “bolsa de palabras visuales” para detectar si el robot ha regresado a un lugar previamente visitado, comparando la apariencia real de la imagen actual con las almacenadas, siendo robusto a cambios de puntos de vista.

- **Gestión de memoria:** Implementa un mecanismo de memoria a corto y largo plazo para limitar la cantidad de datos utilizados en la optimización en tiempo real, permitiendo escalar a entornos grandes sin un coste computacional prohibitivo.
- **Flexibilidad de Sensores:** Soporta una amplia variedad de sensores como LiDAR, estéreo, RGB-D, así como la integración de la odometría.
- **Integración con ROS:** Dispone de paquetes de ROS actualizados que facilitan su integración en arquitecturas robóticas estándar.
- **Generación de mapas 2D/3D:** Puede generar tanto mapas de ocupación 2D como nubes de puntos 3D densas.

RTAB-Map representa una opción madura y validada, adecuada para el objetivo de este Trabajo Fin de Grado (TFG) de realizar mapeo y localización con una cámara OAK-D-Lite.

Considerando estos avances y la disponibilidad de herramientas robustas y de código abierto, este TFG adopta el enfoque de VSLAM basado en grafos, implementado en RTAB-Map, utilizando los datos RGB-D proporcionados por la cámara OAK-D Lite y una IMU externa. Esta elección se alinea con las prácticas actuales para lograr un mapeo y localización precisos y escalables en interiores de manera costo-efectiva. A continuación, se detallará el estado de la técnica en navegación autónoma, que utiliza los mapas y la localización generados por sistemas como RTAB-Map.

2.4. Navegación Autónoma: Localización y Planificación de Rutas

Una vez que el robot es capaz de construir el mapa y de localizarse dentro de él, el siguiente paso es la navegación autónoma. Esta consiste en la capacidad del robot para desplazarse desde un punto de inicio a un punto objetivo de forma segura y eficiente. El estado de la técnica en este campo, particularmente dentro del ecosistema ROS 2, está representado por el conjunto de paquetes de Nav2 [34].

Nav2 es una reescritura completa y modernizada del *stack* de navegación original de ROS. Proporciona una solución de nivel de producción, altamente configurable y flexible, que descompone el problema de la navegación en los siguientes componentes clave:

- **Servidor de Mapa (*Map Server*):** Es el nodo responsable de cargar y servir el mapa estático del entorno, previamente generado por el sistema SLAM.
- **Localización:** Para que la navegación sea posible, el robot debe localizarse continuamente en el mapa. El método estándar en ROS, especialmente para robots con LiDAR, ha sido históricamente Adaptive Monte Carlo Localization (AMCL) [11]. AMCL es un robusto filtro de partículas probabilístico, pero tiene una limitación fundamental para este proyecto: está diseñado para trabajar con datos de escaneo 2D del tipo `sensor_msgs/LaserScan`. En la arquitectura de este proyecto, es el propio RTAB-Map en modo de localización quien se encarga de esta tarea, aprovechando su base de datos visual para localizar el robot.

El sistema de percepción de este TFG, basado en la cámara OAK-D Lite, genera nubes de puntos (`sensor_msgs/PointCloud2`). Si bien es posible convertir una nube de puntos en un falso escaneo 2D, este proceso descarta una gran cantidad de información valiosa. Por ello, se ha optado por una solución de localización nativa para datos visuales. En la arquitectura de este proyecto, es el propio RTAB-Map en modo de localización quien se encarga de esta tarea. Este enfoque aprovecha directamente la rica información visual y de profundidad del mapa para localizar el robot, ofreciendo una solución más integrada y potencialmente más robusta para un sistema basado en VSLAM.

- **Servidor Planificador (*Planner Server*):** Gestiona los algoritmos de planificación global. Estos algoritmos utilizan el mapa completo para encontrar una ruta óptima desde el robot hasta el objetivo.

- **Servidor Controlador (*Controller Server*):** Gestiona los algoritmos de planificación local y control. Su función es generar comandos de velocidad (v, ω) para seguir la ruta global, mientras evita obstáculos inmediatos (estáticos o dinámicos) detectados por los sensores en tiempo real.
- **Navegador con Árboles de Comportamiento (*BT Navigator*):** Una de las mayores innovaciones de Nav2 es el uso de Árboles de Comportamiento (*Behavior Trees*) para orquestar la lógica de navegación. Esto reemplaza las antiguas máquinas de estado, permitiendo definir comportamientos complejos (planificar, seguir ruta, esperar, recuperarse de un fallo) de una manera mucho más flexible y modular [8].

La implementación de este TFG se alinea directamente con este estado de la técnica, utilizando los servidores de mapa, planificador y controlador de Nav2 para la planificación de trayectorias, mientras que la localización es proporcionada por el algoritmo de RTAB-Map.

Capítulo 3

Marco Teórico

Este capítulo describe los fundamentos teóricos necesarios para comprender el diseño, la implementación y la evaluación del sistema de mapeo y navegación autónoma desarrollado en este TFG. Se resumen los conceptos clave relacionados con la cinemática del robot móvil, los sensores utilizados, las técnicas de SLAM, los algoritmos de navegación autónoma y el marco de trabajo ROS 2.

3.1. Cinemática del Robot móvil diferencial

Para controlar el movimiento de la plataforma robótica e interpretar correctamente las mediciones de los sensores, resulta fundamental comprender el modelo cinemático, ya que este constituye la base para diseñar el sistema de control y estimar el movimiento (odometría).

En la figura 3.1 se muestra un robot móvil de cuatro ruedas que se mueve en un plano con una referencia inercial (X_g, Y_g). También es necesario definir un sistema de referencia local cuyo origen se encuentra en el centro de masas del robot (COM).

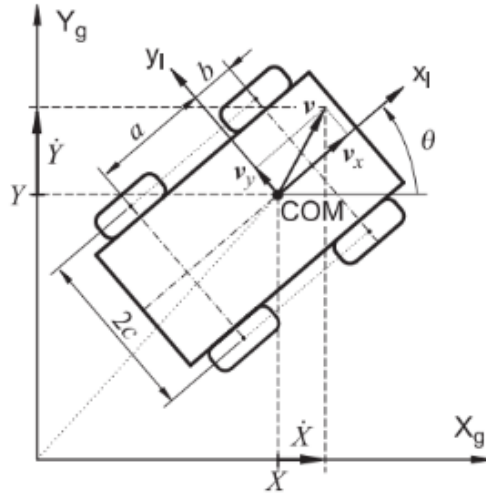


Figura 3.1: Cinemática cuerpo libre. Imagen adaptada de [23]

La posición del robot en el marco inercial se describe por el vector $\mathbf{q}$:

$$\mathbf{q} = \begin{bmatrix} X \\ Y \\ \theta \end{bmatrix} \in \mathbb{R}^3 \quad (3.1)$$

donde (X, Y) es la posición del punto de referencia COM , y θ es la orientación del eje X_l respecto al eje X_g .

La velocidad del robot se describe por el vector $\boldsymbol{\nu}$:

$$\boldsymbol{\nu} = \begin{bmatrix} v \\ \omega \end{bmatrix} \in \mathbb{R}^2 \quad (3.2)$$

donde v es la velocidad lineal instantánea del centro de masas del robot en la dirección de su eje X_g , y ω es su velocidad angular instantánea alrededor del eje Z_g (perpendicular al plano de movimiento).

La ecuación cinemática fundamental relaciona la derivada de la pose en el marco inercial ($\dot{\mathbf{q}}$) con la velocidad en el marco local ($\boldsymbol{\nu}$) mediante una matriz de rotación dependiente de la orientación θ :

$$\dot{\mathbf{q}} = \begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (3.3)$$

Esta ecuación (3.3) muestra cómo la velocidad local (v, ω) se traduce en un cambio de la posición global $(\dot{X}, \dot{Y}, \dot{\theta})$.

Para un robot diferencial ideal con radio de rueda r y distancia entre ruedas $2c$ (ancho de vía), las velocidades locales v y ω se relacionan con las velocidades angulares de las ruedas derecha (ω_R) e izquierda (ω_L) mediante la cinemática directa:

$$v = \frac{r(\omega_R + \omega_L)}{2} \quad (3.4)$$

$$\omega = \frac{r(\omega_R - \omega_L)}{2c} \quad (3.5)$$

Estas ecuaciones son la base de la odometría por encoders, estimando (v, ω) a partir de las mediciones de ω_R y ω_L , para luego integrar (3.3) y obtener la pose estimada.

Para el control, se necesita la cinemática inversa, que calcula las velocidades de rueda necesarias para lograr la velocidad deseada (v_d, ω_d) :

$$\omega_R = \frac{1}{r} (v_d + \omega_d c) \quad (3.6)$$

$$\omega_L = \frac{1}{r} (v_d - \omega_d c) \quad (3.7)$$

Una propiedad crucial del modelo diferencial es su restricción cinemática no holonómica. Esto significa que el robot no puede desplazarse directamente hacia los lados, es decir, el robot no puede moverse instantáneamente en la dirección lateral (eje y_R). Matemáticamente, esto se expresa como:

$$\dot{y}_g = \dot{Y} \cos \theta - \dot{X} \sin \theta = 0 \quad (3.8)$$

Lo que indica que el movimiento del robot debe estar alineado con su eje frontal X_g , y que no es posible tener una componente de velocidad perpendicular en esa dirección.

Esta restricción (3.8) significa que el robot solo puede seguir trayectorias cuya tangente esté alineada con su eje longitudinal X_g . Los algoritmos de planificación de movimiento y control local deben generar comandos (v, ω) que respeten esta limitación [26].

3.2. Sensores y actuadores para la Robótica Móvil Autónoma

La autonomía de un robot móvil depende de su capacidad para percibir el entorno (sensores exteroceptivos), conocer su propio estado de movimiento (sensores propioceptivos e inerciales) y actuar sobre su entorno (actuadores). Este TFG utiliza una combinación de sensores visuales, inerciales y encoders, junto con motores de Corriente Continua (DC) para la actuación.

3.2.1. Sensores para Percepción Visual-Inercial

El sistema de percepción principal elegido para este robot se basa en una fusión de datos de una cámara RGB-D y una IMU externa, una combinación potente para un VSLAM robusto.

Cámaras RGB-D y Percepción de Profundidad estéreo

Las cámaras RGB-D son fundamentales en la robótica moderna al proporcionar, de forma sincronizada, tanto una imagen a color (RGB) como un mapa de profundidad (D) por píxel. Esta información 3D densa del entorno simplifica enormemente tareas como la reconstrucción de escenas, la detección de obstáculos y el VSLAM [22]. La profundidad elimina la ambigüedad de escala inherente a las cámaras monoculares y facilita la reconstrucción directa de la geometría de la escena.

La cámara OAK-D Lite [29] utilizada obtiene la información de profundidad mediante visión estéreo pasiva. Este principio utiliza dos cámaras monocromáticas separadas por una distancia horizontal conocida (línea base, B). Un punto 3D en el mundo se proyecta en diferentes posiciones horizontales (u_L, u_R) en las imágenes capturadas por las cámaras izquierda y derecha respectivamente, debido al efecto de paralaje. La diferencia $d = u_L - u_R$ se denomina disparidad.

Asumiendo un modelo de cámara *pinhole* idealizado, donde los centros ópticos y los planos de imagen de ambas cámaras están perfectamente alineados, la profundidad Z del punto 3D con respecto al plano de las cámaras se puede recuperar mediante triangulación simple:

$$Z = \frac{f \cdot B}{d} \quad (3.9)$$

donde f es la distancia focal de las cámaras (expresada en píxeles). Para que esto sea una ecuación válida, las imágenes deben estar rectificadas. La cámara OAK-D Lite realiza el cálculo de disparidad internamente [28].

Unidades de Medición Inercial (IMU)

La IMU es un sensor electrónico que mide e informa sobre la fuerza específica, la velocidad angular y, en algunos casos, la orientación de un cuerpo. Típicamente, una IMU de 9 grados de libertad (9-DoF) contiene:

- **Tres Giróscopos ortogonales:** Miden la velocidad angular ($\omega_x, \omega_y, \omega_z$) alrededor de los ejes del sensor.
- **Tres Acelerómetros ortogonales:** Miden la aceleración propia o específica (a_x, a_y, a_z) del sensor.
- **Magnetómetros:** Miden el campo magnético terrestre y ayudan a estabilizar la estimación del yaw (orientación absoluta respecto al norte magnético), aunque son sensibles a perturbaciones magnéticas del entorno.

En el contexto de la Odometría Visual-Inercial (VIO) o el Visual Inertial SLAM (VISLAM), las mediciones de la IMU, aunque ruidosas y con deriva a largo plazo, son cruciales. Proporcionan estimaciones de movimiento de alta frecuencia (típicamente 100-1000 Hz), que interpolan el movimiento entre los fotogramas de la cámara (típicamente 10-60 Hz). Esto permite:

- Rastrear movimientos rápidos que causarían desenfoque o pérdida de seguimiento en sistemas puramente visuales.
- Mejorar la estimación de la escala métrica del entorno, especialmente en VSLAM monocular (aunque con RGB-D la escala ya está dada por la profundidad).
- Proveer una estimación inicial de la orientación (especialmente la inclinación o pitch y roll respecto a la gravedad).
- Permitir la continuación de la estimación de odometría (aunque con deriva creciente) durante breves periodos de fallo del sistema visual (oclusiones, falta de textura, iluminación extrema).

La fusión robusta de los datos de la IMU con la información visual es un componente esencial de los sistemas VISLAM modernos, usualmente realizada mediante filtros extendidos de Kalman o, de forma más precisa y computacionalmente intensiva, a través de técnicas de optimización no lineal basadas en grafos [7]. En este TFG, se utilizará una IMU externa cuyos datos se integrarán con los de la cámara OAK-D Lite mediante el algoritmo RTAB-Map.

3.2.2. Actuación y Propiocepción

Mientras que los sensores exteroceptivos (cámara) e inerciales (IMU) permiten al robot percibir su entorno y su movimiento, el sistema de actuación y propiocepción es el responsable de generar y medir el movimiento físico del robot.

Motores DC, Reductoros y Frenada Regenerativa

La tracción del robot se logra mediante motores de DC. Estos son actuadores electromecánicos que convierten la energía eléctrica en movimiento rotatorio. Su velocidad es, en una primera aproximación, proporcional al voltaje aplicado, mientras que su par motor (torque) es proporcional a la corriente consumida [18]. Para adaptar las altas velocidades y el bajo torque de los motores DC a las necesidades de tracción de un robot, se acoplan a cajas reductoras (*gearboxes*) que disminuyen la velocidad angular de salida y multiplican el torque en la misma proporción.

Un fenómeno físico fundamental e inherente a los motores DC es la frenada regenerativa. Cuando un motor en movimiento es forzado a desacelerar (por ejemplo, al recibir un comando de velocidad menor o al frenar), su inercia lo hace funcionar como un generador. En este estado, convierte la energía cinética del robot de nuevo en energía eléctrica, inyectando un pico de corriente y tensión inversa en la línea de alimentación [4]. Si no se gestiona adecuadamente, este pico de tensión puede superar la tensión máxima soportada por la batería o los componentes electrónicos sensibles (como la unidad de procesamiento), causando daños permanentes. Por ello, el diseño del sistema de alimentación debe contemplar mecanismos de protección contra este efecto, como se detallará en el Capítulo 4.

Encoders Rotatorios de Cuadratura

Para la propiocepción, es decir, la medición del estado de los actuadores, se utilizan encoders incrementales de cuadratura. Estos sensores miden la posición angular relativa de un eje en rotación. Generan dos señales digitales pulsantes (Canal A y Canal B) desfasadas 90 grados entre sí, lo que permite determinar tanto la magnitud (contando pulsos) como la dirección del giro [18].

La resolución del encoder, medida en Pulsos por Revolución (ppr), determina la precisión de la medición angular. Los datos de los encoders son esenciales para los controladores de bajo nivel, que los utilizan en un lazo de control para asegurar que las ruedas giren a la velocidad comandada por el sistema de navegación. Aunque la odometría principal de este TFG se basa en VIO, la información de los encoders es indispensable para el control preciso del movimiento.

3.3. RTAB-Map: Mapeo y Localización Basados en Apariencia

La solución SLAM seleccionada para este TFG es RTAB-Map [25], un sistema de Graph-SLAM de código abierto optimizado para sensores RGB-D. Sus principios fundamentales son:

3.3.1. Detección de Cierres de Bucle mediante *Bag-of-Words*

Un componente esencial de RTAB-Map es su robusta detección de cierres de bucle (*loop closure*), que permite corregir la deriva acumulada. Para ello, utiliza una técnica basada en la apariencia visual global de la escena mediante el paradigma de Bag of Visual Words (BoW) [39]. Cada imagen se representa como un histograma de “palabras visuales” (prototipos de características locales). Cuando la representación BoW de la imagen actual es suficientemente similar a la de una imagen almacenada previamente, se genera una hipótesis de cierre de bucle. Esta hipótesis se somete a una

validación geométrica para confirmar la coincidencia y calcular la transformación relativa entre las poses.

3.3.2. Gestión de Memoria y Optimización del Grafo

Para operar eficientemente en entornos extensos o durante largos períodos, RTAB-Map emplea un sistema de gestión de memoria que distingue entre una Memoria de Trabajo (WM) de tamaño limitado para el procesamiento en tiempo real y una Memoria a Largo Plazo (LTM) donde se almacenan representaciones más ligeras de ubicaciones más antiguas. Este mecanismo permite escalar el sistema sin un coste computacional prohibitivo [25].

El grafo de poses del sistema se construye con nodos (poses del robot) y aristas (restricciones de odometría y de cierres de bucle). Cuando se añade un cierre de bucle validado, se realiza una optimización global del grafo (generalmente mediante mínimos cuadrados, usando librerías como g2o [24]) para distribuir el error y mejorar la consistencia del mapa y la trayectoria estimada.

3.3.3. Generación de Mapas y Modo de Localización

A partir del grafo optimizado y los datos sensoriales (RGB-D), RTAB-Map puede generar diversas representaciones del entorno, incluyendo nubes de puntos 3D y mapas de ocupación 2D, estos últimos cruciales para la navegación 2D.

Además del modo SLAM, RTAB-Map puede operar en un modo de localización pura. En este modo, se carga un mapa previamente construido y el sistema utiliza las nuevas observaciones sensoriales para estimar la pose actual del robot dentro de dicho mapa sin modificarlo. Este modo es esencial para la fase de navegación autónoma.

Para navegar, el robot debe primero saber dónde está. El método más extendido es AMCL, que utiliza un filtro de partículas para estimar la pose del robot [40]. Mantiene una nube de hipótesis (partículas) sobre la posible ubicación del robot y las actualiza y re-muestra en función de la odometría y las observaciones de los sensores.

Sin embargo, en este TFG se emplea el modo de localización de RTAB-Map. En este modo (activado con el parámetro `Mem/IncrementalMemory` a `false`), el sistema carga el mapa visual-geométrico previamente construido. Al recibir nuevos datos de la cámara y la IMU, busca correspondencias entre la vista actual y la base de datos de imágenes del mapa. Al encontrar una coincidencia, se puede localizar el robot con gran precisión, proporcionando la transformación `map` → `odom` que Nav2 necesita para operar. Este enfoque basado en reconocimiento de lugares es particularmente robusto a errores de odometría a largo plazo.

3.4. Navegación Autónoma con Nav2

Una vez que el sistema de SLAM ha generado un mapa y es capaz de localizar al robot dentro de él, el siguiente paso es la navegación autónoma. Esta consiste en la capacidad del robot para desplazarse desde un punto de inicio a un punto objetivo de forma segura y eficiente. En el ecosistema ROS 2, esta tarea es gestionada por la pila de navegación **Nav2**, que descompone el problema en los siguientes componentes fundamentales.

3.4.1. Mapas de Costos para la Evitación de Obstáculos

Un concepto central en la navegación de ROS es el mapa de costos (*costmap*) [32]. Es una rejilla 2D donde cada celda no solo indica si está ocupada o libre, sino que tiene un valor que representa el costo o el riesgo de atravesarla. Se construyen en capas:

- **Capa Estática:** Proviene del mapa generado por SLAM.
- **Capa de Obstáculos:** Se actualiza en tiempo real con los datos de los sensores (en este caso, la nube de puntos 3D de la OAK-D Lite proyectada a 2D), añadiendo obstáculos que no están presentes en el mapa original.

- **Capa de Inflación:** Infla los obstáculos, creando un gradiente de costo a su alrededor. Esto disuade al planificador de trazar rutas demasiado cerca de las paredes y objetos, mejorando la seguridad.

Nav2 utiliza dos costmaps: uno global para el planificador de largo alcance y uno local (más pequeño y centrado en el robot) para el controlador de corto alcance.

3.4.2. Planificación Global y Local

- **Planificador Global:** El objetivo es encontrar el camino de menor coste en el costmap global. El algoritmo A* es un método de búsqueda en grafos que garantiza encontrar el camino más corto, si existe. Explora el espacio de búsqueda priorizando los caminos que son prometedores, tanto por el coste ya recorrido como por una estimación heurística del coste restante hasta el objetivo [17].
- **Controlador / Planificador Local:** Su objetivo es generar comandos de velocidad (v, ω) que sigan la ruta global y eviten obstáculos locales. Un enfoque común es el Dynamic Window Approach (DWA), que funciona de la siguiente manera:
 1. Simula trayectorias circulares en un corto horizonte de tiempo, muestreando pares de velocidades (v, ω) que son alcanzables por el robot (respetando su cinemática y límites de aceleración).
 2. Descarta las trayectorias que colisionarían con obstáculos en el costmap local.
 3. Puntúa las trayectorias restantes según criterios como el progreso hacia el objetivo, la distancia a los obstáculos y la similitud con la velocidad actual.
 4. Selecciona la trayectoria con la mejor puntuación y envía su par (v, ω) a los motores [12].

3.4.3. Árboles de Comportamiento para la Lógica de Navegación

A diferencia de las máquinas de estado finitas, Nav2 utiliza Árboles de Comportamiento (*Behavior Trees* o BTs) para orquestar las tareas. Un BT es una estructura de árbol jerárquica de nodos que pueden ser acciones, condiciones o composiciones. Esto permite crear lógicas complejas de forma modular y reactiva, como “Calcula una ruta, síguela hasta que llegues a un obstáculo que la bloquee; si se bloquea, espera y replanifica” [8]. La lógica de navegación de este TFG está definida por el BT Navigator de Nav2, que coordina los servidores del planificador y del controlador.

Capítulo 4

Diseño y Construcción de la Plataforma Robótica Móvil

Este capítulo detalla el proceso de diseño, selección de componentes y configuración de la plataforma robótica móvil desarrollada. El diseño se ha guiado por una filosofía central: crear una plataforma de bajo coste, modular y accesible, capaz de realizar tareas de mapeo y navegación autónoma utilizando componentes básicos pero potentes y software de código abierto. Se describe tanto el *hardware* que constituye el robot físico. El objetivo es presentar una visión completa del sistema, justificando las decisiones de diseño y proporcionando la información necesaria para comprender su funcionamiento y su posterior evaluación en el capítulo de resultados.

4.1. Descripción General del Sistema

La plataforma robótica ha sido diseñada como un sistema integrado para realizar tareas de mapeo y navegación autónoma basadas en visión. Se compone de una base móvil de tracción diferencial, una unidad de percepción visual-inercial, un sistema de procesamiento central y una arquitectura de software modular basada en ROS 2. La Figura 4.1 muestra un diagrama con los componentes principales y el flujo de información y control entre ellos.

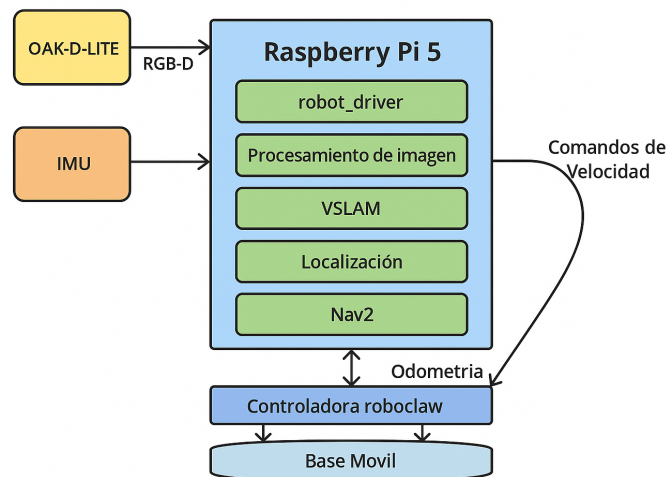


Figura 4.1: Diagrama de bloques de la arquitectura general del sistema. Flujos de datos sensoriales, comandos de control e interacción entre procesador y periféricos.

Como se observa, la RPi5 actúa como el cerebro del sistema, procesando los datos de la cámara OAK-D Lite y la IMU para ejecutar los algoritmos de SLAM y navegación. A su vez, envía co-

mandos de velocidad a un controlador de motores que gestiona los cuatro actuadores, cerrando así el bucle de percepción, decisión y acción.

4.1.1. Plataforma Base: Chasis goBILDA Recon

La base del robot está construida sobre el chasis modular *goBilda Recon*, un sistema de perfil estructural ligero diseñado específicamente para aplicaciones de robótica móvil. Este chasis proporciona una estructura robusta, fácilmente configurable y con un diseño simétrico que facilita el montaje de motores, ruedas, electrónica y sensores. Este chasis destaca por:

- **Modularidad:** Está construido con perfiles de plástico con un patrón de rejilla de agujeros estandarizado. Esto ofrece una enorme flexibilidad para montar componentes de distintos tamaños y formas (unidades de procesamiento, sensores, baterías) de manera segura y ordenada.
- **Robustez y Ligereza:** El uso de plásticos estructurales de alto rendimiento asegura una plataforma rígida capaz de soportar el peso de todos los componentes sin deformarse, manteniendo al mismo tiempo un peso total manejable.
- **Compatibilidad del Ecosistema:** Al ser parte del ecosistema goBILDA, garantiza una compatibilidad mecánica directa con los motores, soportes y otros accesorios de la misma marca, lo que reduce drásticamente la necesidad de fabricar piezas a medida y simplifica el proceso de ensamblaje.

Esta elección responde directamente a la filosofía de diseño del proyecto, por su excelente equilibrio entre costo, modularidad y robustez.

El chasis está configurado para una tracción a las cuatro ruedas, proporcionando una base estable y con buena capacidad de tracción. En la Figura 4.2 se puede observar el chasis.



Figura 4.2: Vista del chasis goBilda Recon utilizado como base de la plataforma.[15]

4.1.2. Sistema de Actuación y Propiocepción

El sistema de actuación responsable de la movilidad del robot está compuesto por cuatro conjuntos de motor y encoder idénticos.

Motores: goBILDA 5203 Series Yellow Jacket

La tracción es proporcionada por cuatro motores planetarios **goBILDA 5203 Series Yellow Jacket** (SKU: 5203-2402-0019). Estos motores tienen un excelente equilibrio entre torque y velocidad; también incluyen un encoder de alta resolución integrado, esencial para el control de bajo nivel. La selección de estos motores se justifica por tres razones principales:

1. **Compatibilidad del Ecosistema:** Se integran mecánicamente de forma perfecta con el chasis goBILDA, eliminando la necesidad de adaptadores personalizados.
2. **Rendimiento Adecuado:** Ofrecen un torque de parada de 24.3 kg·cm, suficiente para mover la plataforma con su carga completa de componentes de forma ágil.
3. **Integración Costo-Efectiva:** El factor más decisivo es que incluyen un encoder de cuadratura de alta resolución (537.7 ppr) ya integrado. Esta solución es significativamente más económica y sencilla de implementar que adquirir motores y encoders por separado.

Sus especificaciones más relevantes son:

Tabla 4.1: Especificaciones técnicas del motor planetario goBILDA 5203 Series[14].

Parámetro	Valor
SKU del Fabricante	5203-2402-0019
Voltaje Nominal	12 VDC
Tipo de Reductora	Planetaria
Relación de Transmisión	19.2:1
Velocidad sin Carga (@ 12V)	312 RPM
Corriente sin Carga (@ 12V)	0.25 A
Torque de Parada (<i>Stall Torque</i> @ 12V)	24.3 kg·cm (338 oz·in)
Corriente de Parada (<i>Stall Current</i> @ 12V)	9.2 A
Encoder Integrado	Sí, de cuadratura
Resolución del Encoder (Eje de Salida)	537.7 Pulsos Por Revolución (PPR)
Peso	437 g

La configuración de cuatro motores independientes, uno por rueda, proporciona una tracción robusta. Cinemáticamente, el sistema opera como un robot de tracción diferencial, donde las dos ruedas de cada lado (izquierdo y derecho) reciben el mismo comando de velocidad. Este tipo de accionamiento es a menudo referido como *skid-steer*, ya que el giro se produce por la diferencia de velocidad entre los lados, induciendo un ligero deslizamiento.

Encoders de Cuadratura Integrados

Cada uno de los motores goBILDA Yellow Jacket incorpora un encoder de cuadratura integrado directamente en el eje del motor, lo que permite medir con precisión su rotación. Tras la reductora de 19.2:1, la resolución efectiva del encoder en el eje de salida es de 537.7 ppr.

Esta alta resolución es especialmente útil para el control de velocidad a bajo nivel, permitiendo al controlador de motores implementar lazos de control precisos, como un controlador Proporcional Integral Derivativo (PID). Gracias a esta retroalimentación, es posible garantizar que las ruedas mantengan las velocidades deseadas con mayor exactitud, incluso ante variaciones en la carga o el terreno.

Aunque la estimación principal de pose del sistema se basa en la odometría visual-inercial VIO, los datos de los encoders pueden utilizarse como fuente adicional de odometría o como respaldo en situaciones donde la percepción visual sea limitada (por ejemplo, con baja iluminación o falta de textura).

En la Figura 4.3 se muestra el motor goBILDA 5203 Series Yellow Jacket junto con su plano dimensional, ambos proporcionados por el fabricante.

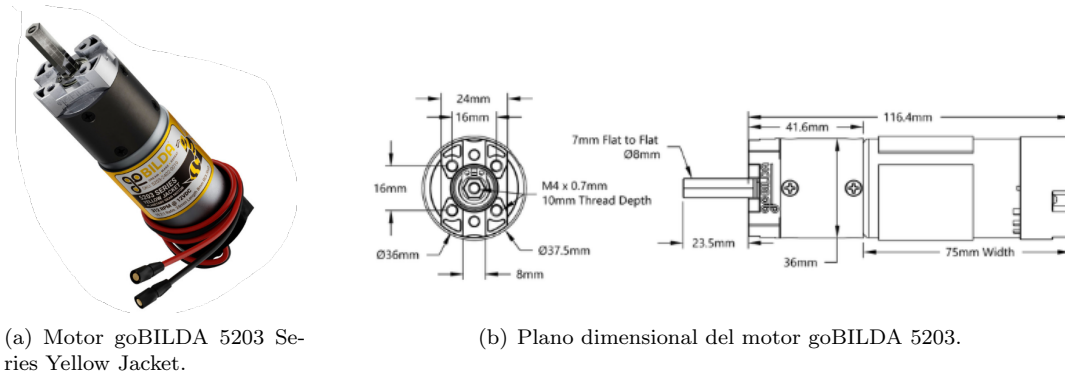


Figura 4.3: Vista general y dimensiones del motor goBILDA 5203 Series Yellow Jacket con encoder integrado. Imágenes adaptadas de [14].

4.1.3. Unidades de Procesamiento

El sistema electrónico centraliza el “pensamiento” y el control del robot. Se organiza jerárquicamente: una unidad de procesamiento de alto nivel se encarga de las tareas complejas (SLAM, navegación), mientras que los controladores de bajo nivel gestionan directamente los actuadores (motores).

Unidad de Procesamiento Principal: Raspberry Pi 5

El cerebro del robot es una RPi5. Se elige este modelo por ser el último y el más potente de la familia Raspberry Pi hasta el momento de desarrollo de este proyecto. Cuenta con las siguientes características

- Un procesador Quad-core ARM Cortex-A76 de 64 bits a 2.4GHz.
- Memoria RAM LPDDR4X (en este caso, de 8 GB).
- Conectividad mejorada (USB 3.0, Gigabit Ethernet, Wi-Fi 5).

Esta capacidad de cómputo es necesaria para ejecutar simultáneamente el sistema operativo, ROS 2 Jazzy, y los algoritmos computacionalmente intensivos como RTAB-Map en tiempo real.

En la Figura 4.4 se muestra una vista superior de la Raspberry Pi 5, donde se pueden apreciar sus principales componentes físicos, como el procesador, los conectores, el sistema de alimentación y los puertos de entrada/salida.

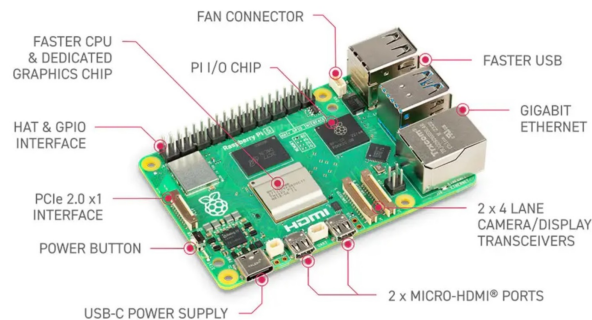


Figura 4.4: Vista superior de la Raspberry Pi 5 con identificación de componentes claves. Imagen adaptada de [9].

Controlador de Motores: RoboClaw 2x7A

Para controlar los cuatro motores goBILDA, se utilizan dos controladoras de motor inteligentes **RoboClaw 2x7A** de Basic Micro [5], una para cada par de ruedas (delanteras y traseras). La elección de la RoboClaw se justifica por sus avanzadas características, que descargan a la RPi5 de las tareas de control de bajo nivel:

- **Control Dual de Motores:** Cada placa puede controlar dos motores DC con escobillas, proporcionando hasta 7.5A de corriente continua por canal (15A de pico).
- **Lazo de Control de Velocidad Cerrado:** La RoboClaw lee directamente las señales de los encoders de cuadratura de los motores. Gracias a su microcontrolador integrado, ejecuta un lazo de control de velocidad interno. Esto significa que la RPi5 solo necesita enviar un comando de velocidad deseada (ej. "gira a 120 RPM"), y la RoboClaw se encarga de manera autónoma de mantener esa velocidad, ajustando la potencia de los motores y compensando variaciones de carga o tensión.
- **Comunicación Serial/USB:** Se comunica con la RPi5 a través de un puerto USB que emula un puerto serie. Esto permite un control robusto y bidireccional, donde no solo se envían comandos, sino que también se puede recibir información de la controladora (corriente, velocidad medida, errores, etc.).

Esta arquitectura, donde una placa de alto rendimiento (RPi5) delega el control motor en tiempo real a una controladora dedicada como la RoboClaw, es una práctica estándar en robótica robusta. Permite que el sistema operativo de propósito general en la RPi5 se centre en tareas complejas sin preocuparse por las interrupciones o la precisión temporal requerida para el control de los motores. En la Figura 4.5 se muestra el controlador.

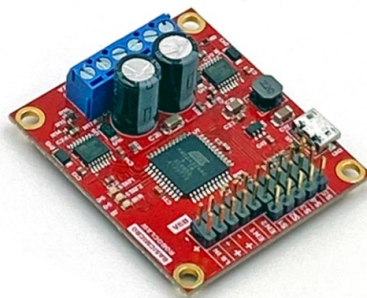


Figura 4.5: Controlador de motores inteligente RoboClaw 2x7A. Imagen adaptada de [5].

La configuración de las controladoras RoboClaw se realizó mediante el software oficial *Motion Studio* [4], que permite ajustar con precisión todos los parámetros operativos. En la Figura 4.6 se muestra una captura de la configuración final utilizada. A continuación, se detallan y justifican los parámetros más importantes para el funcionamiento del robot:

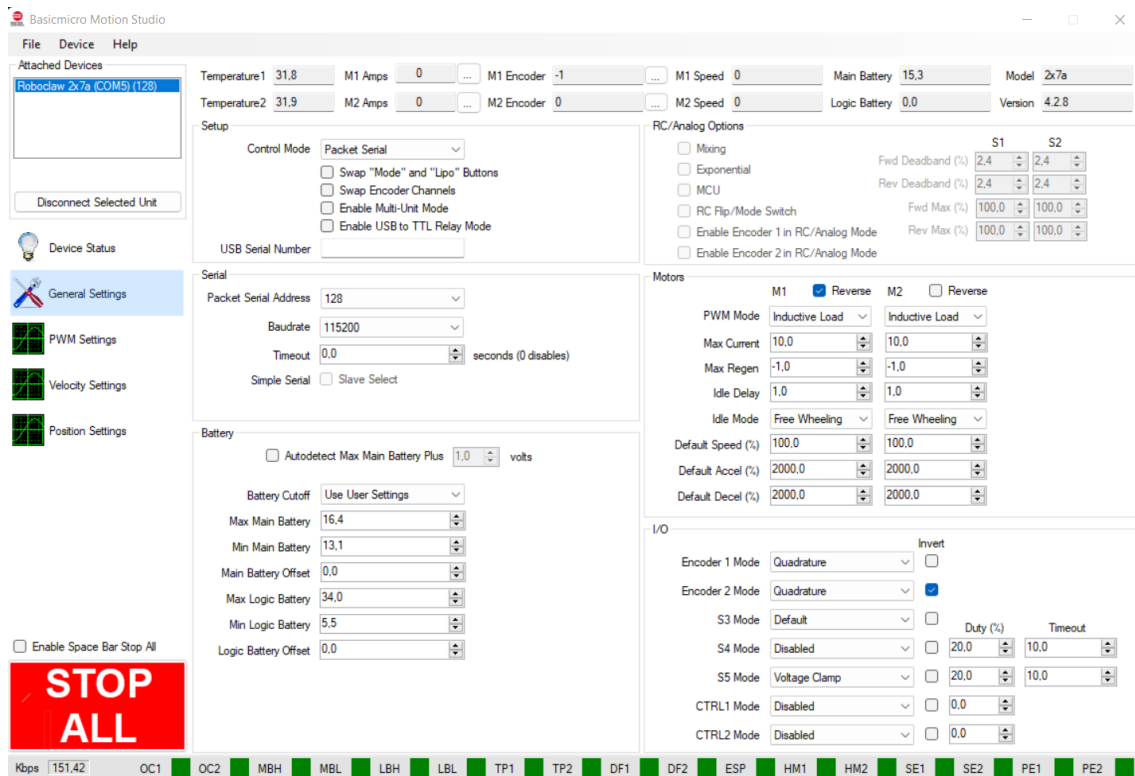


Figura 4.6: Captura de pantalla de la configuración de las controladoras RoboClaw en el software Motion Studio.

Modo de Control y Comunicación Se seleccionó el modo *Packet Serial*, ya que es el protocolo de comunicación más avanzado y robusto que ofrece la RoboClaw. Permite un control bidireccional y el envío de comandos estructurados mediante el puerto Universal Serial Bus (USB). Se configuró una dirección de paquete (*Packet Serial Address*) de 128 (0x80) para la controladora y una velocidad de comunicación (*Baudrate*) de **115200 bps**, un estándar rápido y fiable para la comunicación serie a través de USB.

Configuración de Batería Para proteger la batería de Lithium Polymer (LiPo) de 4S, se establecieron manualmente los límites de tensión (*Use User Settings*). La tensión máxima (*Max Main Battery*) se fijó en 16.4 V, ligeramente por debajo de la tensión de carga completa (16.8V). La razón de este margen de seguridad es proactiva: si el robot opera con la batería casi llena, cualquier evento de frenada regenerativa provocará un aumento instantáneo de la tensión. Al fijar el umbral en 16.4 V, la RoboClaw activará sus mecanismos de protección (como la frenada o el *Voltage Clamp*) antes de que la tensión del bus supere los 16.8V, evitando así la sobrecarga de una batería ya llena y protegiendo al sistema de los picos de tensión asociados. El mínimo (*Min Main Battery*) en 13.1 V (aprox. 3.27V por celda), para prevenir descargas profundas que podrían dañar la batería.

Configuración de Motores y Regeneración Esta es una de las áreas más críticas para la seguridad y el rendimiento:

- **Límite de Corriente:** El Max Current por motor se fijó en 10.0 A. Este valor supera la corriente continua nominal de 7.5A del controlador, permitiendo picos de corriente necesarios para la aceleración, pero protege el *hardware* ante un bloqueo prolongado del motor (*stall*).
- **Límite de Regeneración (Max Regen):** Este parámetro es fundamental para la protección contra los picos de tensión transitorios generados durante la frenada. Un valor de cero deshabilitaría la frenada activa, resultando en una detención no controlada

del robot. Por tanto, es necesario permitir una corriente regenerativa, pero limitarla a un nivel seguro.

Este límite actúa como la segunda capa de protección del sistema, complementando al circuito *Voltage Clamp*. Mientras que el *Voltage Clamp* disipa picos de tensión, el límite de *Max Regen* actúa sobre la fuente del problema, limitando la cantidad de corriente que se genera en primer lugar. El valor de 1.0A (explicaremos esta elección mas adelante en la sección 4.1.6) es un punto de partida seguro: permite una desaceleración efectiva y controlada, pero limita la energía regenerada a un nivel manejable, protegiendo la batería y, de forma indirecta, la electrónica sensible como la RPi5 de sobretensiones bruscas.

Configuración de Entradas/Salidas (I/O):

- **Modo de Encoder:** Se configuró en modo *Quadrature* para decodificar correctamente las señales A/B de los encoders goBILDA integrados.
- **Activación del *Voltage Clamp*:** El pin S5 Mode se configuró en *Voltage Clamp*. Esta es la configuración de software crítica que habilita el circuito de protección físico descrito en la Sección 4.1.7. Con esta opción, la RoboClaw utilizará el pin S5 para enviar la señal de activación al MOSFET del circuito de protección, cuando detecte un evento de sobretensión en la línea de alimentación.

Sintonización del Controlador de Velocidad PIV. Una de las capacidades más avanzadas de la RoboClaw es su bucle de control de velocidad configurable. Si bien un controlador PID es común, para esta aplicación se ha sintonizado y configurado un controlador Proporcional-Integral with Velocity Feedforward (PIV) utilizando el software *Motion Studio*.

Un controlador PIV es una arquitectura de control en cascada que ofrece un rendimiento superior para el control de posición y velocidad en sistemas mecánicos. A diferencia de un PID simple, que actúa únicamente sobre el error, el PIV incorpora de manera explícita la realimentación de la velocidad, lo que permite anticipar el comportamiento dinámico del sistema. La elección de un PIV sobre un PID se justifica porque proporciona un movimiento notablemente más suave y estable, reduciendo el sobreimpulso y las oscilaciones al alcanzar una velocidad o posición deseada. Esto es crucial para la calidad de la odometría y para la estabilidad del sistema VSLAM, que es sensible a movimientos bruscos o vibraciones.

Esta configuración detallada asegura que las controladoras operen de manera eficiente, robustas y, sobre todo, seguras dentro de la arquitectura global del robot.

4.1.4. Unidad de Percepción Visual-Inercial

El éxito de un sistema VSLAM depende críticamente de la calidad y la riqueza de los datos sensoriales. Para este proyecto, se seleccionó una unidad de percepción integrada que proporciona múltiples flujos de datos de forma sincronizada, en lugar de utilizar sensores discretos.

Cámara Inteligente RGB-D: Luxonis OAK-D Lite

La percepción principal del entorno se realiza con una cámara inteligente *Luxonis OAK-D Lite*[29]. Este dispositivo supera las prestaciones de una cámara convencional; es un sistema de visión por computador completo y programable en un formato compacto. Su elección es estratégica y fundamental para el proyecto por las siguientes razones:

- **Percepción 3D Sincronizada:** El OAK-D Lite integra tres cámaras: una cámara central a color (RGB) de alta resolución y un par de cámaras monocromáticas para la percepción de profundidad por estéreo pasivo, como podemos observar en la Figura 4.7 . El dispositivo entrega flujos de datos de imagen y profundidad que están calibrados y sincronizados por *hardware*. Esta sincronización es un requisito indispensable para algoritmos de VSLAM como RTAB-Map, ya que asegura que cada píxel de color corresponde a un valor de profundidad correcto en el mismo instante de tiempo.

- **Procesamiento Embarcado:** El corazón del OAK-D Lite es su Unidad de Procesamiento de Visión (VPU), el modelo **Myriad X (VPU RVC2)** de Intel. Esta VPU es un procesador especializado que permite ejecutar tareas de visión por computador directamente en la cámara, una capacidad conocida como *Edge AI*. Esto tiene una ventaja crítica para un robot basado en una Raspberry Pi 5 que descarga tareas computacionales intensivas como:

- El cálculo del mapa de disparidad y profundidad estéreo.
- La detección y seguimiento de características.
- Y, si es necesario, la inferencia de redes neuronales.

Estos procesos se ejecutan en la VPU, liberando los recursos de la RPi5 para que se dediquen a los algoritmos de alto nivel, como la optimización del grafo de SLAM y la planificación de rutas.

- **Ecosistema de Software Robusto (DepthAI y ROS 2):** Luxonis proporciona una potente Application Programming Interface (API) llamada DepthAI [27], que permite a los desarrolladores crear “*pipelines*” de visión personalizados que se ejecutan en la VPU. Para la integración en robótica, el paquete `depthai-ros` [28] actúa como un puente directo, exponiendo los flujos de datos de la cámara (imágenes RGB, mapas de profundidad y datos de la IMU si la tuviera integrada) como tópicos estándar de ROS 2. Esta integración simplifica enormemente la configuración inicial del sistema de percepción.
- **Potencial para Funcionalidades Avanzadas:** La capacidad de ejecutar redes neuronales en la VPU abre la puerta a futuras mejoras sin necesidad de *hardware* adicional. El mismo dispositivo utilizado para SLAM puede, simultáneamente o de otra manera, realizar reconocimiento de objetos, seguimiento de personas o segmentación semántica para distinguir entre suelo navegable, paredes y obstáculos. Esta versatilidad explica por qué estas cámaras inteligentes constituyen actualmente una de las opciones más innovadoras y eficaces en percepción robótica de bajo coste.

A diferencia de los sensores LiDAR, que ofrecen una percepción precisa basada en láser, pero que suelen ser más costosos y voluminosos, la cámara OAK-D Lite proporciona una solución más compacta, económica y versátil. Su capacidad para generar mapas de profundidad mediante estéreo pasivo, junto con el procesamiento integrado y la integración directa con ROS 2, la convierten en una alternativa ideal para entornos interiores y aplicaciones de bajo presupuesto. Además, al combinar información visual y de profundidad, permite implementar algoritmos de SLAM visual como RTAB-Map sin necesidad de sensores adicionales, lo que simplifica el diseño del sistema robótico.



Figura 4.7: Cámara OAK-D-Lite. Imagen adaptada de [33].

Unidad de Medición Inercial (IMU) Externa

Como se estableció en el marco teórico (Sección 3.2.1), la fusión de los datos inerciales con la información visual es crucial para la robustez de los sistemas VISLAM. Dado que el modelo OAK-

D-Lite no incorpora una IMU integrada, se ha optado por añadir una externa para proporcionar estas mediciones de movimiento de alta frecuencia.

La unidad seleccionada es una **IMU WT901C de WitMotion**, un sensor compacto de 9 grados de libertad (9-DoF) que integra un acelerómetro triaxial, un giroscopio triaxial y un magnetómetro triaxial. Esta combinación permite medir la aceleración lineal, la velocidad angular y la orientación respecto al campo magnético terrestre, lo que resulta fundamental para mejorar la estimación de la pose del robot.

El sensor se comunica con la RPi5 a través de un adaptador de interfaz UART (puerto serie) a puerto USB, como podemos ver en la Figura 4.8, y opera a una frecuencia de muestreo configurable, típicamente en el rango de 50 a 200 Hz. La información de la IMU se integra en el sistema de percepción mediante ROS 2; estos datos se utilizan dentro del algoritmo de RTAB-Map para estabilizar la odometría visual, especialmente en movimientos rápidos o en zonas con baja textura.



Figura 4.8: Sensor inercial IMU WT901C de WitMotion.[43]

4.1.5. Comunicación

Para facilitar el control, la monitorización y la depuración del sistema de forma remota, se ha integrado una solución de conectividad inalámbrica independiente. Se emplea un nanorouter **TP-Link TL-WR802N** [41], un dispositivo compacto que opera en la banda de 2.4 GHz y proporciona una velocidad de transferencia de hasta 300 Mbps.

El módulo se ha configurado en modo **Punto de Acceso (AP)**, creando una red WiFi local a bordo del propio robot. Esta arquitectura de red permite que tanto la unidad de procesamiento del robot (en este caso, la Raspberry Pi 5) como una estación de control externa (un ordenador portátil) se conecten a la misma red. A través de esta conexión, es posible establecer una sesión segura mediante el protocolo Secure Shell (SSH) con la RPi5, lo que permite el control total del robot, el lanzamiento de nodos de ROS 2 y la visualización de datos en tiempo real desde el ordenador externo sin necesidad de una conexión física.



Figura 4.9: Módulo WiFi TP-Link TL-WR802N utilizado en el robot. Imagen tomada de [41]

4.1.6. Sistema de Alimentación

Un sistema de alimentación robusto y estable es un pilar fundamental para el correcto funcionamiento de cualquier robot móvil. Es responsable de proporcionar la energía necesaria a todos los componentes, desde la unidad de procesamiento de alto nivel hasta los potentes motores de tracción. El diseño del sistema de alimentación de este robot se ha centrado en la portabilidad, la entrega de potencia adecuada y, de forma crucial, la protección de los componentes electrónicos sensibles.

Fuente de Alimentación Principal: Batería LiPo

Para garantizar la autonomía y movilidad del robot, se ha seleccionado una batería LiPo. Este tipo de baterías es el estándar en robótica y modelismo debido a su alta densidad energética (gran capacidad en poco peso) y su elevada tasa de descarga, que es capaz de suministrar las altas corrientes que demandan los motores en picos de aceleración.

El modelo específico es una batería Tattu 4S 14.8V 3700mAh 25C [13]. El término “4S” indica que está compuesta por cuatro celdas conectadas en serie, resultando en una tensión nominal de 14.8V (y una tensión máxima de 16.8V con carga completa). En la Tabla 4.2 se resumen sus características principales.



Figura 4.10: Batería LiPo de 4 celdas Gens Ace Tattu (14.8V, 3700mAh). Imagen adaptada de [13].

Tabla 4.2: Especificaciones de la Batería Tattu 4S 3700mAh.

Parámetro	Valor
Tipo de Química	Polímero de Litio (LiPo)
Configuración de Celdas	4S1P (4 en Serie, 1 en Paralelo)
Voltaje Nominal	14.8 V
Voltaje (Carga Completa)	16.8 V
Capacidad	3700 mAh
Tasa de Descarga (Continua)	25C (Equivalente a 150 A)
Conector de Descarga	XT60

Distribución de Potencia y Regulación

Dado que tanto la RPi5 como los controladores de motor RoboClaw 2x7A comparten la misma fuente de alimentación, se han implementado medidas de protección para evitar sobretensiones indeseadas, especialmente provocadas por tensiones regenerativas generadas por los motores durante las frenadas o la inercia. Este tipo de tensiones puede dañar componentes sensibles, como la Raspberry Pi, si no se manejan adecuadamente.

Alimentación de la Unidad de Procesamiento

La Raspberry Pi 5 requiere una alimentación estable de 5V y una corriente significativa (hasta 5A). Dado que la tensión de la batería (14.8V-16.8V) es mucho mayor, se utiliza una placa de expansión de potencia específica: la *Yahboom Power Supply Board for Raspberry Pi 5* [44]. Se eligió esta placa específica por su capacidad para entregar la alta corriente requerida por la RPi5 y por su diseño que se acopla directamente a los puertos de entrada/salida, ofreciendo una solución de alimentación limpia y compacta.

Esta placa actúa como un regulador de tensión de tipo "buck" o reductor, convirtiendo de manera eficiente la alta tensión de la batería a los 5V necesarios. Se eligió esta solución por su capacidad para manejar un amplio rango de tensiones de entrada y por proporcionar la corriente necesaria para alimentar de forma segura la RPi5 y sus periféricos USB, como la cámara OAK-D Lite. La placa se monta directamente sobre la Raspberry Pi, simplificando el cableado. En la Figura 4.11 podemos ver los componentes y los diferentes tipos de conexiones de la placa.

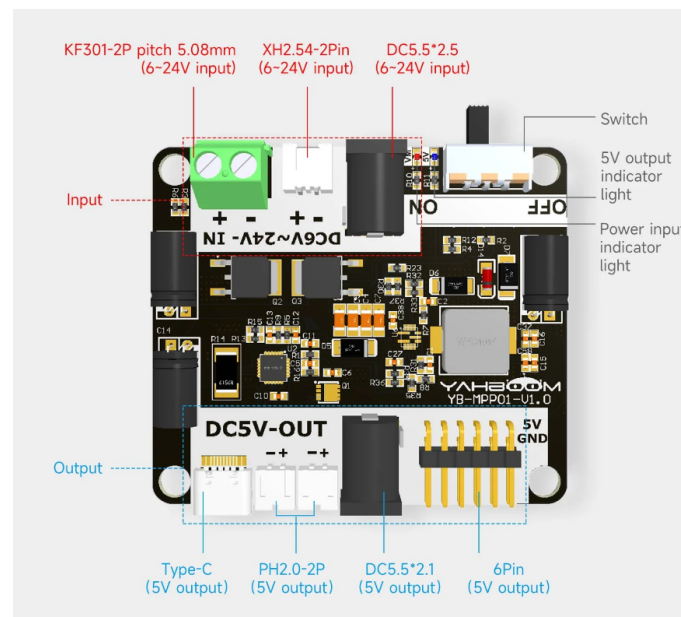


Figura 4.11: Módulo de expansión para RPi5. Imagen adaptada de [44].

Alimentación de Controladores y Protección contra Regeneración

Los controladores de motor RoboClaw 2x7A se alimentan directamente de la tensión de la batería LiPo para poder suministrar la potencia requerida por los motores. Sin embargo, una consideración crítica en los sistemas con motores DC es el fenómeno de la frenada regenerativa. Como hemos mencionado en el Capítulo 3 en la sección 3.2.2, al desacelerar, los motores actúan como generadores, inyectando un pico de tensión inverso en la línea de alimentación. Este proceso no es gradual; genera un pulso de corriente inverso, de alta energía y con una derivada de corriente muy elevada (dI/dt). Este pulso, al circular por la impedancia del sistema (incluida la inductancia de los cables), provoca un pico de tensión transitorio e instantáneo en la línea de alimentación. Este pico puede superar la tensión máxima soportada por los componentes electrónicos y dañarlos permanentemente.

El componente más vulnerable a este fenómeno es la placa de expansión de potencia Yahboom que alimenta la Raspberry Pi 5. Su regulador de conmutación está diseñado para operar con una tensión de entrada estable. Un transitorio de alta tensión, incluso de muy corta duración, puede superar su límite máximo de entrada y causar un fallo catastrófico e irreversible. Por tanto, el diseño de un sistema de protección robusto no es una opción, sino un requisito fundamental.

Para abordar este problema, se ha implementado una estrategia de protección de dos capas:

Capa 1: Mitigación en la Fuente (Limitación de Corriente Regenerativa). La primera línea de defensa es limitar la magnitud del pulso de corriente regenerativa en su origen. Esto se configura en el software *Motion Studio* de la RoboClaw como se ha mencionado en la sección 4.1.3, estableciendo el parámetro **Max Regen**. Tras un análisis del torque de los motores, basado en las especificaciones del motor goBILDA [14], se calculó una constante de torque aproximada de 2.64 kg-cm por amperio. Se ha establecido un valor conservador para el ‘Max Regen’ en el *Motion Studio* de **-1.0 A**. Este límite se traduce en un par de frenado máximo de 2.64 kg-cm (aproximadamente el 11 % del par máximo del motor). Esta configuración asegura una desaceleración suave pero controlada, priorizando la minimización del estrés eléctrico sobre el sistema. Limita de forma proactiva la magnitud del transitorio de tensión, actuando como la primera capa de protección para la electrónica sensible. Este ajuste no elimina la frenada, pero la “suaviza”, reduciendo la agresividad del transitorio y, por tanto, la magnitud del pico de tensión.

Capa 2: Supresión de Transitorios (Circuito *Voltage Clamp*). Esta es la red de seguridad principal. Para suprimir cualquier pico de tensión residual que supere el umbral de seguridad, se ha implementado un circuito de supresión de transitorios, comúnmente conocido como *Voltage Clamp*, siguiendo las directrices del manual de usuario de RoboClaw [4].

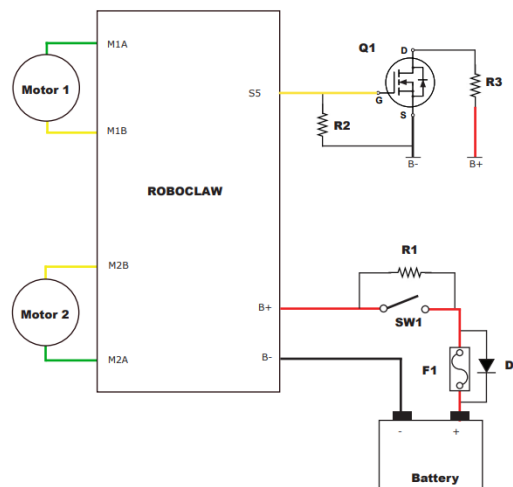


Figura 4.12: Diagrama del circuito *Voltage Clamp* para disipación de energía regenerativa. Adaptado del manual de usuario de RoboClaw [4].

Como se muestra en la Figura 4.12, cuando la RoboClaw detecta que la tensión del bus de alimentación excede un límite seguro, activa una señal en su pin de control (S5). Esta señal activa instantáneamente el MOSFET de potencia (**IRLB8721PbF** [19]) **Q1**, que actúa como un interruptor ultrarrápido. El MOSFET desvía la energía del transitorio a través de una resistencia de disipación de alta potencia (Vishay LTO050 de 10Ω y 50W [42]) **R3**, la cual convierte esta energía eléctrica no deseada en calor.

Este circuito recorta el pico de tensión, impidiendo que se propague por la línea de alimentación y llegue a la placa de la Raspberry Pi 5. La combinación de la limitación de corriente en la fuente y la supresión activa del transitorio proporciona una protección robusta y multinivel, garantizando la integridad del sistema electrónico del robot durante operaciones dinámicas.

4.1.7. Placa de Interfaz Personalizada para Controladores

La decisión de diseñar una Placa de Circuito Impreso (PCB) a medida en lugar de usar una *protoboard* se justifica por la fiabilidad, robustez y la optimización del espacio. Una PCB asegura conexiones eléctricas sólidas que no se desconectarán por las vibraciones del movimiento del robot, un problema común en prototipos. Además, permite integrar de forma compacta y profesional el circuito de protección *Voltage Clamp* y los conectores de los encoders, resultando en un sistema más modular y fácil de mantener. El propósito de esta placa es triple:

- **Albergar el circuito *Voltage Clamp*:** Los componentes del circuito de protección (MOSFET, resistencias, conector de potencia) están soldados directamente en la placa, creando una solución compacta y fiable.
- **Centralizar las conexiones de los encoders:** La placa incluye conectores JST para recibir las señales de los dos encoders de los motores (Canal A, Canal B, 5V, GND) y enrutarlas a los pines correspondientes de la RoboClaw.
- **Simplificar el cableado:** Proporciona conectores Dupont estandarizados para la comunicación con la RPi5 y otras señales, evitando el uso de cables sueltos y propensos a errores de conexión.

A continuación, se detallan y justifican los componentes clave seleccionados para el circuito de protección *Voltage Clamp* integrado en esta placa:

- **MOSFET de Potencia (IRLB8721PbF):** Este es el componente que actúa como interruptor electrónico para desviar la corriente regenerativa. Se ha seleccionado el MOSFET de canal N IRLB8721PbF de International Rectifier [19]. Su elección se basa en las siguientes características críticas:
 - **Tensión de Ruptura Drenador-Fuente (V_{DSS}):** Soporta hasta 30V, un margen de seguridad suficiente por encima de la tensión máxima de la batería de 4S (16.8V).
 - **Capacidad de Corriente (I_D):** Puede manejar una corriente continua de hasta 62A (a 25°C), superando con creces los picos de corriente regenerativa esperados.
 - **Baja Resistencia de Encendido ($R_{DS(on)}$):** Presenta una resistencia muy baja (8.7 m Ω a $V_{GS} = 10V$), lo que minimiza la generación de calor en el propio MOSFET cuando está conduciendo.
 - **Nivel Lógico de Activación:** Una característica fundamental es que puede ser completamente activado con una señal de baja tensión (ej. 3.3V o 5V) como la que proporciona el pin de control de la RoboClaw.
- **Resistencia de Disipación (Vishay LTO050F10R00JTE3):** Es el elemento que convierte la energía eléctrica regenerada en calor. Se ha elegido el modelo LTO050F10R00JTE3 de Vishay [42], una resistencia de película gruesa diseñada para montaje en chasis. Sus especificaciones son:
 - **Potencia Nominal:** 50W, capacidad suficiente para disipar la energía generada durante frenadas bruscas sin sobrecalentarse o dañarse.

- **Resistencia:** 10Ω . Este valor ofrece un buen compromiso: es lo suficientemente bajo para permitir un flujo de corriente significativo y disipar la energía rápidamente, pero lo suficientemente alto para limitar la corriente a un valor seguro para el MOSFET y el resto del circuito.
- **Resistencia *Pull-Down* ($10k\Omega$):** Se incluye una resistencia de $10k\Omega$ conectada entre la puerta (*Gate*) del MOSFET y tierra (*Ground*). Su función es asegurar que la puerta del MOSFET se mantenga en un estado bajo (0V) cuando el pin de control de la RoboClaw no está activo, evitando así activaciones accidentales del circuito de protección debido a ruido o a estados flotantes.
- **Conectores:** Se han seleccionado conectores adecuados para cada función: conectores JST para los encoders [21, 20] y conectores Dupont para las señales de bajo nivel [2, 3], garantizando conexiones seguras y estandarizadas.

En la Figura 4.13 se muestra un diagrama o fotografía de la placa de interfaz diseñada, donde se puede apreciar la disposición de estos componentes.

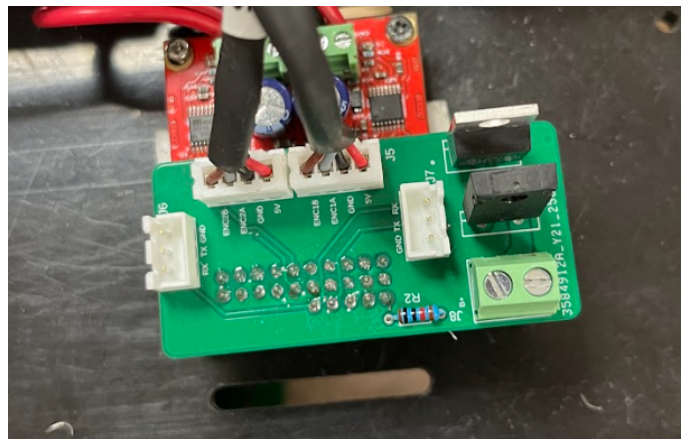


Figura 4.13: Placa de interfaz personalizada para la controladora RoboClaw. Integra el circuito de protección Voltage Clamp y los conectores de encoders.

4.2. Ensamblaje y Disposición de Componentes

El diseño modular del chasis ha permitido una disposición estratégica de los componentes para optimizar el espacio, el centro de masas y la accesibilidad para el mantenimiento. El ensamblaje final integra tanto la electrónica de potencia y control en la parte inferior del chasis, como los sistemas de percepción y procesamiento en la parte superior.

La Figura 4.14 ofrece una vista del interior del chasis, donde se aprecia la disposición de las dos controladoras de motor RoboClaw, las placas de interfaz personalizadas (que albergan los circuitos *Voltage Clamp*) y el compartimento central destinado a la batería. Esta ubicación de la batería, en la posición más baja y centrada posible, contribuye a mantener un centro de gravedad bajo, mejorando la estabilidad del robot.

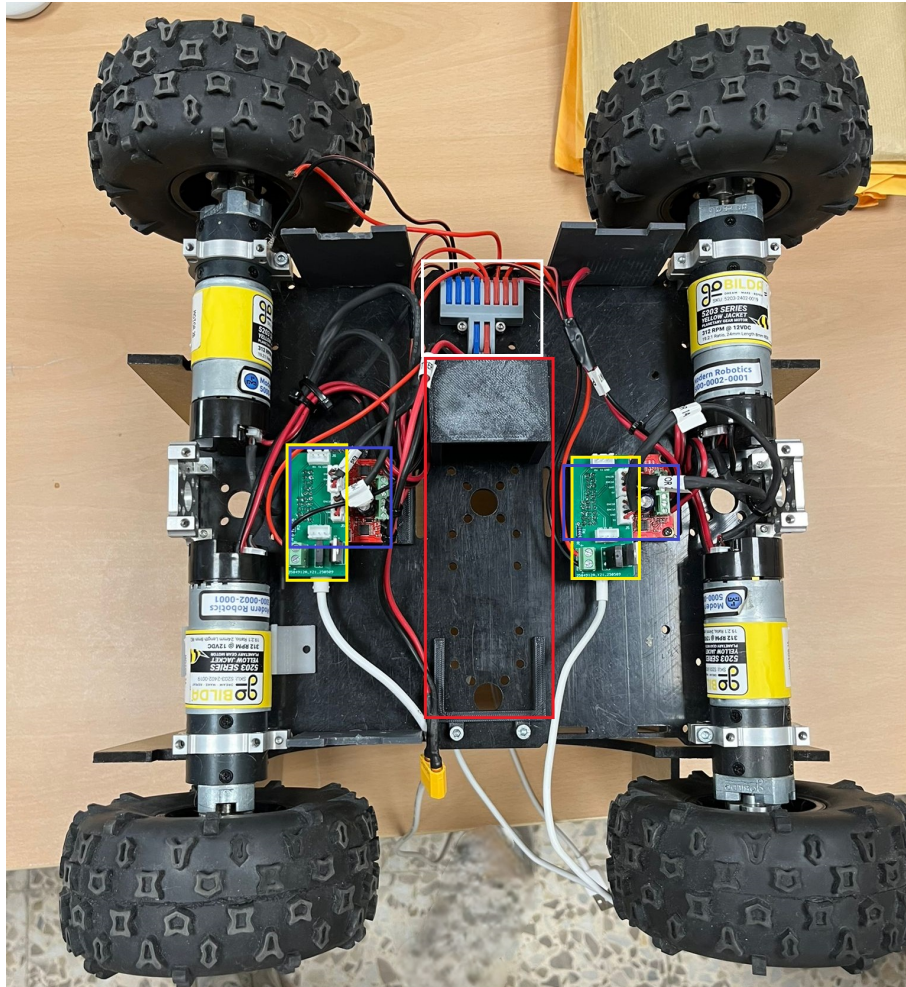


Figura 4.14: Disposición interna de la electrónica de potencia y control en la base del chasis.

En la imagen se han resaltado con colores los principales elementos:

- en amarillo, las placas PCB;
- en azul, las controladoras de motor RoboClaw (ubicadas debajo de las PCB);
- en rojo, el compartimento de la batería;
- en blanco, el módulo distribuidor.

Una vez ensamblada la electrónica interna, se completa la plataforma con los componentes de percepción y procesamiento. La Figura 4.15 muestra una vista del robot completamente ensamblado, con la Raspberry Pi 5 acoplada al módulo de expansión Yahboom, la cámara OAK-D Lite y la IMU externa montadas en una estructura superior. Para el montaje de varios de estos componentes, se diseñaron e imprimieron en 3D una serie de soportes a medida, cuyos planos detallados se pueden consultar en el Anexo B.

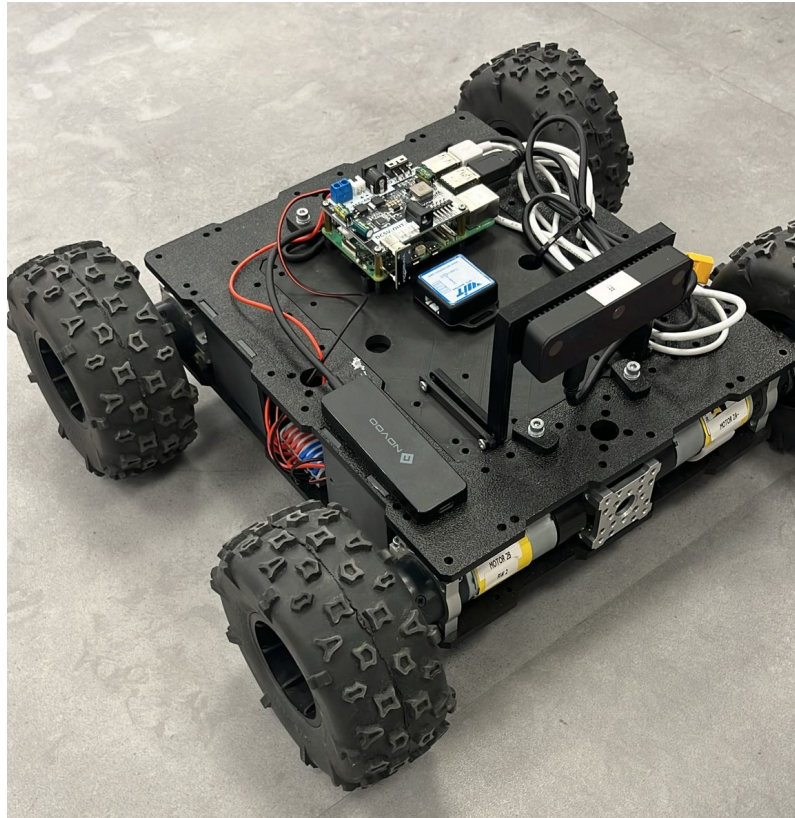


Figura 4.15: Vista del robot completamente ensamblado, mostrando la disposición final de todos los componentes de hardware.

Capítulo 5

Arquitectura de Software

Este capítulo detalla la arquitectura del software desarrollado para el robot, implementada sobre el *framework* ROS 2 Jazzy Jalisco. Se comienza con una introducción a conceptos básicos y, a continuación, se presenta el árbol de transformaciones espaciales, un pilar en la robótica móvil. Luego, se proporciona una descripción de la configuración del espacio de trabajo (*workspace*), que organiza todos los paquetes de *software*. Finalmente, se profundiza en la implementación de los nodos de control y percepción.

5.1. Conceptos Fundamentales de ROS 2

Para el desarrollo del software se seleccionó la distribución ROS 2 Jazzy Jalisco, ya que es la versión de ROS 2 oficialmente soportada en Ubuntu 24.04 LTS. De este modo, se asegura la coherencia entre el sistema operativo y el *framework*, así como la disponibilidad de actualizaciones y soporte durante todo el ciclo de vida del proyecto.

ROS 2 es un conjunto de librerías y herramientas de software de código abierto para el desarrollo de aplicaciones robóticas. A diferencia de su predecesor, su arquitectura de comunicación se basa en el estándar industrial *Data Distribution Service (DDS)*, lo que elimina la necesidad de un nodo maestro central y mejora la robustez y el rendimiento en tiempo real [30]. Para entender la arquitectura de este TFG, es necesario conocer los siguientes conceptos:

- **Nodos:** Son las unidades ejecutables del sistema. Cada nodo es un proceso responsable de una tarea específica (ej. controlar los motores, leer datos de la cámara, planificar una ruta). En este proyecto, el `robot_driver_node`, `rtabmap` y los servidores de Nav2 son ejemplos de nodos.
- **Tópicos:** Son los canales de comunicación a través de los cuales los nodos intercambian datos. Los nodos pueden publicar mensajes en un tópico o suscribirse a un tópico para recibir mensajes. Por ejemplo, el nodo de navegación publica comandos de velocidad en el tópico `/cmd_vel`, y el `robot_driver_node` se suscribe a él.
- **Servicios:** Mientras que los tópicos son para flujos de datos continuos, los servicios se utilizan para una comunicación de tipo petición/respuesta síncrona. Un nodo cliente envía una petición a un nodo servidor y espera bloqueado hasta recibir una respuesta. Son útiles para tareas que deben completarse antes de continuar, como activar un sensor o solicitar un mapa.
- **Acciones:** Son similares a los servicios, pero están diseñadas para tareas de larga duración que requieren retroalimentación y la posibilidad de ser canceladas. Una acción consta de tres partes: un objetivo, un flujo de retroalimentación y un resultado final. El cliente no se bloquea, sino que recibe actualizaciones periódicas sobre el progreso de la tarea. La navegación autónoma es el ejemplo canónico:

- **Objetivo:** El cliente envía el objetivo (X, Y, θ) al servidor de acciones de Nav2.

- **Retroalimentación:** Mientras el robot navega, el servidor envía la retroalimentación, como la distancia restante al objetivo.
- **Resultado:** Cuando el robot llega (o falla), el servidor envía un resultado final.

El uso de acciones es fundamental para la interacción con Nav2.

- **Transformaciones (TF2):** La gestión de las relaciones espaciales entre los diferentes componentes de un robot (chasis, ruedas, sensores) es un problema fundamental. ROS 2 utiliza la librería *Transform Library v2 (TF2)* para gestionar un árbol de transformaciones en tiempo real [10]. TF2 permite a cualquier nodo preguntar por la posición y orientación de un sistema de referencia con respecto a cualquier otro en cualquier momento, simplificando enormemente los cálculos geométricos.

5.2. Árbol de Transformaciones

El correcto funcionamiento de los sistemas de mapeo, localización y navegación depende de un árbol de transformaciones bien definido. La Figura 5.1 ilustra la estructura que se ha implementado en este proyecto.

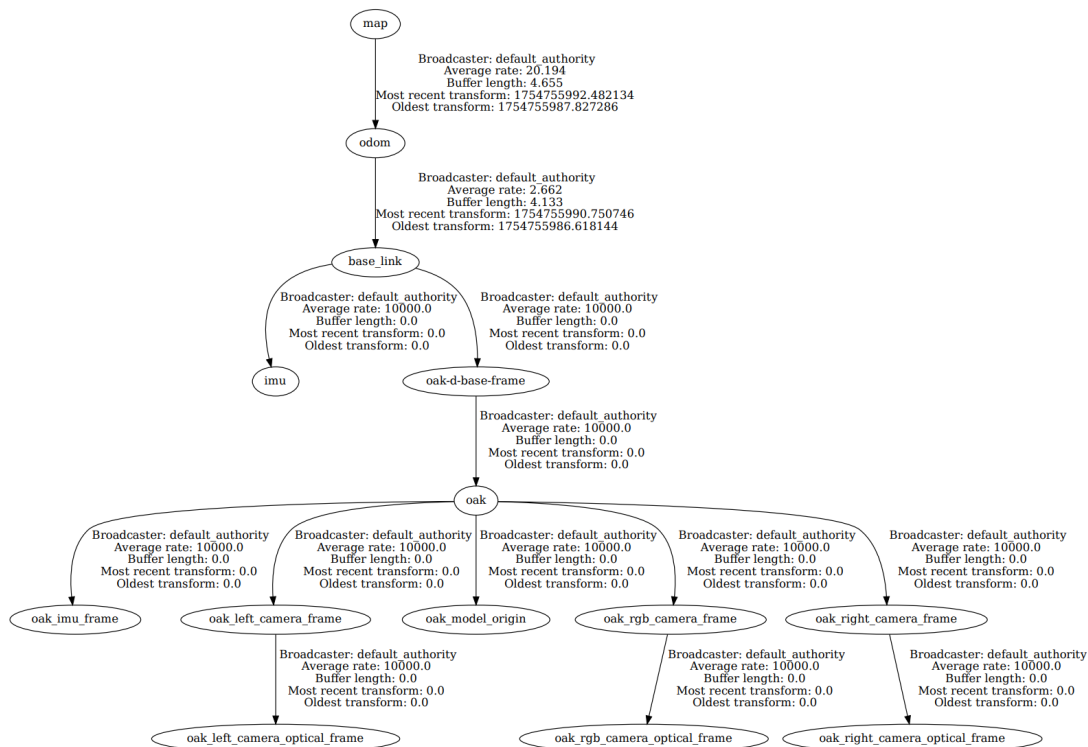


Figura 5.1: Diagrama del árbol de transformaciones del robot. Muestra la relación jerárquica entre los sistemas de referencias principales.

Los sistemas de referencia principales (*frames*) y las transformaciones que los conectan son:

- **map → odom:** Esta es la transformación que corrige la deriva. Es publicada por el sistema de SLAM o localización (en nuestro caso, **RTAB-Map**). Proporciona la posición del origen del sistema de referencia de la odometría (*odom*) con respecto al mapa global (*map*).
- **odom → base_link:** Representa la estimación continua, pero con deriva, de la pose del robot. Es publicada por una fuente de odometría. En este proyecto, la fuente principal es el nodo de odometría visual-inercial de **RTAB-Map**. El nodo *robot_driver_node* también publica una

odometría de ruedas en el tópico `/odom/wheel`, que puede ser usada como entrada auxiliar. También podemos usar la publicada por el Filtro de Kalman si deseamos fusionar todos los sensores.

- **base_link** → **[sensores]**: Son transformaciones estáticas que definen la geometría física del robot. Se publican una sola vez y describen la posición y orientación de los sensores respecto al centro del robot (**base_link**). Las más relevantes en este proyecto son:
 - **base_link** → **oak-d-base-frame** (para la cámara OAK-D Lite).
 - **base_link** → **imu** (para la IMU externa).

5.3. Configuración del Espacio de Trabajo (*Workspace*)

El desarrollo de software en ROS 2 se organiza en *workspaces*, que son directorios que contienen los paquetes de software del proyecto. Para este TFG, se ha creado un *workspace* principal llamado **Robot**. Este aloja tanto el paquete personalizado desarrollado para el control del robot como los paquetes de terceros (*drivers*) necesarios para interactuar con el *hardware*. Este enfoque de compilación desde la fuente asegura la total compatibilidad entre los paquetes y la versión de ROS 2 utilizada (Jazzy Jalisco).

El código fuente completo, junto con los archivos de lanzamiento y configuración, está disponible públicamente en un repositorio de GitHub para su consulta y reutilización ¹.

La estructura del directorio de fuentes (**Robot/src/**) contiene los siguientes paquetes clave, como se ilustra en la Figura 5.2:

```

luis@luis-desktop:~/Robot/src$ tree -d -L 2
.
├── depthai-ros
│   ├── depthai_bridge
│   ├── depthai_descriptions
│   ├── depthai_examples
│   ├── depthai_filters
│   ├── depthai-ros
│   ├── depthai_ros_driver
│   └── depthai_ros_msgs
├── robot_driver
│   ├── config
│   ├── launch
│   ├── mapas
│   ├── resource
│   ├── robot_driver
│   └── test
├── teleop_twist_qt
│   ├── images
│   └── src
└── witmotion_ros
    ├── config
    ├── include
    ├── launch
    ├── src
    └── witmotion-uart-qt

25 directories
  
```

Figura 5.2: Diagrama de la estructura de paquetes dentro del *workspace* Robot.

¹El repositorio se encuentra en: <https://github.com/lucastillo3420/Robot>

Paquete Personalizado: robot_driver

Este es el paquete principal desarrollado específicamente para este TFG. Contiene la lógica fundamental que actúa como la capa de abstracción de *hardware* del robot. Su responsabilidad es traducir los comandos abstractos de ROS 2 en acciones concretas del hardware y, a la inversa, leer los datos del hardware y publicarlos como información estándar en ROS 2. El nodo principal de este paquete, `robot_driver_node`, se describirá en detalle en la Sección 5.4.

Driver de la Cámara: depthai-ros

Para la integración de la cámara OAK-D Lite, se utiliza el paquete oficial de Luxonis, `depthai-ros` [28]. Este paquete, y en particular su nodo principal `depthai_ros_driver`, se encarga de interactuar con la API *DepthAI* para configurar la cámara y publicar sus flujos de datos (imágenes RGB, mapas de profundidad) como tópicos estándar en ROS 2. Incluirla en el *workspace* permite una gestión unificada de todo el *software* y facilita la personalización de sus archivos de configuración. Las configuraciones con respecto a la cámara se explicarán en detalle en la Sección 5.5.

Driver de la IMU: witmotion_ros

La comunicación con la IMU externa WT901C se gestiona a través del paquete de código abierto `witmotion_ros` [1]. Su nodo *driver* se encarga de leer los datos del puerto serie (UART), decodificar el protocolo del sensor y publicar la información de aceleración, velocidad angular y orientación en el tópico `/imu` con el formato estándar `sensor_msgs/Imu`.

Herramienta de Teleoperación: teleop_twistqt

Para el control manual del robot, especialmente durante la fase de mapeo, se incluye el paquete `teleop_twistqt` [31]. Este paquete proporciona un nodo con una sencilla interfaz gráfica (basada en QT) que permite enviar comandos de velocidad `geometry_msgs/Twist` al tópico `/cmd_vel` utilizando un *joystick* virtual.

5.4. Arquitectura de Bajo Nivel

La arquitectura de bajo nivel es gestionada por un nodo principal llamado `robot_driver_node`, que actúa como la interfaz directa entre el mundo del *software* de ROS 2 y el *hardware* físico del robot (las controladoras RoboClaw). Su diseño es modular y se apoya en varias clases especializadas, como se muestra en el diagrama de la Figura 5.3.

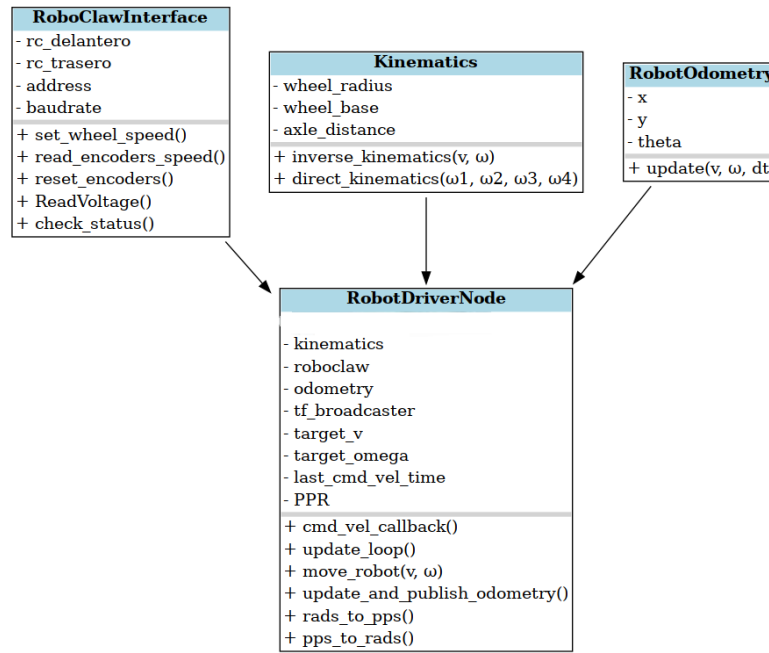


Figura 5.3: Diagrama de clases simplificado de la arquitectura del nodo RobotDriverNode.

La arquitectura interna del nodo realiza las siguientes funciones:

- **Interfaz de Hardware (RoboClawInterface):** Esta clase encapsula toda la complejidad de la comunicación con las controladoras RoboClaw. Gestiona la apertura de los puertos serie, la construcción de los comandos en formato *Packet Serial*, el envío de comandos de velocidad y la lectura de los encoders.
- **Modelo Cinemático (Kinematics):** Contiene la implementación de los modelos de cinemática directa e inversa para la plataforma de cuatro ruedas. Es utilizada para traducir los comandos de velocidad del robot a velocidades de las ruedas y viceversa.
- **Estimador de Odometría (RobotOdometry):** Esta clase mantiene el estado de la pose estimada del robot (X, Y, θ) a partir de la odometría de las ruedas. Integra las velocidades calculadas por la cinemática directa a lo largo del tiempo para actualizar la pose.

El nodo principal, **RobotDriverNode**, coordina estas clases en un bucle de control, asegurando una respuesta continua y fluida.

Recepción de Comandos de Velocidad

El nodo se suscribe al tópico estándar `/cmd_vel` (de tipo `geometry_msgs/Twist`). Los sistemas de alto nivel, como Nav2 o un nodo de teleoperación, publican en este tópico las velocidades lineales (v) y angulares (ω) deseadas para el robot.

Cálculo Cinemático y Comunicación con *Hardware*

Al recibir un comando de velocidad, el nodo realiza los siguientes pasos en su bucle principal:

1. Utiliza la clase **Kinematics** para aplicar la cinemática inversa, según las ecuaciones (3.6)-(3.7), convirtiendo el comando (v, ω) en las cuatro velocidades angulares deseadas para cada rueda $(\omega_1, \omega_2, \omega_3, \omega_4)$.
2. Convierte estas velocidades de rad/s a Pulsos por Segundo (pps), la unidad que entienden los encoders de la RoboClaw.

3. A través de la clase `RoboClawInterface`, se envían los comandos de velocidad en pps a las dos controladoras RoboClaw a través de sus respectivos puertos USB. Esta clase abstrae la complejidad del protocolo de comunicación *Packet Serial*.

Cálculo y Publicación de Odometría de Ruedas

Simultáneamente, el nodo es responsable de generar la odometría a partir de los encoders:

1. A través de la clase `RoboClawInterface`, lee las velocidades actuales de las ruedas en pps directamente desde los encoders.
2. Convierte estas velocidades de pps a rad/s.
3. Utiliza la clase `Kinematics` para aplicar la cinemática directa, según las ecuaciones (3.4)-(3.5), calculando la velocidad lineal y angular real del robot $(v_{real}, \omega_{real})$.
4. La clase `RobotOdometry` integra estas velocidades a lo largo del tiempo para calcular la pose estimada del robot (X, Y, θ) en el sistema de referencia `odom`.
5. Finalmente, publica esta información en el tópico `/odom/wheel` (de tipo `nav_msgs/Odometry`), para que pueda ser utilizada por otros nodos como RTAB-Map.

Este nodo centraliza toda la lógica de bajo nivel, proporcionando una abstracción limpia para que los sistemas de alto nivel puedan controlar el robot simplemente publicando en `/cmd_vel`, sin necesidad de conocer los detalles del *hardware* subyacente.

5.4.1. Modos de Operación

El robot opera principalmente en dos modos, dependiendo de la tarea que se esté realizando: teleoperación manual para mapeo y control autónomo para navegación.

Modo de Teleoperación para Mapeo

Para la fase inicial de creación del mapa, es necesario guiar al robot manualmente a través de todo el entorno para que el sistema SLAM pueda recopilar datos de todas las áreas. Para ello, se utiliza un nodo de teleoperación, como `teleop_twistqt` o `teleop_twistjoy`.

El flujo de datos en este modo es el siguiente:

1. El operador, utilizando un teclado o un joystick, ejecuta el nodo de teleoperación.
2. El nodo de teleoperación publica mensajes `geometry_msgs/Twist` en el tópico `/cmd_vel` según las teclas pulsadas.
3. El `robot_driver_node` se suscribe a `/cmd_vel`, recibe estos mensajes y los convierte en comandos de velocidad para las RoboClaw.

Modo de Navegación Autónoma

Una vez que se ha guardado un mapa, el robot puede navegar de forma autónoma. En este modo, el control del tópico `/cmd_vel` pasa del nodo de teleoperación al *stack* de navegación Nav2.

1. Un objetivo de navegación se envía al servidor de acciones de Nav2 (ej. a través de RViz2).
2. El **servidor controlador** de Nav2, tras consultar la ruta del planificador global y el mapa de costes local, comienza a publicar mensajes `geometry_msgs/Twist` en `/cmd_vel` para guiar al robot.
3. El `robot_driver_node` recibe estos comandos y los ejecuta exactamente igual que en el modo de teleoperación.

5.5. CONFIGURACIÓN DEL PIPELINE DE PERCEPCIÓN DE LA CÁMARA OAK-D LITE37

4. El sistema cierra el bucle, ya que la odometría publicada por el *driver* y la localización de RTAB-Map alimentan continuamente a Nav2 para que pueda ajustar su planificación en tiempo real.

La transición entre estos dos modos se gestiona simplemente ejecutando el nodo de teleoperación o el *stack* de Nav2, ya que ambos utilizan la misma interfaz estandarizada (*/cmd_vel*) para controlar el robot.

5.5. Configuración del Pipeline de Percepción de la Cámara OAK-D Lite

La percepción del robot depende enteramente del paquete `depthai-ros` [28]. El archivo de lanzamiento que vamos a usar de este paquete es `camera.launch.py`; es altamente modular y se configura a través de un archivo de parámetros `camera.yaml`.

Una consideración fundamental de diseño es la limitación computacional de la cámara. Aunque su VPU (modelo RVC2) es potente para el procesamiento de imágenes y la ejecución de redes neuronales, no tiene la capacidad para ejecutar el algoritmo de procesamiento de imágenes completamente. Por tanto, el algoritmo debe ejecutarse en el *'host'*, es decir, en la Raspberry Pi 5. Esto convierte a la RPi5 en el principal cuello de botella computacional del sistema y hace que la optimización de la carga de trabajo sea crítica.

El proceso de rectificación de imágenes estéreo es gestionado por el nodo `image_proc`. Es una tarea intensiva que, si no se controla, puede saturar la Unidad de Procesamiento Central (CPU) de la RPi5. Para asegurar un rendimiento estable del SLAM, se ha realizado un compromiso deliberado entre la calidad de la imagen y la carga computacional, configurando los siguientes parámetros en el archivo `camera.yaml`:

- **Resolución y Frecuencia:** Se ha reducido la resolución de los flujos de imagen RGB y de profundidad de **1080x720** a **640x480** y la frecuencia de **30 FPS** a **15 FPS**. Esta configuración reduce drásticamente la carga sobre la CPU, permitiendo que el nodo `image_proc` realice la rectificación sin causar una latencia excesiva o la pérdida de mensajes, lo cual sería catastrófico para la odometría visual. Esta decisión de reducir la resolución y frecuencia de captura se validó posteriormente en las pruebas experimentales (véase Capítulo 7 sección 7.4), donde se observa cómo la carga de la CPU disminuye considerablemente, permitiendo un funcionamiento estable.
- **Publicación de TF desde Calibración:** El parámetro `i_publish_tf_from_calibration` se ha establecido en `true`. Esta es una configuración crucial, ya que instruye al *driver* para que publique las transformaciones internas de la cámara (la posición relativa entre las cámaras estéreo izquierda/derecha y la cámara RGB), que son leídas del fichero de calibración de fábrica del dispositivo. RTAB-Map necesita esta información geométrica precisa para funcionar correctamente.

5.5.1. Nodos y tópicos clave del pipeline de percepción

Al ejecutar el archivo de lanzamiento `camera.launch.py`, no solo se inicia un único nodo, sino un conjunto de procesos que trabajan coordinadamente para procesar y publicar los datos de la cámara. Los nodos principales que se inician, como se observa en la lista de nodos de ROS 2, son:

- `/oak`: El nodo principal del *driver* `depthai_ros_driver`, responsable de la comunicación directa con el *hardware* de la cámara OAK-D Lite.
- `/oak_container`: Un contenedor de nodos componibles. En ROS 2, esto permite que varios nodos se ejecuten en un mismo proceso, optimizando la comunicación al evitar copias de memoria innecesarias entre ellos.
- `/rectify_color_node`: Un nodo del paquete estándar `image_proc`. Su función es suscribirse a la imagen en crudo (`/image_raw`) y a la información de calibración (`/camera_info`) de

la cámara RGB, para luego publicar una imagen rectificada (`/image_rect`), es decir, una imagen sin distorsiones de la lente.

- `/oak_state_publisher`: Nodo que publica la descripción física de la cámara y sus transformaciones estáticas internas.

Este conjunto de nodos publica una gran cantidad de tópicos. Sin embargo, para las tareas de SLAM y navegación, solo un subconjunto de ellos es fundamental. Estos tópicos, resumidos en la Tabla 5.1, constituyen la interfaz de datos principal que el sistema de percepción ofrece al resto del robot.

Tabla 5.1: Tópicos esenciales publicados por el pipeline de la cámara y su uso en el proyecto.

Tópico	Tipo de Mensaje	Descripción y Uso en el Proyecto
<code>/oak/rgb/image_rect</code>	<code>sensor_msgs/Image</code>	Imagen a color rectificada (sin distorsión). Entrada visual principal para RTAB-Map.
<code>/oak/rgb/camera_info</code>	<code>sensor_msgs/CameraInfo</code>	Parámetros intrínsecos de la cámara RGB (matriz de calibración, distorsión). Indispensable para que RTAB-Map interprete la geometría de la imagen.
<code>/oak/stereo/image_raw</code>	<code>sensor_msgs/Image</code>	Imagen de profundidad. Cada píxel contiene el valor de la distancia en metros. Entrada de profundidad principal para RTAB-Map.
<code>/tf</code> y <code>/tf_static</code>	<code>tf2_msgs/TFMessage</code>	Árbol de transformaciones. El <i>driver</i> publica aquí la geometría interna de la cámara (ej. la pose de la cámara RGB respecto a la estéreo), crucial para la fusión de datos.

Como se detalla en la Tabla 5.1, RTAB-Map, se suscribe principalmente a los tópicos de imagen de color rectificada y de profundidad, junto con sus respectivos mensajes de `/cámara_info`. La correcta sincronización entre estos tópicos es fundamental y es una de las razones por las que se ha ajustado la frecuencia de publicación a 15 FPS para asegurar que la RPi5 pueda procesar cada par de imágenes (color + profundidad) sin retrasos ni pérdidas.

Capítulo 6

Control de Alto Nivel: Mapeo y Navegación

Este capítulo detalla la implementación y el funcionamiento del software de alto nivel del robot, desarrollado en ROS 2. Se divide en dos funcionalidades principales: primero, el proceso de mapeado del entorno utilizando RTAB-Map en una configuración 2.5D; y segundo, la navegación autónoma utilizando el mapa generado y el *stack* Nav2.

6.1. Mapeo y Localización con RTAB-Map

El primer paso para la autonomía del robot es la creación de un mapa preciso del entorno. Aunque el sistema de percepción se basa en una cámara RGB-D que captura información 3D, el objetivo final es generar un mapa de ocupación de rejilla 2D (*2D occupancy grid map*). Este tipo de mapa se considera el formato adecuado para este proyecto, ya que por el momento no resulta necesario trabajar con mapas 3D.

El proceso de mapeo se inicia ejecutando el archivo de lanzamiento `rtabmap.MapaVisual.py`, que orquesta todos los nodos necesarios. La fase de recolección de datos se realiza guiando al robot manualmente por el entorno mediante teleoperación, asegurando que la cámara explore visualmente todas las áreas de interés.

6.1.1. Arquitectura de nodos durante el mapeo.

La Figura 6.1 muestra el grafo de nodos y tópicos generado por `rqt_graph` durante la ejecución del proceso de mapeo. Este diagrama ilustra el flujo de información entre los componentes clave del sistema.

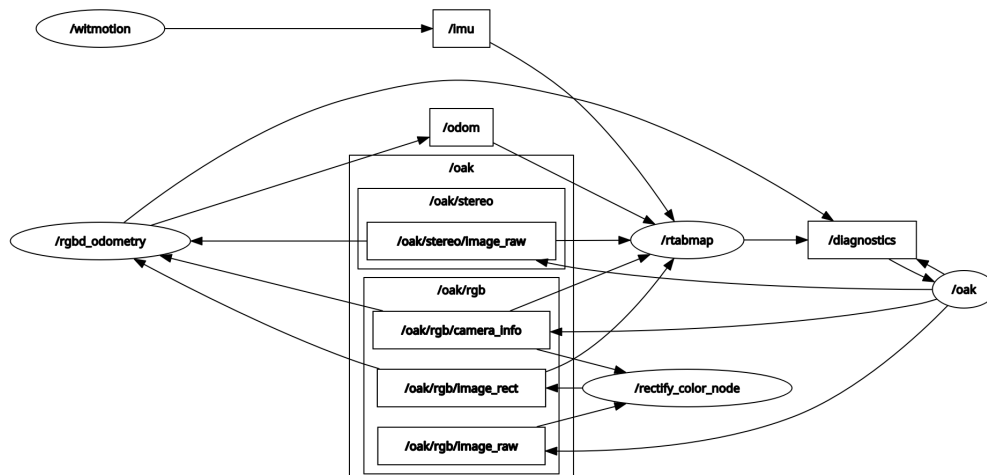


Figura 6.1: Grafo de nodos y tópicos de ROS 2 durante el proceso de mapeo con RTAB-Map.

El flujo de datos principal es el siguiente:

1. Los *drivers* de *hardware* publican los datos crudos: el nodo `/oak` publica los flujos de imagen de la cámara (`/oak/rgb/image_raw`, `/oak/stereo/image_raw`) y el nodo `/witmotion` publica los datos de la IMU (`/imu`).
2. El nodo `/rectify_color_node` procesa la imagen RGB cruda para corregir distorsiones, publicando la imagen rectificada en `/oak/rgb/image_rect`.
3. El nodo de odometría visual de RTAB-Map, `/rgbd_odometry`, se suscribe a la imagen rectificada, la imagen de profundidad y los datos de la IMU. Fusiona esta información para generar una estimación de odometría continua pero con deriva, que publica en el tópico `/odom`.
4. El nodo principal de SLAM, `/rtabmap`, utiliza la misma información sensorial (imágenes, profundidad, IMU) y la odometría del nodo `/rgbd_odometry` para construir el grafo de poses, detectar cierres de bucle y generar el mapa del entorno.

6.1.2. Configuración de RTAB-Map para Mapeo 2.5D

RTAB-Map es un sistema de SLAM 3D por naturaleza. Sin embargo, para la navegación 2D, se ha configurado específicamente para operar en un modo que se puede denominar 2.5D. En este modo, el sistema sigue utilizando toda la información 3D de la nube de puntos para la odometría y la detección de cierres de bucle, pero proyecta esta información en un plano para generar un mapa de ocupación 2D y produce un mapa compatible con los planificadores estándar.

Esta configuración se logra mediante los siguientes parámetros clave en el archivo de lanzamiento:

- **Reg/Force3DoF: "true"**: Este es el parámetro más importante. Fuerza a que la optimización del grafo de SLAM se realice solo en 3 grados de libertad (x, y, yaw), asumiendo que el robot se mueve en un plano. Esto previene la acumulación de errores en altura (eje z), inclinación (*pitch*) y alabeo (*roll*).
- **Grid/3D: "false"**: Desactiva explícitamente la generación de un mapa 3D (OctoMap), ahorrando una cantidad significativa de recursos computacionales.
- **Grid/FromDepth: "true"**: Indica al sistema que el mapa de rejilla 2D debe generarse a partir de la proyección de la nube de puntos 3D obtenida de la imagen de profundidad.
- **Grid/MaxObstacleHeight: "0.4"**: Durante la proyección de la nube de puntos 3D al plano 2D, este parámetro instruye al sistema para que ignore todos los puntos que estén a más de 0.4 metros por encima del suelo del robot. Esto es crucial para evitar que elementos como

techos, lámparas o la parte superior de las estanterías se proyecten como obstáculos en el mapa de navegación.

- **Grid/MaxGroundHeight: "0.10"**: De forma similar, se ignoran los puntos que estén por debajo de 0.10 metros del suelo del robot, lo que ayuda a filtrar ruido del suelo y pequeñas imperfecciones del terreno.

La Figura 6.2 muestra un mapa del entorno del laboratorio generado mediante el proceso descrito. El mapa es una rejilla de ocupación donde se pueden distinguir claramente los diferentes elementos:

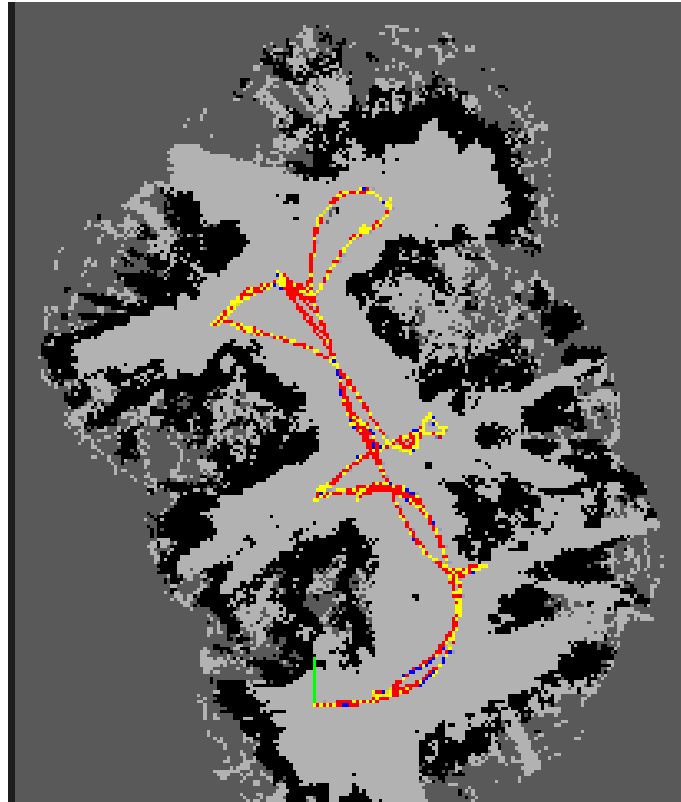


Figura 6.2: Mapa de ocupación 2D del entorno del laboratorio generado con RTAB-Map.

- **Zonas Grises Claras:** Representan el espacio libre explorado por el robot, por donde se pueden planificar trayectorias.
- **Zonas Negras:** Corresponden a los obstáculos detectados con alta probabilidad, como paredes, columnas o mobiliario. La apariencia a veces “engrosada” o “ruidosa” de los bordes es característica del VSLAM, ya que se deriva de la proyección de una nube de puntos 3D que puede contener pequeñas imprecisiones.
- **Zonas Grises Oscuras:** Es el espacio desconocido, áreas que el robot no ha podido observar, ya sea porque están fuera de su alcance u ocultas detrás de otro objeto.
- **Líneas Rojas, Amarillas y Azules (Superpuestas):** Representan el grafo de poses de RTAB-Map. Las líneas rojas y amarillas indican las conexiones de odometría (enlaces secuenciales entre poses), mientras que las líneas azules indican los cierres de bucle detectados, que son las correcciones cruciales que permiten obtener un mapa globalmente consistente.

El resultado es un mapa topológicamente correcto y métricamente preciso, adecuado para ser utilizado por el sistema de navegación Nav2, que se describe en la siguiente sección.

6.1.3. Modo de Localización Pura

Una vez que se dispone de un mapa, el robot ya no necesita mapear, sino únicamente localizarse dentro de ese mapa conocido. Para esta tarea, se ejecuta RTAB-Map en modo de localización pura. Este es el modo que se utiliza durante la fase de navegación autónoma con Nav2.

Principio de Funcionamiento: Localización Basada en Apariencia

En el modo de localización, el proceso es conceptualmente diferente al de SLAM. El sistema carga la base de datos del mapa previamente generado, pero no la modifica. Al recibir nuevos datos de la cámara y la IMU, el sistema realiza los siguientes pasos:

1. Estimación de Odometría: El nodo `rgbd_odometry` sigue funcionando, proporcionando una estimación de movimiento a corto plazo (`odom` $\rightarrow$ `base_link`). Esta estimación, como toda odometría, acumula deriva con el tiempo.
2. Localización: El nodo `rtabmap` compara la apariencia de la imagen actual (usando su firma BoW) con la base de datos de imágenes del mapa. Cuando encuentra una coincidencia fuerte ocurre un evento de localización.
3. Corrección de la Pose: Al localizarse, RTAB-Map calcula la pose global correcta del robot dentro del mapa y publica la transformación `map` $\rightarrow$ `odom`. Esta transformación actúa como una corrección que resetea la deriva acumulada en la odometría.

Este enfoque de localización, basado en el reconocimiento de lugares, es extremadamente robusto. A diferencia de AMCL, que puede perderse si la odometría deriva demasiado, RTAB-Map puede recuperarse del denominado **problema del robot reubicado** (si el robot se levanta y se mueve a otro lugar) simplemente reconociendo visualmente dónde se encuentra.

Configuración y Justificación frente a AMCL

La transición del modo SLAM al modo de localización se gestiona con un archivo de lanzamiento diferente, con los siguientes cambios de parámetros clave:

- `Mem/IncrementalMemory: "false"`: Este es el cambio fundamental. Le dice a RTAB-Map que no añada nuevas observaciones al mapa, forzándolo a operar únicamente en modo de localización.
- `delete_db_on_start: False`: Es crucial establecer este parámetro en falso para que el sistema utilice la base de datos del mapa previamente guardado en lugar de empezar con una vacía.

La elección de RTAB-Map para la localización, en lugar del estándar de ROS 2 AMCL, es una decisión de diseño deliberada y justificada. Como se mencionó en el capítulo 2, AMCL está diseñado para sensores de tipo `LaserScan` 2D. Utilizarlo con una cámara RGB-D requeriría proyectar la nube de puntos 3D a un “falso” escaneo 2D, un proceso que descarta la mayor parte de la rica información 3D y visual. Al usar RTAB-Map también para la localización, se aprovecha toda la información del sensor y la base de datos visual, creando un sistema de percepción más integrado, coherente y robusto.

6.2. Navegación Autónoma con Nav2

Una vez generado y guardado el mapa (Sección 6.1.1), podemos comenzar la fase de navegación autónoma. El robot debe ser capaz de desplazarse desde su posición actual hasta un punto objetivo (*goal*) de forma segura. Para esta tarea se utilizan los paquetes de navegación de ROS2 Nav2. El proceso se ejecuta lanzando el archivo `navegacion.launch.py`, que orquesta los nodos necesarios para la navegación.

6.2.1. Arquitectura del Sistema de Navegación

El archivo `navegacion.launch.py` pone en marcha la pila de navegación junto con elementos del *hardware* del robot. La configuración de toda la pila de navegación se define en un único archivo de parámetros `Parametros_Nav2.yaml`. Los nodos principales de Nav2 que se inician son:

- **map_server:** Carga el mapa 2D del entorno ('Mapa_Lab.yaml') y lo sirve al resto del sistema.
- **planner_server:** Gestiona los algoritmos de planificación global.
- **controller_server:** Gestiona los algoritmos de planificación local y control.
- **bt_navigator:** Coordina a todos los demás servidores utilizando un Árbol de Comportamiento.
- **lifecycle_manager:** Gestiona el ciclo de vida (activación, desactivación, limpieza) de todos los nodos de Nav2.

6.2.2. Configuración de Parámetros de Nav2

Costmap Global

El costmap global, utilizado para la planificación de rutas a largo plazo, se configura con dos capas principales:

- **static_layer:** Replica el mapa del entorno previamente generado, estableciendo los obstáculos fijos y conocidos, como las paredes.
- **inflation_layer:** Crea un área de seguridad de **0.30 metros** alrededor de todos los obstáculos. Este radio de inflación asegura que las rutas planificadas mantengan una distancia prudencial de las paredes, superior al radio físico del robot (0.25 m).

Costmap Local

El costmap local es una ventana móvil de 3×3 metros, centrada en el robot y utilizada para la evasión de obstáculos en tiempo real. Se configura con:

- **obstacle_layer:** Añade dinámicamente obstáculos al mapa utilizando los datos de la nube de puntos 3D (`/cloud_obstacles`) generada por la cámara OAK-D Lite, ignorando puntos por encima de 0.4 m de altura.
- **inflation_layer:** Aplica el mismo radio de inflación de **0.30 metros** a los obstáculos dinámicos para garantizar una evasión segura.

Planificadores y Controladores

Se han seleccionado los siguientes plugins para la gestión del movimiento:

- **Planificador Global (NavfnPlanner):** Se utiliza el planificador clásico de ROS 2 basado en el algoritmo de Dijkstra para generar la ruta óptima inicial en el mapa global. Se le permite trazar rutas a través de zonas desconocidas para tener mayor flexibilidad.
- **Controlador Local (RegulatedPurePursuitController):** Este plugin se encarga de seguir la ruta global. Es un algoritmo de seguimiento de caminos que calcula las velocidades del robot para interceptar un punto adelantado en la trayectoria. Se ha configurado para un movimiento suave y seguro, con una velocidad lineal máxima de 0.1 m/s y una velocidad rotacional máxima de 0.2 rad/s.
- **Navegador (BT Navigator):** Orquesta todo el proceso utilizando el árbol de comportamiento por defecto de Nav2, que incluye lógicas para planificar, seguir la ruta y recuperarse de fallos mediante la replanificación.

La Figura 6.3 muestra la vista de operación estándar, donde se visualizan los datos procesados por los diferentes nodos de Nav2.

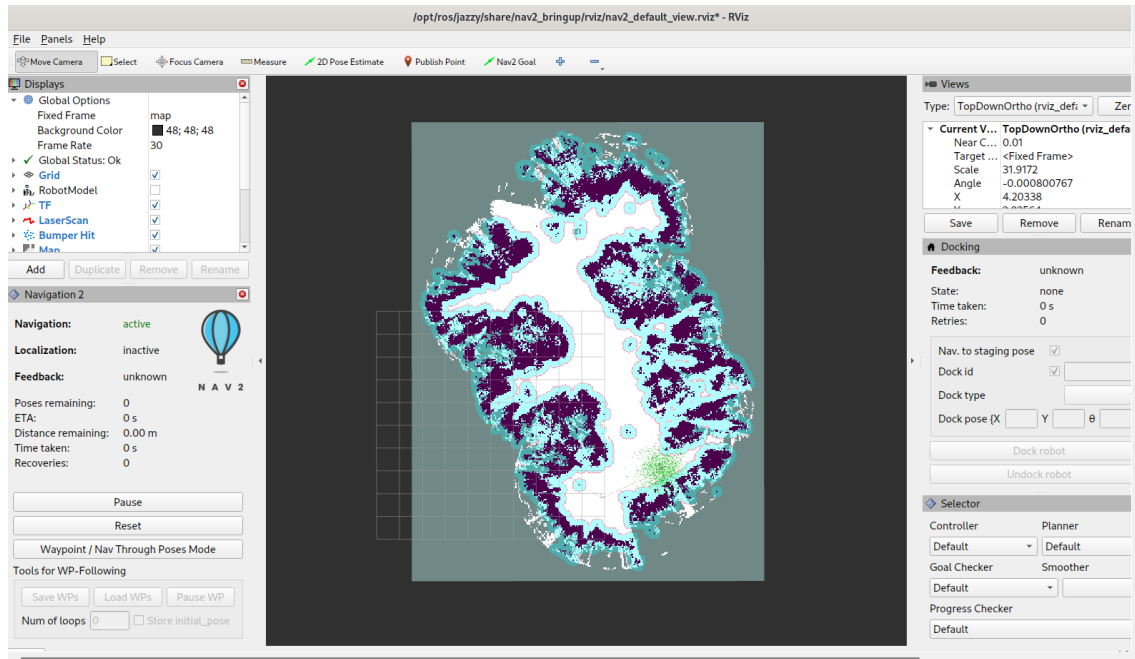


Figura 6.3: Interfaz de RViz2 durante la ejecución del *stack* de navegación Nav2, mostrando el mapa, el costmap global y la posición del robot.

En esta visualización, se pueden identificar los elementos clave del sistema en acción:

- **Mapa Estático (Gris y Negro):** El mapa cargado por el `map_server`.
- **Costmap Global (Púrpura y Cian):** La capa de coste utilizada por el planificador global. Las zonas púrpuras son los obstáculos donde el centro del robot no puede pasar, y las zonas cian representan la capa de inflación, que penaliza la cercanía a los obstáculos.
- **Interfaz de Nav2:** El panel de la izquierda permite activar el sistema (*Navigation active*), dar una estimación inicial de la pose (*2D Pose Estimate*) y enviar un objetivo de navegación (*Nav2 Goal*). Se puede apreciar que en esta captura el estado de *Localization* aparece como inactivo, un punto crítico que se analizará en el capítulo de resultados.

El flujo de operación consiste en localizar inicialmente al robot en el mapa y, a continuación, asignarle un objetivo. El sistema debería, entonces, calcular y visualizar una ruta y comenzar a moverse hacia el destino, actualizando su posición en tiempo real.

Capítulo 7

Pruebas, Resultados y Discusión

Este capítulo presenta el proceso experimental llevado a cabo para implementar y validar el sistema de SLAM y navegación. Esta sección detalla la evolución iterativa del sistema, mostrando los desafíos encontrados, las hipótesis planteadas y las soluciones aplicadas para alcanzar una configuración robusta y funcional. El objetivo es documentar el proceso de ingeniería y depuración llevado a cabo.

7.1. Metodología y Entorno de Pruebas

Las pruebas se llevaron a cabo utilizando la plataforma robótica descrita en el Capítulo 4, compuesta por una base diferencial de cuatro ruedas, una cámara OAK-D Lite, una IMU WitMotion y una Raspberry Pi 5 como unidad de procesamiento a bordo. El entorno principal donde se desarrollaron las pruebas fue en el laboratorio de robótica.

El laboratorio es un espacio interior estructurado con mobiliario de oficina, paredes lisas, buena iluminación y un suelo de textura uniforme. Este entorno representa un caso de uso ideal para VSLAM.

7.2. Pruebas de Bajo Nivel

Antes de integrar el sistema completo de SLAM y navegación, se realizaron una serie de pruebas a bajo nivel. El objetivo de estas pruebas era validar, de forma aislada y sin la complejidad de ROS 2, los componentes más fundamentales del robot: el control de los actuadores y la correcta implementación de los modelos cinemáticos. Al verificar que el robot puede ejecutar movimientos básicos de forma precisa basándose en su odometría interna, se establece una base sólida sobre la cual construir las capas de software de alto nivel.

7.2.1. Metodología de Prueba: Control por Coordenadas

Para estas pruebas, se desarrolló un script de python autónomo, `control_por_coordenadas.py`, que utiliza la librería `roboclaw_3` para comunicarse directamente con las controladoras de los motores. El script implementa una tarea de navegación punto a punto ('go-to-goal') basada en la siguiente lógica:

1. **Recepción de Objetivo:** El script solicita al usuario una coordenada objetivo (X_{obj}, Y_{obj}) y una velocidad máxima V_{max} .
2. **Fase 1: Rotación en el Eje:** El robot calcula el ángulo necesario para apuntar desde su posición actual (inicialmente $(0,0)$) hacia el punto objetivo. Utilizando la **cinemática inversa** (función `cin_inversa`), se comanda una rotación pura hasta que la orientación del robot, estimada por la odometría, coincide con el ángulo objetivo con una tolerancia definida.

3. **Fase 2: Avance Lineal:** Una vez alineado, el robot utiliza de nuevo la **cinemática inversa** para comandar un movimiento lineal hacia el objetivo. Durante el avance, se implementa un control de velocidad proporcional para desacelerar suavemente a medida que se acerca al destino, así como una pequeña corrección angular para mantener la trayectoria recta.
4. **Realimentación por Odometría:** Durante todo el proceso, se leen continuamente los encoders. Utilizando la **cinemática directa**, calcula el desplazamiento y el giro real del robot para actualizar su pose estimada (x, y, θ) . Esta pose es la que se utiliza como realimentación para los bucles de control de giro y avance.

El propósito de esta prueba es, por tanto, verificar que el robot se mueve hacia una posición deseada y que el ciclo completo de *Control* $\rightarrow$ *Movimiento* $\rightarrow$ *Lectura de Sensores* $\rightarrow$ *Odometría* funciona de forma coherente.

7.2.2. Resultados Experimentales

Se realizaron varias pruebas en un entorno de suelo plano para minimizar factores externos. En cada caso, se enviaron al robot unas coordenadas objetivo (X_{obj}, Y_{obj}) mediante el script de control. Una vez que el robot completaba su trayectoria y se detenía en la posición que su odometría interna consideraba correcta, se midió con una cinta métrica la posición real alcanzada por el robot desde el punto de partida hasta el centro de su base. La diferencia entre la coordenada objetivo y la posición real medida corresponde al **error absoluto** de navegación.

El error se calculó siguiendo la ecuación.

$$e = \sqrt{(x_{obj} - x_{real})^2 + (y_{obj} - y_{real})^2} \quad (7.1)$$

donde:

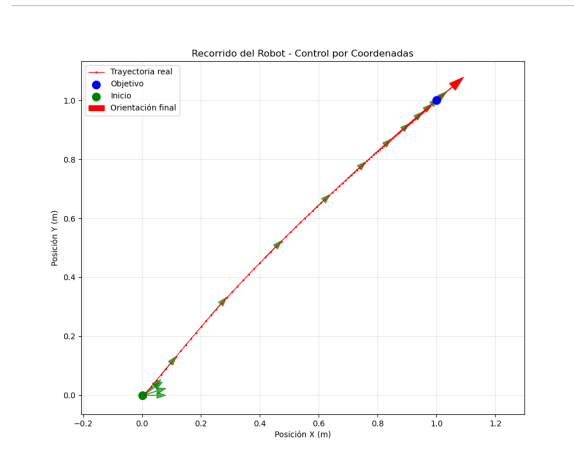
- (x_{obj}, y_{obj}) son las coordenadas objetivo,
- (x_{real}, y_{real}) son las coordenadas realmente alcanzadas por el robot,
- e es el error absoluto.

Tabla 7.1: Resultados de las pruebas de navegación a bajo nivel.

Prueba	Posición objetivo (m)	Posición final Medida (m)	Error (m)
1	(1.0, 0.0)	(1.04, 0.0)	0.04
2	(0.0, 1.0)	(0.04, 1.03)	0.05
3	(1.0, 1.0)	(1.03, 1.03)	0.042
4	(3.0, 0.0)	(3.08, 0.0)	0.8
5	(-1.0, 0.0)	(-1.04, -0.27)	0.27



(a) Robot durante la prueba en el entorno de suelo plano.



(b) Trayectoria estimada por la odometría para un objetivo en (1.0, 1.0).

Figura 7.1: (a) Robot en la prueba. (b) Trayectoria registrada por la odometría donde se observa la rotación inicial y el posterior avance lineal.

7.2.3. Análisis y Discusión de Resultados

Los resultados de las pruebas a bajo nivel son satisfactorios y permiten validar los modelos cinemáticos implementados. La Tabla 7.1 muestra que el error de posicionamiento final es de solo unos pocos centímetros, incluso para desplazamientos de más de dos metros. La Figura 7.1(b) confirma visualmente que el robot ejecuta la estrategia de movimiento esperada (girar y luego avanzar).

Es importante destacar que el error de posición aumenta con la distancia recorrida y la magnitud de los giros. Este comportamiento es característico de la deriva de odometría, un error acumulativo inherente a la estimación de la pose a partir de encoders de rueda, causado por factores como deslizamientos de las ruedas, imperfecciones de la superficie y tolerancias mecánicas.

En conclusión, estas pruebas demuestran que las funciones de cinemática inversa (para el control), cinemática directa y odometría (para la estimación de la pose) son correctas y funcionales. El error observado se debe a la deriva inherente del método, no a fallos en la implementación. Por tanto, se valida que los algoritmos desarrollados y probados en este script son una base robusta y fiable para ser integrados en el nodo `robot_driver_node` dentro de la arquitectura principal de ROS 2.

7.3. Desafíos de la Odometría y Fusión de Sensores

La base de cualquier sistema SLAM es una fuente de odometría fiable. En este proyecto se evaluaron dos enfoques principales: la fusión de encoders de rueda con la IMU y la odometría puramente visual-inercial VIO.

7.3.1. Pruebas con Fusión de Encoders e IMU

El primer enfoque consistió en utilizar la odometría de las ruedas, publicada por el nodo `robot_driver_node`, y fusionarla con los datos de la IMU externa mediante el filtro de Kalman del paquete `robot_localization`. El objetivo era obtener una odometría robusta que sirviera de entrada para RTAB-Map.

Sin embargo, las pruebas iniciales revelaron un problema fundamental: el deslizamiento de las ruedas. Al operar sobre superficies lisas como las del laboratorio, las aceleraciones y giros provocaban que las ruedas patinaran, generando un error significativo entre la rotación medida por los encoders y el desplazamiento real del robot. El resultado, como se puede observar en la Figura 7.2, fue una estimación de odometría completamente errática e inutilizable.

La flecha roja, que representa la odometría filtrada, no es correcta, demostrando que la odometría de ruedas no era una fuente fiable en este contexto. La flecha amarilla representa la orientación correcta en este caso. También se comprobó que, cuando queda poca batería, las controladoras RoboClaw comienzan a fallar, introduciendo mucho ruido en la medición.

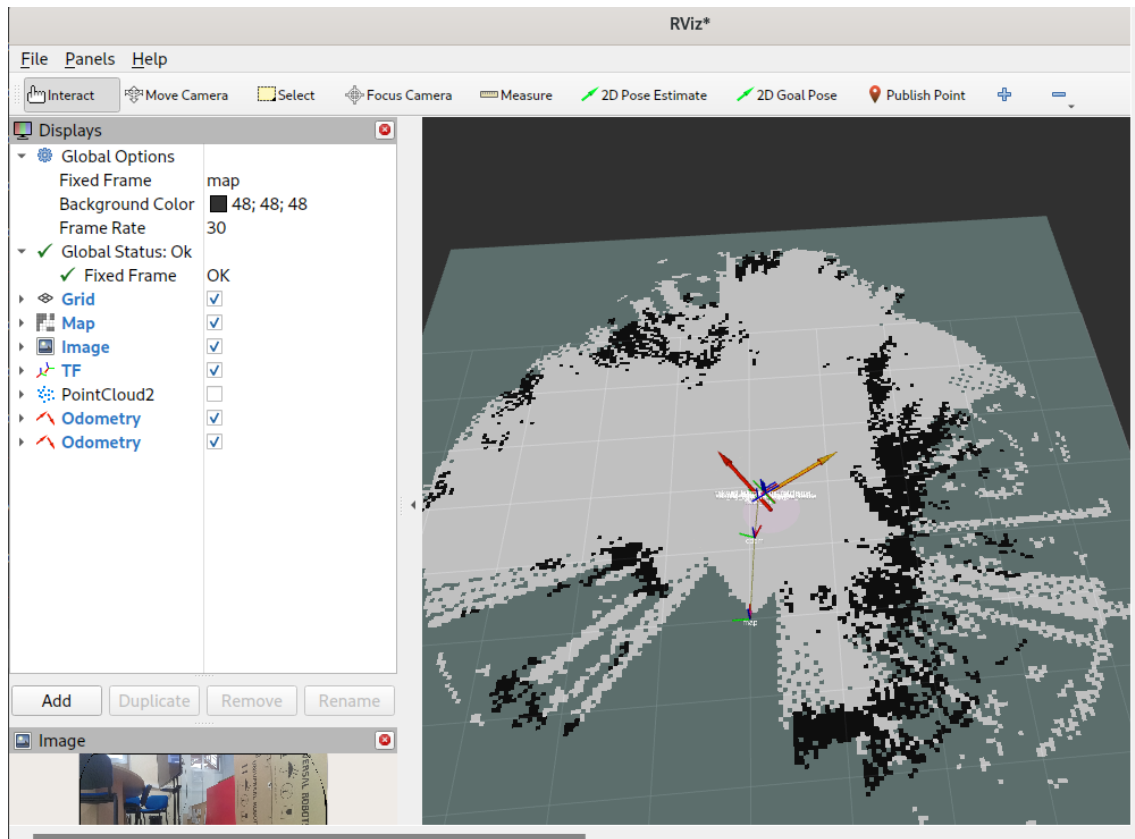


Figura 7.2: Visualización en RViz de la odometría generada por la fusión de encoders e IMU. La flecha roja muestra una estimación de pose errática causada por el deslizamiento de las ruedas.

7.4. Implementación y Optimización del SLAM Visual-Inercial (VIO)

Se procedió a configurar el sistema para que el nodo `rtabmap_odom` de RTAB-Map generara la odometría fusionando los datos de la cámara OAK-D Lite y la IMU externa. Aunque el sistema comenzó a funcionar, surgieron una serie de desafíos directamente relacionados con la capacidad de procesamiento de la Raspberry Pi 5.

7.4.1. Sobrecarga Computacional y Pérdida de Seguimiento

En las primeras ejecuciones con los parámetros por defecto de RTAB-Map y de la cámara, se observó que la odometría era inestable. El árbol de transformaciones se conectaba, pero se rompía intermitentemente, especialmente durante los giros o movimientos rápidos. Esto se manifestaba en el mapa generado como saltos o segmentos desconectados, como se puede ver en la Figura 7.3. El robot mapeaba un pequeño tramo; la odometría se perdía ('Odometry lost'), se reseteaba ('Odometry automatically reset'), y comenzaba a mapear un nuevo segmento desalineado con el anterior.

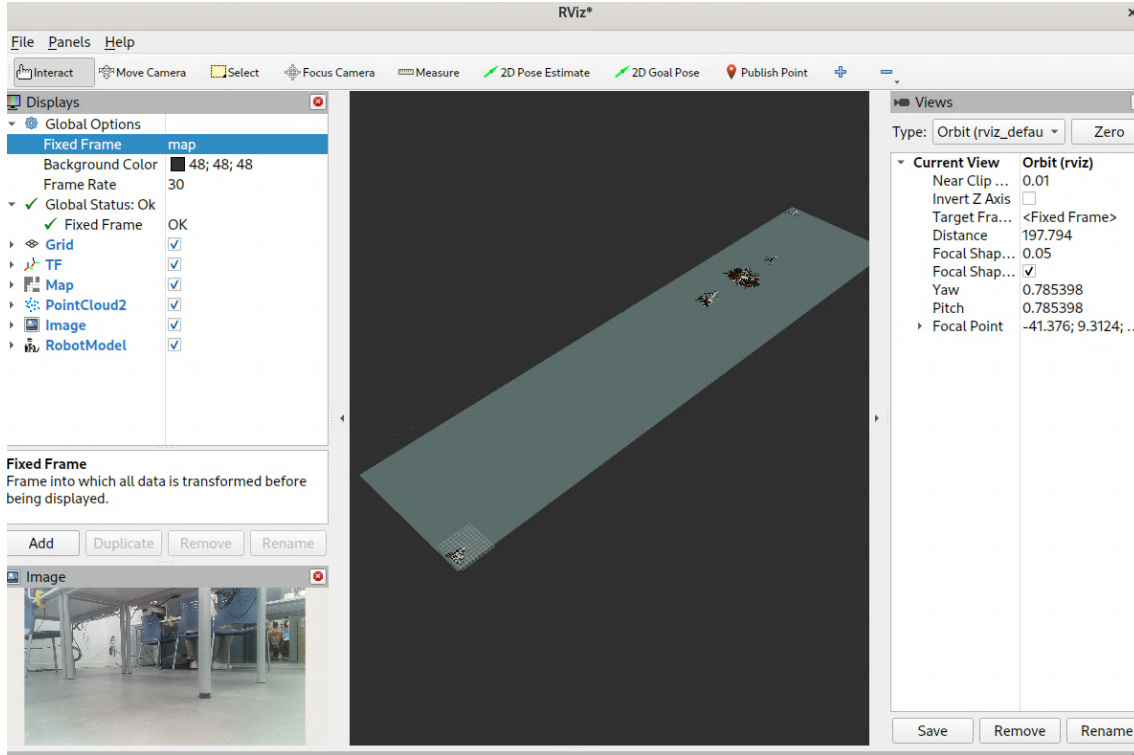


Figura 7.3: Mapa fragmentado generado durante las pruebas iniciales. Cada segmento representa un periodo en el que la odometría funcionó antes de perderse y resetearse, un síntoma de sobrecarga computacional en la Raspberry Pi 5.

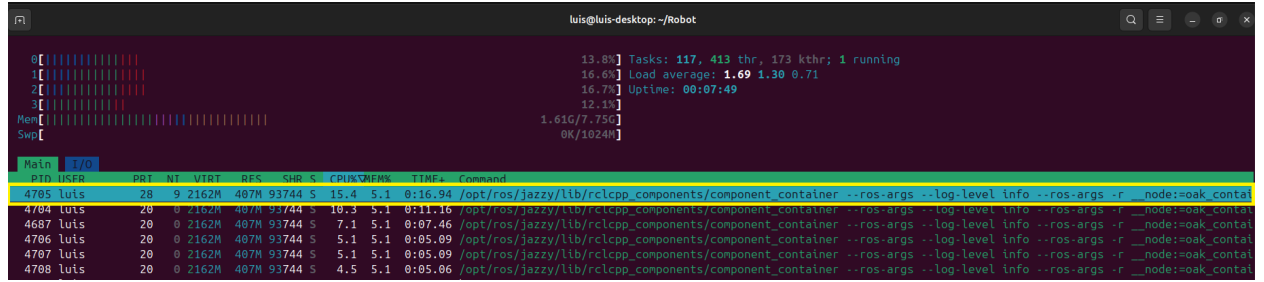
El análisis de los logs reveló que la Raspberry Pi 5 no era capaz de procesar los flujos de imágenes de alta resolución a la frecuencia requerida por RTAB-Map. La solución fue una optimización agresiva de los parámetros tanto en el *driver* de la cámara como en RTAB-Map para reducir la carga computacional:

Ajustes en la cámara: La carga computacional es proporcional a la cantidad de píxeles que debe procesar por segundo ($\text{Ancho} \times \text{Alto} \times \text{FPS}$). Para solucionar este problema en el archivo `camera.yaml`, se redujo la resolución de las cámaras estéreo y RGB a 640x480, y la frecuencia a 15 FPS.

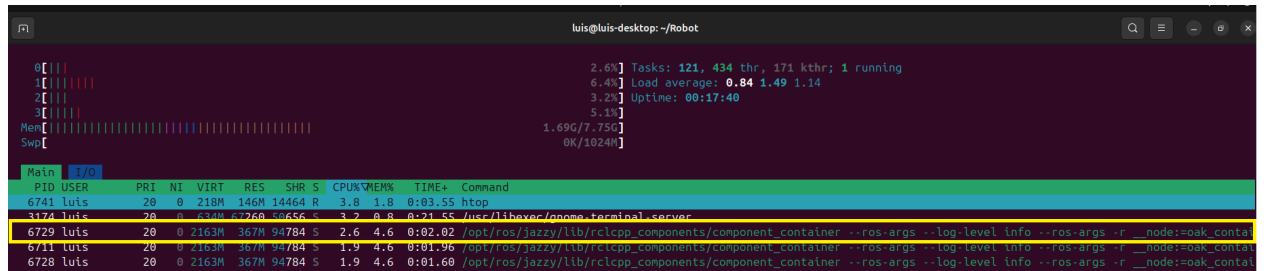
Con el fin de validar esta configuración, se realizó una prueba comparativa, contrastando las dos configuraciones: la original (1080x720, 30 FPS) y la nueva (640x480, 15 FPS). El consumo de la CPU se monitorizó mediante el comando `htop`, mostrando diferencias significativas en la carga computacional. La Tabla 7.2 resume los resultados obtenidos. En las figuras 7.4(a) y 7.4(b) se muestra, señalado en amarillo, el proceso encargado de la rectificación de las imágenes.

Tabla 7.2: Comparativa de la carga de la CPU del nodo de la cámara.

Parámetro	Configuración Base	Configuración Optimizada
Resolución de Cámara	1280x720	640x480
Tasa de Fotogramas (FPS)	30	15
Carga de CPU Promedio	15.4 %	2.6 %
Comportamiento del Sistema	Inestable, pérdida de seguimiento	Fluido, seguimiento estable



(a) Estado de la CPU con resolución 1080x720 y 30 FPS.



(b) Estado de la CPU con resolución 640x480 y 15 FPS.

Los resultados de esta prueba aislada, en la que únicamente se ejecutó el lanzador del *driver* de la cámara (*camera.launch*), son concluyentes. En la configuración base, el contenedor que aloja los nodos de la cámara consumía un 15.4 % de la CPU. Al aplicar la configuración optimizada, esta carga se redujo drásticamente a un 2.6 %, lo que representa una reducción de casi el 85 % en el consumo de recursos de la capa de percepción. Concretamente, esta reducción no solo libera ciclos de CPU, sino que también permite que cada tarea de procesamiento (como la extracción de características y el emparejamiento) se complete en menos tiempo, disminuyendo la latencia total del sistema. El resultado es un flujo de datos más ligero y manejable, lo que permite a los nodos de odometría y SLAM operar de forma fiable y en tiempo real, evitando la pérdida de seguimiento que ocurría cuando intentaban procesar los datos más pesados de la configuración original.

Esta configuración permitió disminuir la carga en la CPU, manteniendo la calidad suficiente para el SLAM.

Ajustes en RTAB-Map: El nodo de RTAB-Map realiza dos tareas principales: la odometría visual, que debe ejecutarse en tiempo real, y el proceso de SLAM (actualización del mapa y cierre de bucles), que es más pesado.

En la consola se mostraba la advertencia:

```
[WARN] [rtabmap]: RTAB-Map processing rate is low, real-time constraint is not met! (0.29 s > 0.10 s)
```

Esto indica que el tiempo que el sistema tardó en procesar cada frame (0.29) es mayor que el tiempo permitido para mantener la operación en tiempo real (0.10). En nuestro caso, el sistema trata de procesar los datos 3.5 veces por segundo.

Para solucionar este problema en el *launch file*, se redujo la frecuencia de detección de 3.5 Hz a 2 Hz (*Rtabmap/DetectionRate: 2.0*). Este parámetro controla la frecuencia del proceso del SLAM.

Para este problema, también se valoró tener una estación a tierra, es decir, comunicar la RPi5 con un ordenador, para que éste sea quien procese todos los datos. El problema en esta solución se presentó cuando la RPi5 enviaba los tópicos de la cámara a través del wifi; la frecuencia de publicación de estos caía drásticamente, lo que dificultaba la sincronización de los datos para mapear.

7.4.2. Filtrado de la Nube de Puntos para Mapeo 2D

Una vez que la odometría se estabilizó, el sistema fue capaz de generar mapas más coherentes. Sin embargo, al proyectar la nube de puntos 3D para crear el mapa de ocupación 2D, el resultado era ruidoso. Los puntos del suelo y del techo se proyectaban incorrectamente como obstáculos, creando barreras inexistentes y un mapa poco fiable para la navegación, como podemos ver en la imagen 7.4

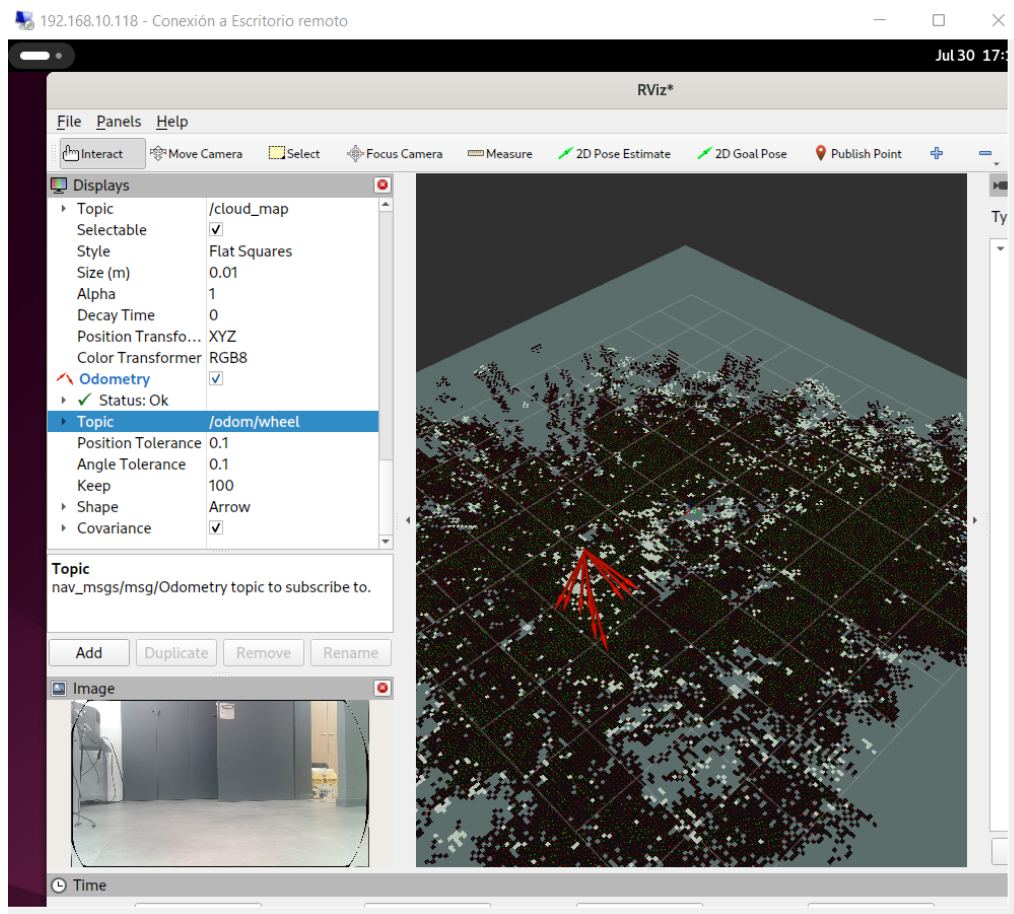


Figura 7.4: Mapa generado durante las pruebas iniciales. Podemos apreciar que los puntos negros no se corresponden con ningún objeto.

La solución fue configurar los parámetros de filtrado del mapa de rejilla en RTAB-Map. Se establecieron los siguientes umbrales:

- **Grid/MaxObstacleHeight:** 0.4: Ignorar cualquier punto detectado a más de 40 cm por encima del suelo del robot (filtrando techos, lámparas, etc.).
- **Grid/MaxGroundHeight:** 0.10: Considerar cualquier punto a menos de 10 cm del suelo como parte del terreno navegable.

Esta configuración actúa como un filtro de corte, asegurando que solo los objetos en un rango de altura relevante (entre 10 cm y 40 cm) se consideren obstáculos en el mapa 2D final.

7.4.3. Pruebas de Estabilidad

Una vez optimizados los parámetros de rendimiento y filtrado, se realizaron una serie de pruebas para evaluar la estabilidad y precisión de la odometría y de la creación del mapa. Se verificó el comportamiento del sistema en tres escenarios antes de proceder al mapeo de un entorno completo.

Prueba Estática (Robot Inmóvil)

La primera prueba consistió en ejecutar el SLAM con el robot inmóvil durante varios minutos. Esta prueba tenía dos objetivos: uno era verificar que no se estimaba un movimiento inexistente y el otro objetivo era comprobar que se formaba correctamente la proyección de la imagen en el plano. El robot estaba orientado hacia una cajonera, como se muestra en la figura 7.5, para comprobar que era capaz de ver correctamente el vértice del mueble.

En la figura 7.5 se han señalado con líneas de distintos colores las aristas principales de la cajonera y el entorno (rojo, azul y amarillo), y esas mismas correspondencias se aprecian en el mapa generado de la figura 7.6.



Figura 7.5: Entorno de prueba del mapa estático.

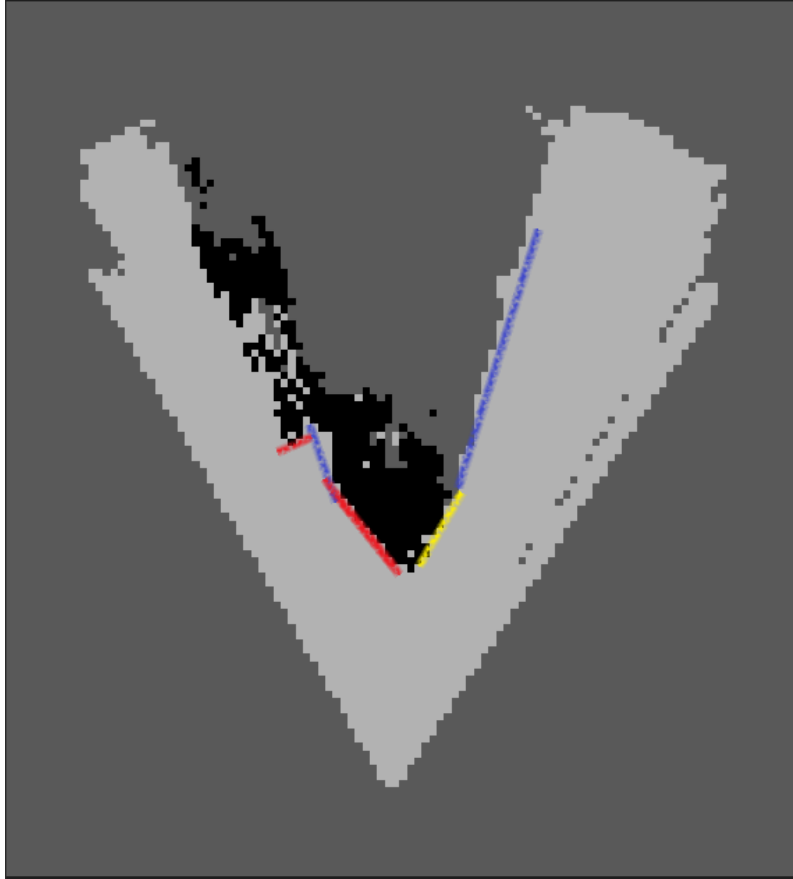


Figura 7.6: Mapa estático de la cajonera.

Tras un análisis del mapa generado (Figura 7.6), se observa que las superficies que en la realidad son perfectamente planas, como el frontal de la cajonera, presentan un notable “rizado” o irregularidad. Esto es producto de dos efectos combinados: el ruido en la medición de la profundidad y el efecto de cuantificación de la rejilla del mapa.

Por un lado, el algoritmo de visión estéreo de la cámara introduce un ruido inherente en la estimación de la profundidad. Pequeñas variaciones en la textura o la iluminación de las superficies, que en este caso son muy planas, provocan ligeras imprecisiones en el cálculo de la distancia. Esto genera una nube de puntos donde, por ejemplo, una superficie a 1.00 metro puede producir mediciones ruidosas a 1.00 m, 1.01 m y 0.99 m. Al proyectarse sobre la rejilla de ocupación 2D, estos puntos no caen en una única línea, creando ruido en el mapa.

Por otro lado, este “rizado” se ve acentuado por el proceso de discretización al proyectar la nube de puntos 3D continua sobre la rejilla de ocupación 2D, que tiene una resolución finita (en este caso, celdas de 5x5 cm). Una línea perfectamente recta en el mundo real que no esté alineada exactamente con los ejes de la rejilla se representará inevitablemente como una aproximación escalonada. La combinación de este efecto de cuantificación con el ruido de la profundidad es lo que produce el característico borde dentado que se observa en el mapa.

Sin embargo, se ha observado experimentalmente que este ruido disminuye a medida que se escanea repetidamente la misma zona. La reducción del ruido se logra porque el mapa no es binario, sino una rejilla probabilística. Cada celda acumula mediciones de los sensores: las observaciones consistentes refuerzan la probabilidad de ocupación, mientras que las inconsistentes, fruto del ruido, tienden a cancelarse. De este modo, con múltiples pasadas del robot, las superficies reales convergen hacia una representación más precisa, mientras que el ruido se filtra y desaparece.

El resultado de esta prueba fue el esperado; la odometría del robot permaneció quieta y se puede ver en el mapa de la figura 7.6 que se forma el vértice correspondiente al entorno.

Prueba de Movimiento Lineal (Ida y Vuelta)

La segunda prueba evaluó la precisión de la estimación de la distancia y de la deriva al trasladarse. Se hizo avanzar el robot en línea recta una distancia aproximadamente de 1.80 m, y luego retroceder al punto de origen para ver si era capaz de estimar correctamente la traslación. En la figura 7.7 se muestra el resultado obtenido.

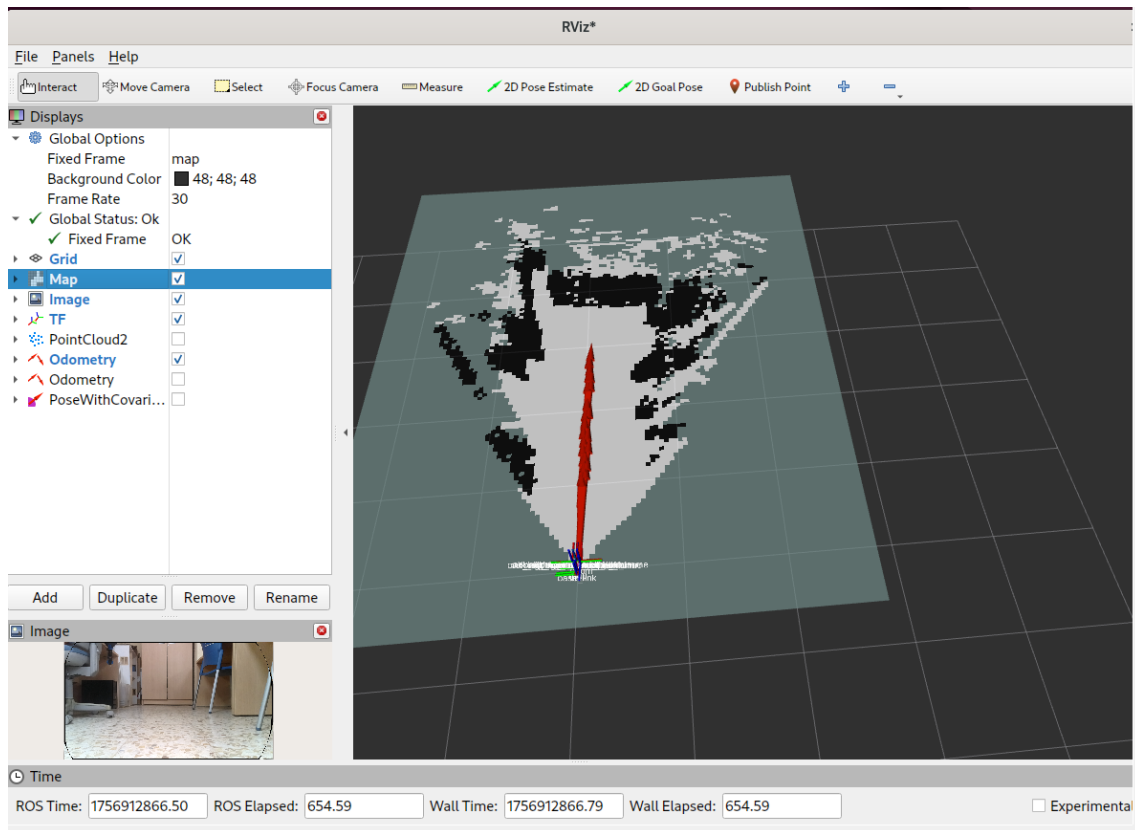


Figura 7.7: Movimiento lineal del robot.

Se observó que, al finalizar el recorrido, la pose final correspondía al punto de partida y que el mapa creado se correspondía con el entorno donde se realizó la prueba. Demostrando que la escala métrica proporcionada por la cámara RGB-D y procesada por RTAB-Map es consistente y precisa, como se puede constatar al comparar ambas imágenes en la Figura 7.7. En la parte inferior izquierda de la imagen, se puede ver el escenario donde se realizó la prueba y, en la parte derecha, el mapa. La correspondencia entre la escena y el mapa es directa:

- La silla y el mobiliario de oficina a la izquierda de la imagen de la cámara se traducen en la masa de obstáculos (píxeles negros) en el lado izquierdo del mapa.
- De forma análoga, la mesa y las sillas a la derecha se representan como el conjunto de obstáculos en el mismo lado del mapa.
- El armario al final del pasillo genera ocupación central, y como no cubre todo el ancho del pasillo, deja un espacio libre en el lateral izquierdo.
- El espacio central, correspondiente al suelo de baldosas visible, se mapea correctamente como una amplia zona navegable (píxeles grises).

La característica forma de 'V' del área explorada es una representación directa del campo de visión de la cámara. El sistema solo puede mapear lo que ve, por lo que el espacio detrás y a los lados del robot permanece desconocido (color gris oscuro de fondo). Las flechas rojas que se ven representan la visualización de la odometría, la trayectoria de ida y vuelta del robot.

Prueba de Rotación Pura (Giro de 360 Grados)

La tercera y más exigente prueba consistió en hacer girar el robot 360 grados. Este tipo de movimiento es complejo para los sistemas de odometría visual, ya que el campo de visión cambia rápidamente. El resultado de esta maniobra, mostrado en la figura 7.8, confirma que tras hacer la rotación completa, la orientación final del robot era prácticamente la misma que la orientación inicial. Probando así que la fusión de la odometría visual con la IMU funciona correctamente.

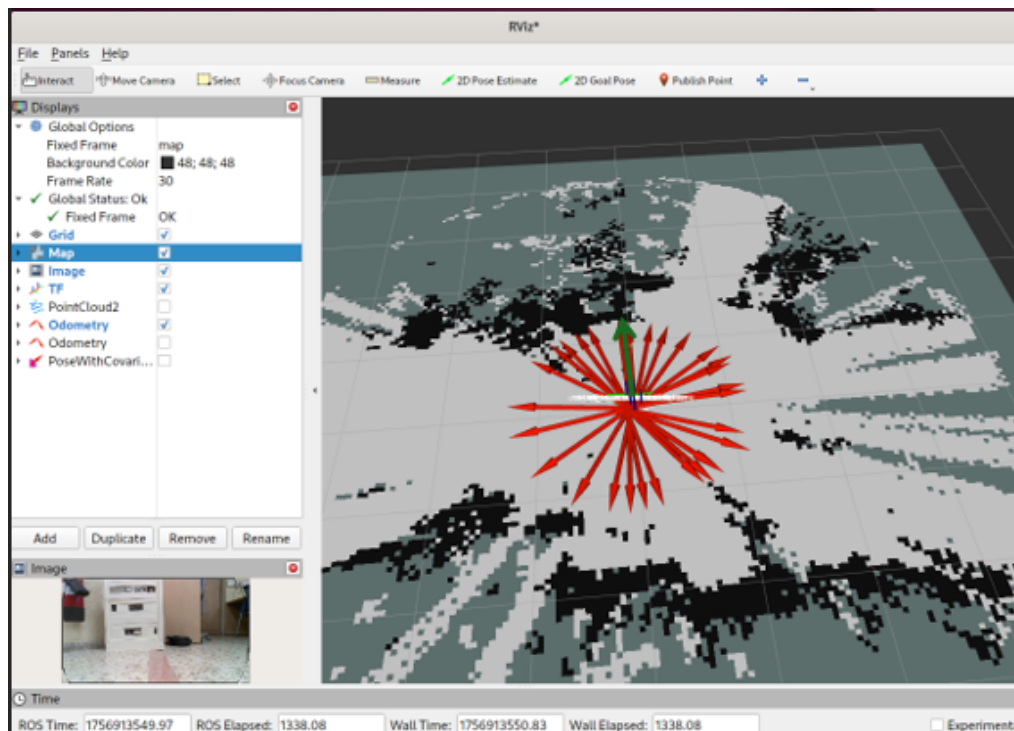


Figura 7.8: Resultado de la rotación del robot. La flecha verde muestra la posición inicial.

Cabe destacar que, durante la ejecución de esta maniobra, el sistema de odometría visual experimentó dos pérdidas temporales de localización. Este comportamiento es habitual en giros completos sobre el propio eje, donde la escasez de puntos de referencia visuales estables y el movimiento rápido dificultan el seguimiento. Sin embargo, en ambas ocasiones, al realizar un pequeño movimiento de corrección en sentido contrario, el algoritmo de localización de RTAB-Map fue capaz de reconocer el entorno previamente mapeado y recuperar la trayectoria correcta sin necesidad de intervención externa. Este evento, lejos de ser un inconveniente, demuestra la robustez y capacidad de recuperación del sistema integrado ante situaciones límite.

7.4.4. Resultado del Mapeo Final

Tras el proceso iterativo de optimización y configuración, se logró generar mapas 2D consistentes y funcionales, como el que se muestra en la Figura 6.2. El mapa resultante representa de manera efectiva la estructura general del entorno, donde la proyección de las nubes de puntos 3D al plano 2D captura la geometría esencial con un nivel de detalle adecuado para la navegación. La disposición espacial obtenida permite identificar claramente las áreas transitables y los obstáculos, definiendo un espacio navegable coherente. Esto confirma que la configuración final del sistema de SLAM produce mapas útiles para su uso en la planificación de movimientos.

7.5. Pruebas de Localización Autónoma

Con un mapa 2D del entorno previamente generado y guardado, se procedió a evaluar la capacidad del sistema para realizar la localización pura. Para esta fase, se ejecutó RTAB-Map en modo de localización (con el parámetro `Mem/IncrementalMemory` en `false`), de modo que el robot utilizara el mapa existente para estimar su pose sin modificarlo.

El objetivo de la prueba era verificar si el robot, al ser movido por el entorno, era capaz de reconocer lugares ya visitados y mantener una estimación de pose coherente y sin deriva.

7.5.1. Análisis del Proceso de Localización

Para analizar el funcionamiento interno del algoritmo de localización, se utilizó la interfaz gráfica `rtabmap_viz`. Esta herramienta nos permite ver en tiempo real lo que está procesando el sistema. La Figura 7.9 muestra una captura de esta interfaz en un momento clave. En la parte superior, la ventana “*Loop closure detection*” muestra la imagen actual (superior) y una imagen de la base de datos (inferior) que el sistema ha identificado como una coincidencia visual. Las líneas de color cian conectan las características visuales (*features*) que coinciden entre ambas imágenes.

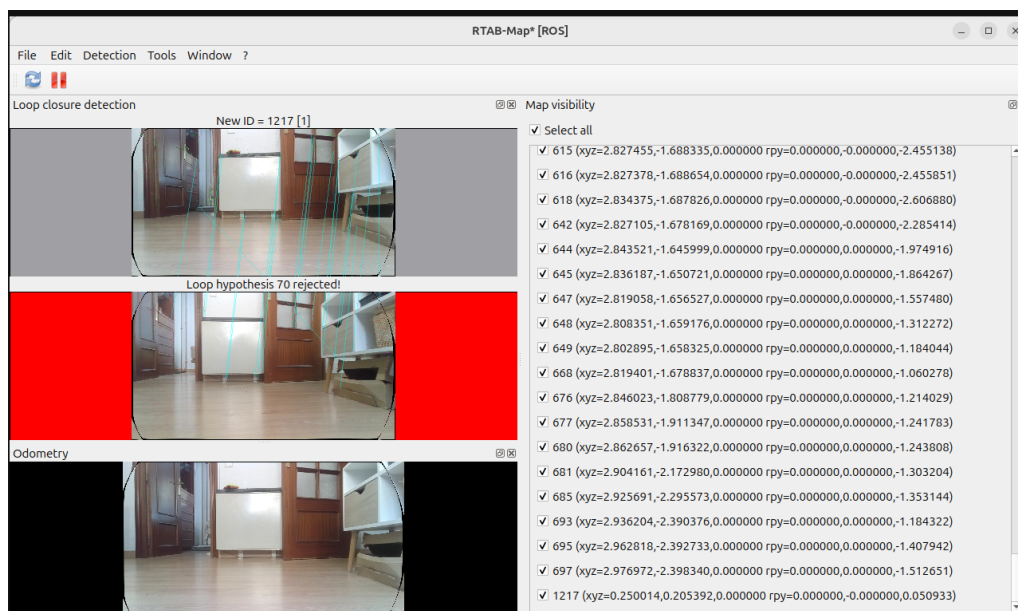


Figura 7.9: Ventana de `rtabmap_viz` mostrando una hipótesis de cierre de bucle. A pesar de las numerosas correspondencias visuales (líneas cian), la hipótesis es rechazada.

Se observa un gran número de correspondencias consistentes entre las dos imágenes, lo que indica un reconocimiento visual muy fuerte. Sin embargo, el texto superpuesto indica “**Loop hypothesis 70 rejected!**”. Esto significa que, a pesar de que el algoritmo de reconocimiento encontró una buena coincidencia, el sistema la descartó en una etapa posterior de validación.

7.5.2. Análisis de los Datos de Diagnóstico

Para profundizar en la causa de este rechazo, se analizaron los datos publicados en el topic de diagnóstico `/info` de RTAB-Map, utilizando el comando `ros2 topic echo /info`. Este topic proporciona métricas internas detalladas sobre el estado del algoritmo. La Tabla 7.3 resume los valores más relevantes observados durante el evento de la Figura 7.9.

Los datos de la Tabla 7.3 confirman el diagnóstico. El sistema está identificando correctamente lugares ya visitados con una confianza muy alta (casi 80% de similitud y 18 inliers, lo cual es un buen número). Sin embargo, la hipótesis es rechazada consistentemente.

Tabla 7.3: Métricas clave del topic `/info` durante una hipótesis de localización.

Métrica (stats_keys)	Valor (stats_values)	Interpretación
Loop/Highest_hypothesis_id	70.0	El nodo del mapa con mayor coincidencia visual es el ID 70.
Loop/Highest_hypothesis_value	0.7924	La calidad de la coincidencia visual es del 79.24 %, un valor muy alto.
Loop/Reactivate_id	70.0	El sistema está intentando activamente localizarse en la pose del nodo 70.
Loop/RejectedHypothesis	1.0	Indica que se acaba de rechazar una hipótesis de bucle.
Loop/Visual_inliers	18.0	Se encontraron 18 características visuales consistentes (inliers).
Timing/Hypotheses_validation/ms	0.0128	El proceso de validación fue extremadamente rápido (0.01 ms).

7.5.3. Discusión de los Resultados de Localización

El análisis demuestra un comportamiento dual en el sistema. Por un lado, el componente de reconocimiento de lugares basado en apariencia (BoW) funciona correctamente. Como se ha evidenciado, el sistema es capaz de encontrar correspondencias visuales fiables entre la vista actual y el mapa almacenado con una confianza muy alta (casi 80 % de similitud y 18 inliers).

Es crucial destacar que, durante la fase de mapeo (modo SLAM), estas fuertes coincidencias visuales se traducen en localizaciones exitosas. Esto prueba que el sistema es capaz de reconocer lugares ya visitados y de optimizar el grafo para mantener un mapa coherente mientras se construye.

Sin embargo, el fallo ocurre sistemáticamente al cambiar a modo de localización pura. A pesar de la fuerte evidencia visual, las hipótesis de localización son rechazadas. La causa raíz de esta discrepancia no reside en la calidad del reconocimiento visual, sino que puede deberse al mecanismo de validación geométrica y, fundamentalmente, a la gestión de la memoria de RTAB-Map.

- **Gestión de la Memoria de Trabajo:** En modo de mapeo, RTAB-Map mantiene una memoria de trabajo dinámica y en constante crecimiento. Cuando el robot se localiza, compara la vista actual con esta memoria activa, lo que facilita la aceptación de cierres de bucle y la localización. En contraste, en modo de localización (`Mem/IncrementalMemory: false`), el sistema, por defecto, carga solo una pequeña vecindad del mapa global en la memoria de trabajo para optimizar el uso de recursos.
- **Validación Geométrica:** El segundo mecanismo que puede causar el rechazo de una hipótesis es la validación geométrica. Este proceso actúa como una comprobación de “sentido común”. Después de encontrar una coincidencia visual, RTAB-Map verifica si esta coincidencia es geoméricamente coherente. Para ello, intenta calcular una transformación 3D (rotación y traslación) que alinee los puntos de la vista actual con los puntos de la vista recordados. Si no puede encontrar un modelo matemático que explique esta relación de forma robusta, la hipótesis es descartada. Una causa común para este tipo de fallo es una deriva excesiva en la odometría, que provoca que la posición estimada del robot esté tan lejos de la posición real que la coincidencia visual se considera geoméricamente imposible.

Para corroborar esta hipótesis, se forzó a RTAB-Map a cargar la base de datos completa en su memoria de trabajo activa al inicio, utilizando el parámetro `Mem/InitWMWithAllNodes: true`. A pesar de este cambio, que teóricamente debería facilitar la localización, el problema persistió. El sistema continuó rechazando hipótesis visualmente válidas.

Para eliminar la deriva de la odometría como posible causa, se crearon mapas nuevos bajo condiciones controladas, intentando la localización inmediatamente después y sin apenas mover el robot.

A pesar de estas optimizaciones, el problema persistió. El sistema continuó rechazando sistemáticamente hipótesis que eran visualmente muy fuertes. Se replicó el experimento en un orde-

nador portátil, obteniendo el mismo resultado negativo, lo que permitió descartar que el fallo fuera específico del *hardware* de la Raspberry Pi.

7.5.4. Conclusión sobre la Localización Pura

La evidencia experimental demuestra de forma concluyente que el modo de localización pura de RTAB-Map, en la configuración probada, no es suficientemente robusto. Aunque el sistema es capaz de identificar lugares ya visitados con una alta confianza visual. Hasta el momento, no ha sido posible determinar con certeza la causa de este fallo de validación. Incluso bajo condiciones controladas, con mapas recién creados y tras forzar la carga completa de la memoria, el sistema sigue rechazando sistemáticamente las hipótesis de localización a pesar de la fuerte evidencia visual. Esto sugiere que puede existir una incompatibilidad en la configuración actual o que sea un problema de sintonización o de integración entre dependencias.

Dado que la localización pura no se comporta de manera estable, se plantea como siguiente paso explorar una arquitectura alternativa más robusta dentro del ecosistema ROS 2: delegar la tarea de localización a un nodo especializado como AMCL, que trabaja sobre un mapa 2D estático y datos de escaneo láser (reales o simulados).

7.6. Pruebas de Localización con AMCL y LaserScan

Ante los desafíos encontrados con la localización basada en la apariencia visual de RTAB-Map, se exploró una estrategia de localización alternativa: el AMCL.

Como se describió en el Marco Teórico, AMCL es un algoritmo robusto que utiliza un filtro de partículas para estimar la pose del robot en un mapa 2D existente. Sin embargo, su principal requisito es una fuente de datos de tipo *LaserScan*, que nuestro robot no posee de forma nativa.

7.6.1. Generación de LaserScan Sintético

Para salvar esta incompatibilidad, se integró el paquete de ROS 2 `depthimage_to_laserscan` [35]. La función de este nodo es suscribirse a la imagen de profundidad (`/oak/stereo/image_raw`) y a la información de la cámara (`/oak/stereo/camera_info`) publicadas por el *driver* de la OAK-D Lite. A partir de estos datos, el nodo proyecta la información de profundidad en un plano horizontal y la convierte en un mensaje de tipo `sensor_msgs/LaserScan`, simulando así el comportamiento de un LiDAR 2D.

La arquitectura del sistema se modificó para que el nodo AMCL se suscribiera a este topic `/scan` sintético y a la odometría fusionada (`/odom`) generada por el filtro de Kalman.

7.6.2. Discusión de Resultados y Problemas de Sincronización

Al ejecutar el sistema con esta configuración, se encontró un nuevo y persistente problema de sincronización; esta vez, relacionado con las transformaciones (TF). Los logs de AMCL mostraban repetidamente errores del tipo:

```
[amcl-2] [INFO] [1756227135.325501023] [amcl]: Message Filter dropping
message: frame 'base_link' at time 1756227134.259 for reason
'the timestamp on the message is earlier than all the data in
the transform cache'
```

```
[controller_server-5] [INFO] [1756227137.534205466] [local_costmap.local_costmap]:
Message Filter dropping message: frame 'base_link' at time
1756227137.158 for reason 'the timestamp on the message is
earlier than all the data in the transform cache'
```

Este error indica que el *timestamp* de los mensajes `/scan` que llegaban a AMCL y al `controller_server` era considerablemente más antiguo que la última transformación disponible en el árbol TF para el frame `base_link`. AMCL necesita que la diferencia temporal entre el escaneo y la transformación

sea muy pequeña para poder ubicar correctamente el escaneo en el mapa. Si el desfase es demasiado grande, descarta el escaneo por considerarlo obsoleto.

Las causas de este desfase temporal en la Raspberry Pi 5 se atribuyen a la latencia acumulada en la cadena de procesamiento:

1. El *driver* de la cámara captura y publica la imagen de profundidad.
2. El nodo `depthimage_to_laserscan` recibe la imagen, la procesa (una tarea computacionalmente no trivial) y publica el `/scan`.
3. El nodo AMCL recibe el `/scan` y solicita el TF correspondiente a su *timestamp*.

La suma de los retardos en cada uno de estos pasos, magnificada por la carga del sistema en la Raspberry Pi, resulta en un desfase temporal que el filtro de mensajes de AMCL no puede tolerar. A pesar de numerosos intentos de ajuste en los parámetros de sincronización, no se logró una operación estable.

En conclusión, aunque la estrategia de generar un *LaserScan* sintético es conceptualmente viable, hasta el momento no ha sido posible resolver completamente los problemas de latencia y sincronización de *timestamps*. Sin embargo, estos inconvenientes parecen ser más manejables que los asociados a la localización puramente visual, lo que sugiere que esta línea de trabajo podría constituir una alternativa prometedora a explorar en un futuro cercano.

7.7. Pruebas de Navegación Autónoma con Nav2

El objetivo final del sistema, una vez generado un mapa y establecida una localización robusta, es la navegación autónoma. Para esta fase, se configuró Nav2, utilizando como base el mapa 2D previamente generado con RTAB-Map.

7.7.1. Configuración y Puesta en Marcha del Sistema de Navegación

El sistema de navegación se lanzó utilizando el archivo `navegacion.launch.py`, el cual inicia todos los componentes necesarios para la navegación. Este archivo de lanzamiento se configuró de forma modular para permitir diferentes modos de operación:

- **Modo de Navegación Pura:** Se ejecuta el archivo `rtabmap.launchNavegacion.py` para el sistema de localización, seguido del archivo `navegacion.launch.py` con el nodo `depthimage_to_laserscan` y `amcl` comentado, utilizando únicamente la localización visual de RTAB-Map.
- **Modo con Simulación de LaserScan:** Se ejecuta el archivo `camera.launchNavegacionScan.py` junto con `navegacion.launch.py` con todos los nodos activos.

Como se puede observar en la Figura 7.10, el sistema se inicia correctamente. La interfaz de RViz2 muestra que el mapa ha sido cargado por el `map_server`, el árbol de transformaciones (TF) está completo y conectado, y el panel *Navigation 2* indica que el sistema está activo.

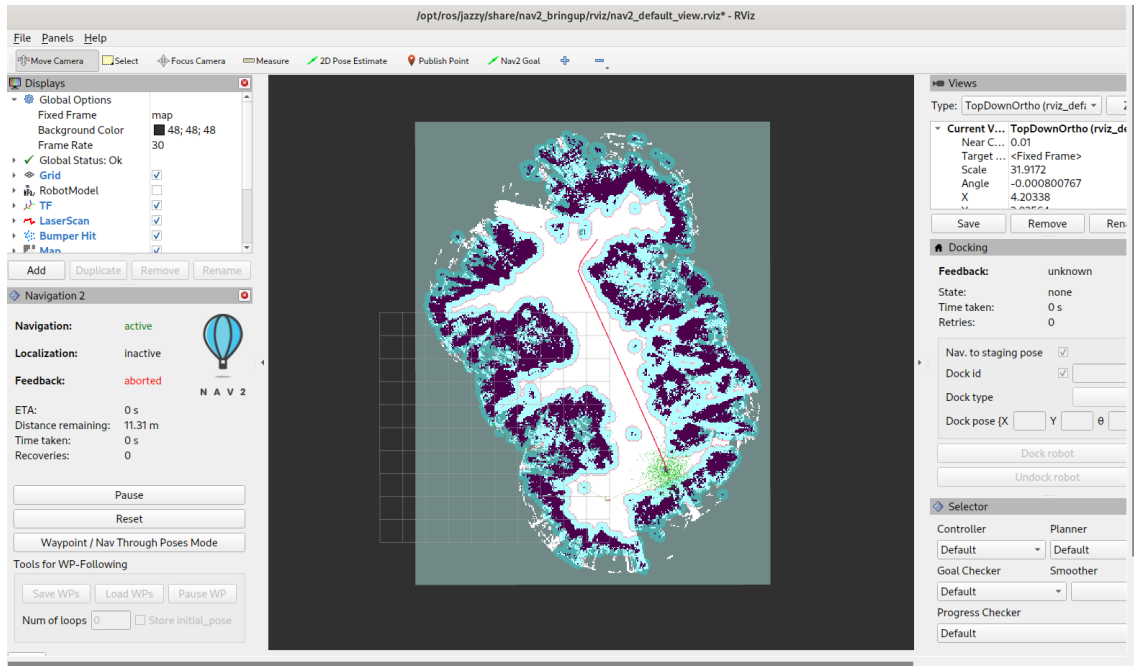


Figura 7.10: Vista de RViz2 con el *stack* de navegación Nav2 activo. Se observa el mapa cargado, el costmap global (púrpura y cian), la nube de partículas de la localización (verde) y el árbol TF completo.

Al proporcionar un objetivo de navegación con la herramienta “*Nav2 Goal*”, el planificador global (**planner_server**) demuestra ser funcional, calculando una ruta óptima (visible como una línea roja punteada en la Figura 7.10) desde la pose estimada del robot hasta el destino, evitando los obstáculos del mapa estático.

7.7.2. Discusión de Resultados: Fallo en la Localización

A pesar de que la pila de navegación se lanzó y configuró correctamente, las pruebas de navegación física no pudieron completarse con éxito debido a un fallo persistente en el módulo de localización.

Como se puede ver en el panel “Navigation 2” de la Figura 7.10, el estado de **Localization** se muestra como **inactive**. Adicionalmente, la nube de partículas verdes, que representa las hipótesis de la pose del robot de RTAB-Map, aparece dispersa en lugar de converger en una única ubicación precisa.

Este comportamiento indica que el sistema de localización de RTAB-Map, aunque funcionó correctamente durante la fase de mapeo, no fue capaz de localizarse de forma robusta y continua en el mapa previamente guardado. Al no tener una estimación de pose fiable y estable, el robot no sabe dónde está. En consecuencia, aunque el controlador local (**controller_server**) generaba comandos de velocidad para seguir la ruta planificada, estos comandos eran incoherentes y provocaban que el robot se moviera de forma errática y sin rumbo, ya que la base de su sistema de coordenadas (**odom** → **base_link**) no estaba correctamente anclada al mapa (**map**).

Se concluye que el problema de inestabilidad en la localización visual, discutido en la sección anterior (Sección 7.5), es el impedimento final para lograr una navegación autónoma completa. La resolución de este problema de localización se plantea como la principal línea de trabajo futura.

Capítulo 8

Gestión del Proyecto

En este capítulo se describe la gestión del proyecto desde una perspectiva de ingeniería, abarcando la planificación de las tareas, la estimación del tiempo dedicado y el presupuesto detallado tanto del *hardware* empleado como de los recursos humanos involucrados.

8.1. Planificación y Ciclo de Vida

El desarrollo de este proyecto ha seguido un ciclo de vida iterativo, típico en proyectos de investigación donde el diseño y la implementación evolucionan en paralelo. Aunque no se ha seguido un diagrama de Gantt estricto, el trabajo se ha estructurado en fases con objetivos definidos, permitiendo solapar tareas de *hardware* y *software* para optimizar el tiempo.

- **Fase 1: Investigación y Familiarización (Mes 1):** Esta fase inicial se centró en la familiarización con el ecosistema de ROS 2 y las tecnologías claves del proyecto. Se realizó un estudio del estado de la técnica que culminó con las decisiones fundamentales sobre la arquitectura del sistema: la selección de la plataforma base comercial, los sensores principales (OAK-D Lite, IMU externa) y la unidad de procesamiento central (Raspberry Pi 5).
- **Fase 2: Adquisición de *Hardware* y Control Básico (Mes 2):** Se procedió a la adquisición de todos los componentes comerciales. En paralelo, se comenzó con la implementación del software de bajo nivel, configurando las controladoras RoboClaw y estableciendo la comunicación inicial con los actuadores. El objetivo era tener una base móvil funcional lo antes posible para poder realizar pruebas incrementales.
- **Fase 3: Desarrollo del Software de bajo nivel (Meses 2-3):** Solapándose con la fase anterior, se desarrolló y validó el software de control fundamental. Esto incluyó la implementación de las funciones de cinemática (directa e inversa) y la creación del nodo `robot_driver_node` en ROS 2. Se realizaron pruebas de control por coordenadas, como se detalla en el Capítulo 7, para validar la correcta publicación de la odometría de ruedas y la recepción de comandos de velocidad.
- **Fase 4: Diseño de la Electrónica de Potencia (Meses 3-4):** De forma simultánea al desarrollo de software, se diseñó la electrónica de protección. Esta fase abarcó el diseño y la fabricación de la placa de interfaz personalizada para las controladoras RoboClaw, que integra el crucial circuito de protección *Voltage Clamp* y centraliza las conexiones de los encoders.
- **Fase 5: Integración del SLAM y Fusión de Sensores (Meses 4-5):** Esta fue la fase más compleja, centrada en la integración de todo el sistema. Se configuró el *pipeline* de percepción con la cámara y la IMU, se implementó el filtro de Kalman para la fusión de odometría de ruedas e inercial, y se configuró RTAB-Map. Una parte significativa del tiempo se dedicó a la depuración de problemas de rendimiento y sincronización de sensores en la Raspberry Pi 5.

- **Fase 6: Pruebas Finales y Documentación (Mes 6-7):** Se implementa la localización y la navegación y se realizan las pruebas experimentales de mapeo y localización con la arquitectura distribuida, analizar los resultados y proceder a la redacción de la presente memoria.
- **Fase 7: Documentación (Mes 7-8):** La última fase se dedicó a la redacción de la memoria

En la figura 8.1 podemos observar esta planificación.

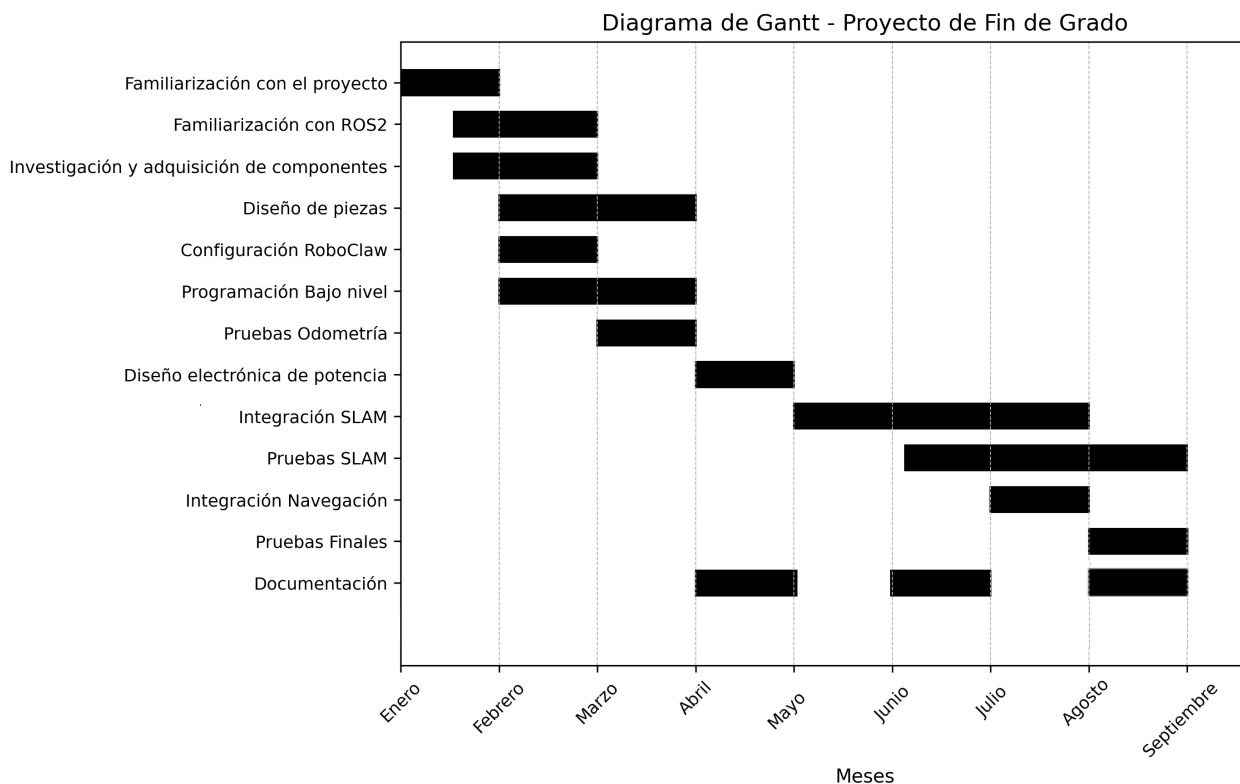


Figura 8.1: Diagrama de Gantt donde se muestra la planificación del proyecto.

8.2. Presupuesto

A continuación, se detalla el presupuesto estimado del proyecto, desglosando los costes en recursos humanos (personal) y recursos materiales.

8.2.1. Coste de Personal

Para la estimación del coste de los recursos humanos, se ha considerado el tiempo dedicado por el alumno y el tiempo de supervisión del tutor.

- **Alumno:** Se estima una dedicación total aproximada de 600 horas al proyecto, distribuidas a lo largo de ocho meses. Para estimar el coste, se ha tomado como referencia el salario promedio recibido con un contrato de prácticas curriculares, que supone un valor aproximado de 5 €/hora. Bajo este criterio, el coste total del trabajo del alumno se estima en torno a 3000 €.
- **Tutor:** Se estima un tiempo de supervisión, reuniones de seguimiento y revisión de la memoria de aproximadamente 40 horas. Considerando que el salario medio de un profesor es de 35 €/hora, el coste total asociado a la tutorización es de 1400 €.

El coste total estimado de los recursos humanos para este proyecto es de **4400 €**.

8.2.2. Coste de Material

En la Tabla 8.1 se desglosan los costes de los componentes de *hardware* principales utilizados para la construcción del robot. Los precios reflejan el coste de mercado en el momento de la adquisición y pueden variar. Se han omitido componentes menores como cableado, conectores o tornillos.

Tabla 8.1: Presupuesto detallado del *hardware* del robot.

Componente	Proveedor (Ejemplo)	Precio Estimado (€)
Chasis goBILDA Recon	goBILDA	349.99
Motor Planetario goBILDA 5203 (x4)	goBILDA	197.96
Controladora RoboClaw 2x7A (x2)	Basic Micro / RobotShop	175.9
Raspberry Pi 5 (8GB)	RaspiPC / PC Componentes	108.00
Cámara Luxonis OAK-D Lite	Luxonis / Mouser	128,00
IMU WitMotion WT901C	Amazon / WitMotion	45,00
Batería LiPo 4S (3700mAh)	Tienda RC / Amazon	53,00
Placa de Alimentación RPi5 Yahboom	Yahboom / Amazon	10,00
Nanorouter WiFi TP-Link TL-WR802N	Amazon	23,00
Coste Total de Material		1.090,00 €

A este coste de componentes principales es necesario añadir los costes asociados a la fabricación de piezas a medida y prototipado, que son fundamentales para la integración del sistema.

- **Piezas Impresas en 3D:** Para el correcto montaje de sensores y periféricos, se han diseñado e impreso en 3D un total de 5 piezas personalizadas. Se estima un tiempo total de impresión de aproximadamente 3 horas. Considerando un coste medio de material (filamento PLA/PETG) y amortización de la impresora de 2,00 €/hora, el coste de estas piezas asciende a **6,00 €**. Este valor es aproximado, ya que no incluye el tiempo de diseño ni las posibles impresiones fallidas durante la fase de prototipado.
- **Fabricación y Componentes de la PCB:** La placa de interfaz personalizada, que alberga el circuito de protección *Voltage Clamp* y centraliza las conexiones, ha sido un componente de diseño propio. El coste se desglosa en:
 - *Fabricación de la PCB:* Se encargó la producción de las placas de circuito impreso a un proveedor especializado con un coste aproximado de **15,00 €** para un lote mínimo de 5 unidades.
 - *Componentes Electrónicos de la PCB:* El coste de los componentes electrónicos específicos soldados en la placa (MOSFET de potencia, resistencia de disipación, conectores JST y Dupont, etc.), detallados en la Sección 4.1.7, se estima en **20,00 €**.

Sumando estos costes de fabricación y prototipado ($6.00 \text{ €} + 15.00 \text{ €} + 20.00 \text{ €} = 41.00$) al coste de los componentes principales, se obtiene el coste total de material del robot.

8.2.3. Resumen de Costes

El presupuesto total del proyecto se obtiene sumando los costes de personal y de material.

- Coste de Personal: 4400 €
- Coste de Material: 1131 €
- **COSTE TOTAL DEL PROYECTO: 5531 €**

Este presupuesto refleja el valor completo del desarrollo de ingeniería realizado, destacando que el coste del *hardware* de la plataforma robótica final es relativamente bajo, lo que cumple con uno de los objetivos principales del proyecto.

Capítulo 9

Conclusiones

Se presentan a continuación las conclusiones extraídas tras la finalización del proyecto, así como las posibles líneas de trabajo futuro que se abren a partir de los resultados obtenidos.

9.1. Conclusión

El presente Trabajo de Fin de Grado ha abordado con éxito el diseño, la construcción y la implementación de una plataforma robótica móvil autónoma desde sus fundamentos. Se han cumplido con éxito una parte fundamental de los objetivos planteados, logrando desarrollar un sistema funcional capaz de realizar el mapeo de su entorno utilizando técnicas de VSLAM de forma robusta, y sentando una base sólida y validada experimentalmente sobre la cual construir un sistema de navegación autónoma completo.

Se ha completado el estudio de la selección e integración de todos los componentes *hardware* necesarios, desde la plataforma base comercial y los actuadores, hasta la electrónica de control y percepción. Se ha diseñado un sistema de alimentación robusto, incluyendo un circuito de protección contra la frenada regenerativa, y se ha implementado una arquitectura de control jerárquica que delega las tareas de bajo nivel en controladoras dedicadas, una práctica estándar en robótica.

A nivel de software, se ha desarrollado un *driver* personalizado que gestiona la cinemática y la odometría de las ruedas. La validación experimental de este sistema de bajo nivel demostró su correcto funcionamiento, aunque evidenció las limitaciones inherentes a la deriva por deslizamiento.

En el control de alto nivel, se integró y optimizó el sistema de VSLAM de alto nivel, RTAB-Map, fusionando los datos de la cámara RGB-D y una IMU externa. El principal desafío encontrado fue la sobrecarga computacional en la Raspberry Pi 5. Tras un proceso iterativo de optimización de parámetros, se logró generar mapas 2D consistentes y métricamente precisos, validando así la viabilidad del sistema.

La fase de localización autónoma, sin embargo, presentó desafíos significativos que impidieron alcanzar el objetivo principal en su totalidad. Se abordó el problema desde dos arquitecturas distintas:

1. **Localización con RTAB-Map:** El primer enfoque consistió en utilizar el modo de localización pura de RTAB-Map. A pesar de que el sistema demostró una excelente capacidad para identificar lugares ya visitados (con una alta confianza visual y un gran número de *inliers*), se encontró un fallo sistemático e irresoluble en la etapa de validación interna del algoritmo, que rechazaba constantemente las hipótesis de localización correctas.
2. **Localización con AMCL:** Como alternativa, se pivotó a la arquitectura estándar de ROS 2, delegando la localización al nodo AMCL y proporcionándole un escaneo láser simulado. Aunque esta arquitectura es conceptualmente válida, su implementación se vio obstaculizada por problemas de sincronización de tiempos entre la odometría de alta frecuencia y los datos de los sensores, lo que resultó en el descarte de mensajes y la incapacidad de AMCL para converger a una pose estable.

Finalmente, en cuanto a la navegación, se configuraron los parámetros fundamentales de los servidores de Nav2 (`planner_server`, `controller_server`, `bt_navigator`) y su ciclo de vida. Sin embargo, no fue posible validar su funcionamiento de forma robusta en un entorno real, ya que su operación depende críticamente de una estimación de pose estable que, como se ha detallado, no se pudo lograr.

Se concluye que el proyecto ha servido como una aplicación práctica integral de los conocimientos adquiridos en el Grado de Ingeniería Electrónica Industrial y Automática, abarcando desde el diseño electrónico y la programación de bajo nivel hasta la implementación de algoritmos complejos y la robótica autónoma.

9.2. Desarrollos Futuros

Los resultados y desafíos de este proyecto abren varias líneas de trabajo futuro interesantes para mejorar las capacidades del robot:

- **Estabilización de la Localización Autónoma:** La principal línea de trabajo es resolver los problemas de inestabilidad en la localización de RTAB-Map. Esto implicaría experimentar con diferentes estrategias de odometría, ajustar los parámetros de validación geométrica de cierres de bucle y, potencialmente, explorar algoritmos de localización alternativos como AMCL con los problemas de sincronización ya resueltos.
- **Implementación de la Navegación Completa:** Una vez lograda una localización robusta, el siguiente paso es la implementación completa del *stack* de navegación Nav2, realizando pruebas exhaustivas de planificación de rutas y evasión de obstáculos dinámicos.
- **Exploración de Funcionalidades de IA en la Cámara:** Aprovechar la VPU de la cámara OAK-D Lite para ejecutar redes neuronales en el borde, permitiendo al robot realizar tareas de detección y reconocimiento de objetos en tiempo real sin sobrecargar la Raspberry Pi 5.
- **Interfaz de Usuario de Alto Nivel:** Actualmente, la interacción con el robot se realiza a través de RViz, una herramienta para desarrolladores. Un desarrollo futuro podría centrarse en la Interacción Humano-Robot, creando una interfaz de usuario simplificada (por ejemplo, una aplicación web o móvil) que permita a un usuario no técnico seleccionar un destino en el mapa y monitorizar el progreso del robot, haciendo que el sistema sea accesible para aplicaciones prácticas.

Apéndice A

Anexo 1: Diseño de Circuito y PCB

En este anexo se incluyen los planos completos del circuito impreso diseñado para la plataforma robótica y las distintas etapas del proceso de diseño.

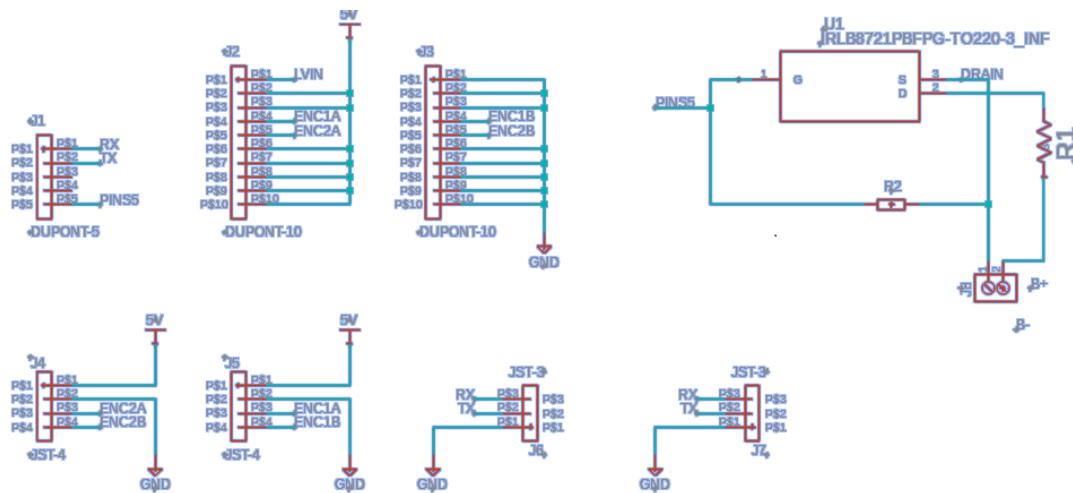


Figura A.1: Circuito de la PCB.

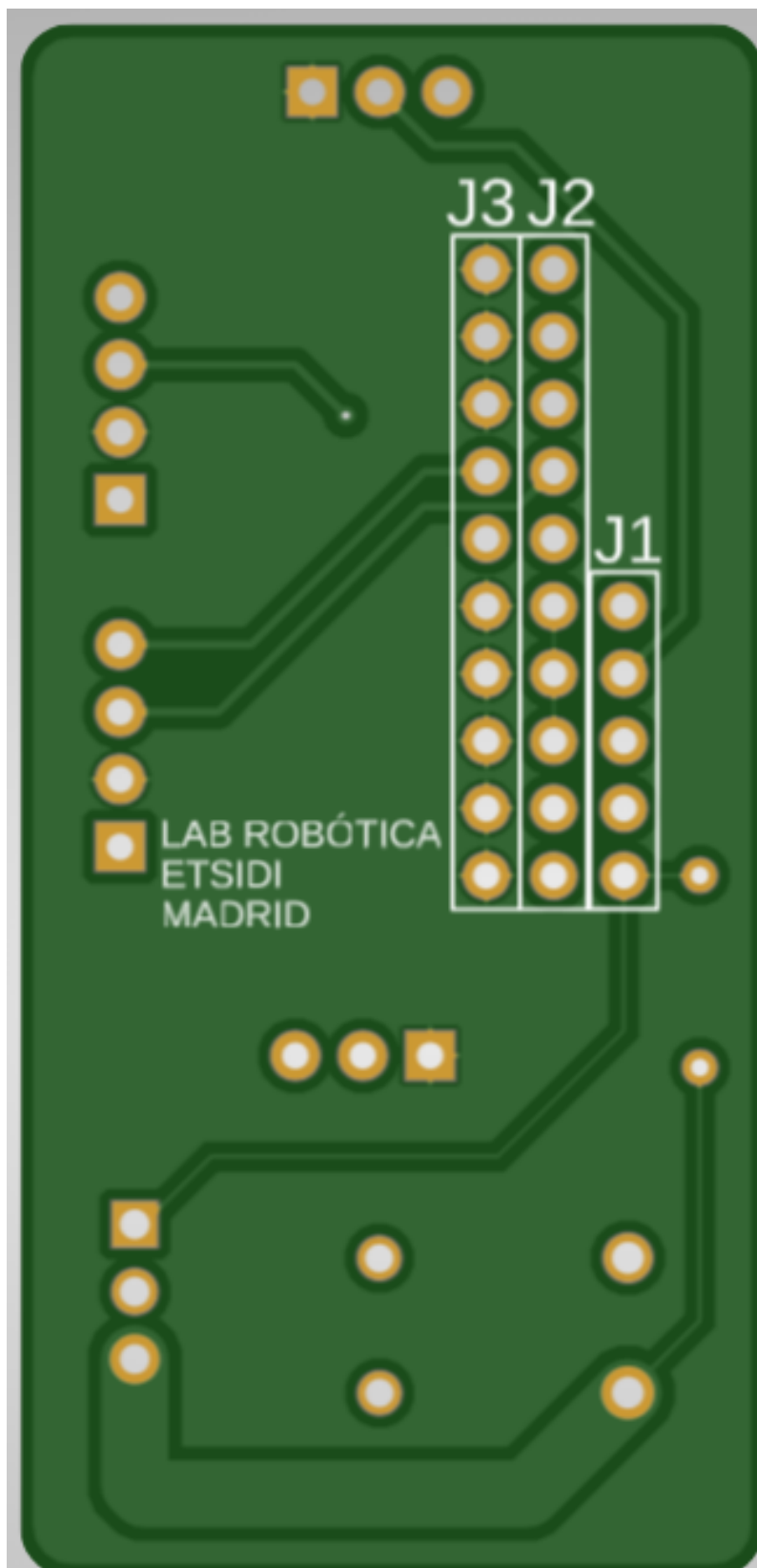


Figura A.3: Vista de la PCB completa.

Apéndice B

Anexo 2: Diseño de Piezas Mecánicas

Este anexo contiene los planos de las piezas mecánicas diseñadas específicamente para este proyecto.

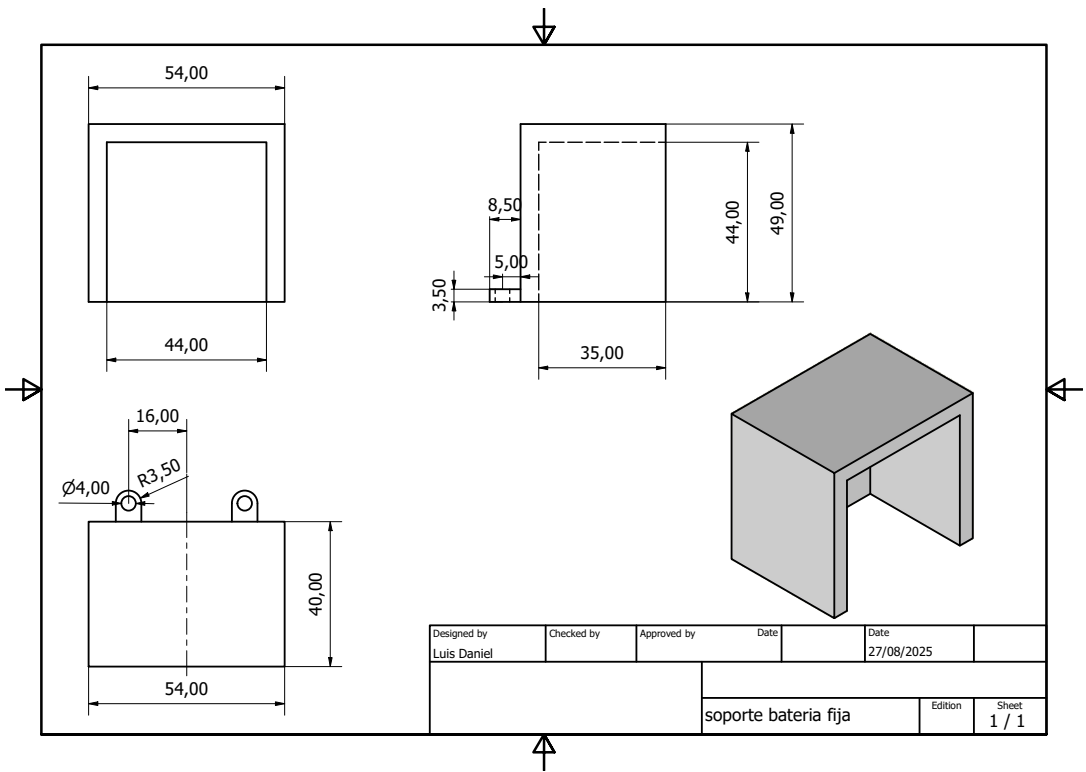


Figura B.1: Plano del soporte para la batería.

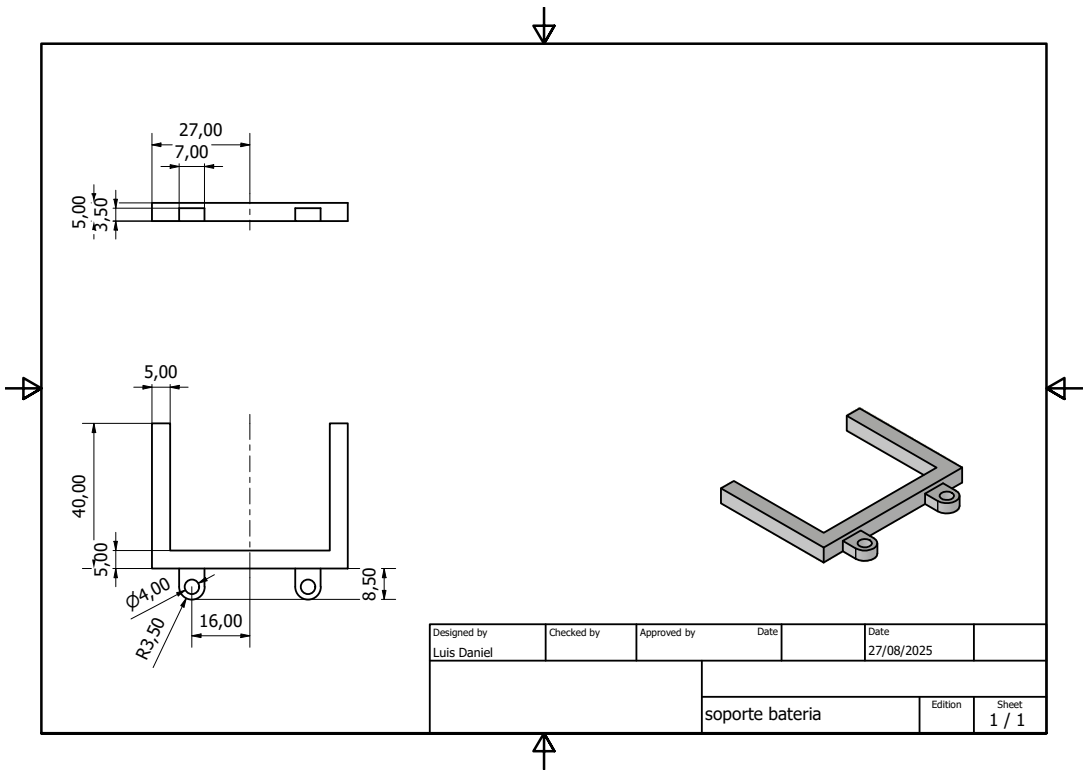


Figura B.2: Plano del soporte para la batería.

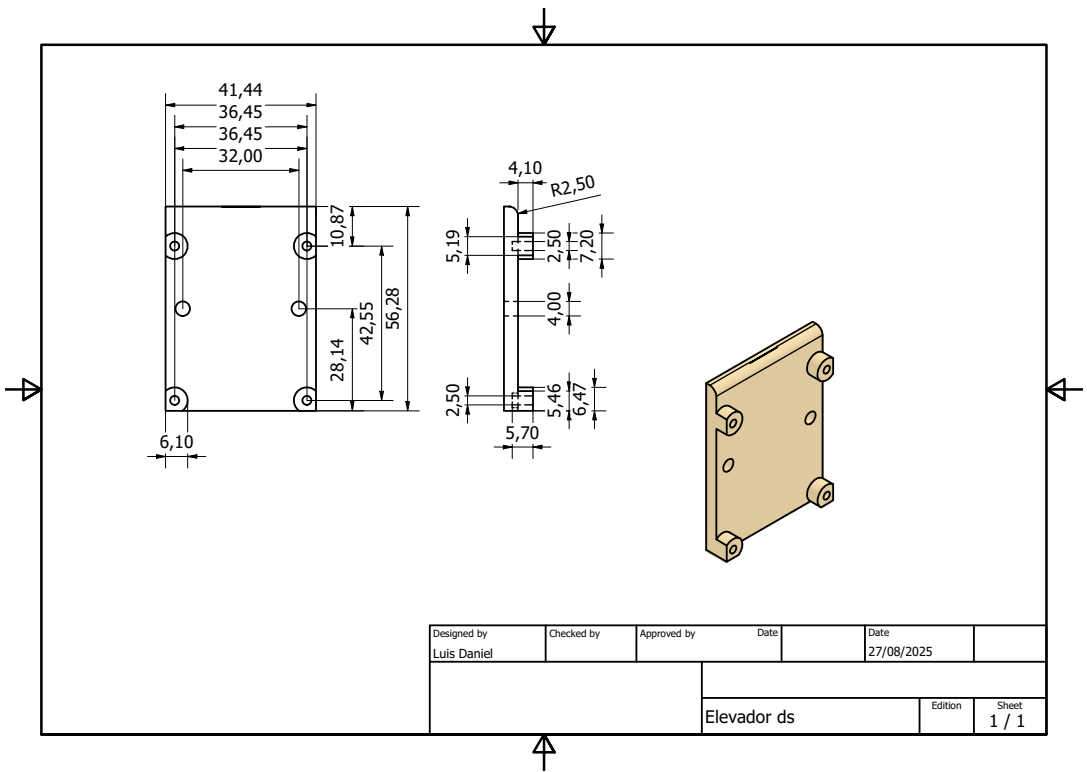


Figura B.3: Plano de la base de montaje para la RoboClaw

Bibliografía

- [1] Adam Stambler et al. witmotion_ros - ros 2 drivers for witmotion imu sensors. https://github.com/ElettraSciComp/witmotion_IMU_ros, 2024. Accedido: [16/08/2025]. Repositorio oficial del driver ROS 2 para sensores WitMotion.
- [2] Amphenol FCI. Conector para pcb, cabezal con clavija, paso 2.54mm, 10 vías, 1 fila, chapado en oro. <https://es.rs-online.com/web/p/conectores-hembra-para-pcb/2053161>, 2024. Accedido: 2024-10-27.
- [3] Amphenol FCI. Conector para pcb, cabezal con clavija, paso 2.54mm, 5 vías, 1 fila, chapado en oro. <https://es.rs-online.com/web/p/conectores-hembra-para-pcb/2053155>, 2024. Accedido: 2024-10-27.
- [4] Basic Micro, Inc. Roboclaw series user manual (revision 5.7). https://downloads.basicmicro.com/docs/roboclaw_user_manual.pdf, 2015. Manual de usuario de las controladoras RoboClaw.
- [5] Basic Micro, Inc. Roboclaw 2x7a motor controller. https://www.basicmicro.com/Roboclaw-2x7A-Motor-Controller_p_55.html, 2024. Accedido: 2024-10-27. Especificaciones técnicas del controlador de motor RoboClaw 2x7A.
- [6] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, Jose Neira, Ian Reid, and John J Leonard. Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. *IEEE Transactions on Robotics*, 32(6):1309–1332, 2016.
- [7] Carlos Campos, Richard Elvira, Juan J Gómez Rodríguez, José M M Montiel, and Juan D Tardós. ORB-SLAM3: An accurate open-source library for visual, visual-inertial and multi-map SLAM. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021.
- [8] Michele Colledanchise and Petter Ögren. Behavior trees in robotics and ai: An introduction. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1–2. IEEE, 2018. Una introducción conceptual a los Árboles de Comportamiento en robótica.
- [9] Electronilab. Raspberry pi 5 - 4gb ram, 2024.
- [10] Tully Foote. tf: The transform library. In *ROS: the Robot Operating System*, 2013. Aunque se refiere a tf en ROS 1, es la referencia conceptual para el sistema de transformaciones que evolucionó a tf2.
- [11] Dieter Fox. KLD-sampling: Adaptive particle filters. In *Advances in Neural Information Processing Systems 14 (NIPS)*, pages 713–720, 2001. Describe el muestreo adaptativo KLD, una técnica clave para la eficiencia de AMCL.
- [12] Dieter Fox, Wolfram Burgard, and Sebastian Thrun. The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, 4(1):23–33, 1997.
- [13] Gens ace / Tattu. Tattu 6000mah 14.8v 25c 4s1p lipo battery pack. <https://www.amazon.es/Gens-ace-Tattu-Multicopteros-Helic%C3%B3pteros/dp/B017GRB82S>, 2024. Accedido: 2024-10-27. Especificaciones de la batería LiPo utilizada.

- [14] GoBILDA. *5203 Series Yellow Jacket Planetary Gear Motor*. GoBILDA, 2024. Consultado el 3 de julio de 2025.
- [15] goBILDA (Base 10 Assets, LLC). Recon chassis kit. <https://www.gobilda.com/recon-chassis/>, 2025. Imagen utilizada en el Trabajo de Fin de Grado.
- [16] Giorgio Grisetti, Rainer Kummerle, Cyrill Stachniss, and Wolfram Burgard. A tutorial on graph-based SLAM. *IEEE Intelligent Transportation Systems Magazine*, 2(4):31–43, 2010.
- [17] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. El artículo seminal que introduce el algoritmo A*.
- [18] Austin Hughes and Bill Drury. *Electric Motors and Drives: Fundamentals, Types and Applications*. Newnes, 4th edition, 2019.
- [19] Infineon Technologies / International Rectifier. Datasheet irlb8721pbf - hexfet power mosfet. Hoja de datos del MOSFET de potencia IRLB8721PbF, 2009. Accedido: 2024-10-27.
- [20] JST. Conector para pcb, cabezal con clavija, paso 2.0mm, 3 vías, 1 fila, chapado en estaño. <https://es.rs-online.com/web/p/conectores-macho-para-pcb/8201557P>, 2024. Accedido: 2024-10-27.
- [21] JST. Conector para pcb, cabezal con clavija, paso 2.0mm, 4 vías, 1 fila, chapado en estaño. <https://es.rs-online.com/web/p/conectores-macho-para-pcb/8201551P>, 2024. Accedido: 2024-10-27.
- [22] Christian Kerl, Jürgen Sturm, and Daniel Cremers. Robust odometry estimation for rgb-d cameras. *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3748–3754, 2013.
- [23] Krzysztof Kozłowski and Pazderski. Modeling and control of a 4-wheel skid-steering mobile robot: From theory to practice. In *Budapest Tech Jubilee Conference*, 2004. Consultado en línea el 27 de abril de 2015.
- [24] Rainer Kümmerle, Giorgio Grisetti, Hauke Strasdat, Kurt Konolige, and Wolfram Burgard. g2o: A general framework for graph optimization. In *2011 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3607–3613. IEEE, 2011.
- [25] Mathieu Labbé and François Michaud. RTAB-Map as an open-source LiDAR and visual SLAM library for large-scale and long-term online operation. *Journal of Field Robotics*, 36(2):416–446, 2019.
- [26] Steven M. LaValle. *Planning algorithms*. Cambridge University Press, 2006.
- [27] Luxonis. Depthai api documentation. <https://docs.luxonis.com/projects/api/en/latest/>, 2024. Accedido: 2024-10-27. Documentación de la API para crear pipelines de visión en dispositivos OAK.
- [28] Luxonis. depthai-ros - ROS driver for DepthAI. <https://github.com/luxonis/depthai-ros>, 2024. Accedido: 2024-10-27. Repositorio y documentación del driver ROS para cámaras OAK.
- [29] Luxonis. OAK-D-Lite. <https://docs.luxonis.com/projects/hardware/en/latest/pages/DM9095.html>, 2024. Accedido: 2024-10-27.
- [30] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022. Artículo de referencia sobre el diseño y la arquitectura de ROS 2.
- [31] Miguel Hernando et al. teleop_twist_qt - qt based teleop for twist messages. https://github.com/mhernando/teleop_twist_qt, 2024. Paquete base para la teleoperación, modificado para cinemática diferencial.

- [32] Open Source Robotics Foundation (OSRF) et al. costmap_2d - ROS wiki. http://wiki.ros.org/costmap_2d, 2024. Documentación conceptual sobre los mapas de costo, cuyo principio se aplica en Nav2.
- [33] PySource. Install and run oak-d lite on windows - depthai, 2022. Consultado el 3 de julio de 2025.
- [34] ROS 2 Navigation Working Group et al. Nav2 Documentation. <https://navigation.ros.org/>, 2024. Accedido: [15/08/2025]. Documentación oficial del stack de navegación Nav2 para ROS 2.
- [35] ROS Perception Contributors. depthimage_to_laserscan - ros wiki. http://wiki.ros.org/depthimage_to_laserscan, 2024. Documentación del paquete para convertir imágenes de profundidad a LaserScan.
- [36] Max Schwarz, Tobias Rodehutsors, Michael Schreiber, and Sven Behnke. Hybrid driving-stepping locomotion with the wheeled-legged robot momaro. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2016.
- [37] Roland Siegwart and Illah R. Nourbakhsh. *Introduction to Autonomous Mobile Robots*. MIT Press, Cambridge, Massachusetts, 2011.
- [38] R Silva Ortigoza, JR García Sánchez, VR Barrientos Sotelo, MA Molina Vilchis, VM Hernández Guzmán, and G Silva Ortigoza. Una panorámica de los robots móviles. *Revista Electrónica de Estudios Telemáticos*, 6(3), 2007.
- [39] Josef Sivic and Andrew Zisserman. Video Google: A text retrieval approach to object matching in videos. In *Proceedings of the Ninth IEEE International Conference on Computer Vision (ICCV)*, volume 2, pages 1470–1477. IEEE, 2003.
- [40] Sebastian Thrun. Probabilistic algorithms in robotics. *AI Magazine*, 21(4):93, 2000. Accedido: 2024-10-27.
- [41] TP-Link. Tl-wr802n - 300mbps wireless n nano router, 2024. Consultado el 4 de julio de 2025.
- [42] Vishay Sfernice. Datasheet lto 50 thick film power resistor. <https://es.rs-online.com/web/p/resistencias-de-montaje-en-orificio-pasante/1679624>, 2024. Accedido: 2024-10-27. Hoja de datos de la resistencia de disipación.
- [43] WitMotion Shenzhen Co., Ltd. Wt901c 9-axis digital inclinometer accelerometer. <https://www.wit-motion.com/proztsz/43.html>, 2025. Imagen utilizada en el Trabajo de Fin de Grado.
- [44] Yahboom. Robot raspberry pi 5 power supply board. <https://category.yahboom.net/products/power-board-pi5>, 2024. Accedido: 2024-10-27. Placa de expansión y regulación de voltaje para RPi5.